FreeRTOS (IDF)

[??]

This document provides information regarding the dual-core SMP implementation of FreeRTOS inside ESP-IDF. This document is split into the following sections:

Sections

- FreeRTOS (IDF)
  - Overview
  - Symmetric Multiprocessing
  - Tasks
  - SMP Scheduler
  - Critical Sections
  - Misc
  - Single-Core Mode
  - API Reference

Overview

The original FreeRTOS (hereinafter referred to as **Vanilla FreeRTOS**) is a compact and efficient real-time operating system supported on numerous single-core MCUs and SoCs. However, to support dual-core ESP targets, such as ESP32, ESP32-S3, and ESP32-P4, ESP-IDF provides a unique implementation of FreeRTOS with dual-core symmetric multiprocessing (SMP) capabilities (hereinafter referred to as **IDF FreeRTOS**).

IDF FreeRTOS source code is based on Vanilla FreeRTOS v10.5.1 but contains significant modifications to both kernel behavior and API in order to support dual-core SMP. However, IDF FreeRTOS can also be configured for single-core by enabling the CONFIG_FREERTOS_UNICORE option (see Single-Core Mode for more details).

> **Note**
>
> This document assumes that the reader has a requisite understanding of Vanilla FreeRTOS, i.e., its features, behavior, and API usage. Refer to the Vanilla FreeRTOS documentation for more details.

Symmetric Multiprocessing

Basic Concepts

Symmetric multiprocessing is a computing architecture where two or more identical CPU cores are connected to a single shared main memory and controlled by a single operating system. In general, an SMP system:

- has multiple cores running independently. Each core has its own register file, interrupts, and interrupt handling.
- presents an identical view of memory to each core. Thus, a piece of code that accesses a particular memory address has the same effect regardless of which core it runs on.

The main advantages of an SMP system compared to single-core or asymmetric multiprocessing systems are that:

- the presence of multiple cores allows for multiple hardware threads, thus increasing overall processing throughput.
- having symmetric memory means that threads can switch cores during execution. This, in general, can lead to better CPU utilization.

Although an SMP system allows threads to switch cores, there are scenarios where a thread must/should only run on a particular core. Therefore, threads in an SMP system also have a core affinity that specifies which particular core the thread is allowed to run on.

- A thread that is pinned to a particular core is only able to run on that core.
- A thread that is unpinned will be allowed to switch between cores during execution instead of being pinned to a particular core.

SMP on an ESP Target

ESP targets such as ESP32, ESP32-S3, ESP32-P4 and ESP32-H4 are dual-core SMP SoCs. These targets have the following hardware features that make them SMP-capable:

- Two identical cores are known as Core 0 and Core 1. This means that the execution of a piece of code is identical regardless of which core it runs on.
- Symmetric memory (with some small exceptions).
  - If multiple cores access the same memory address simultaneously, their access will be serialized by the memory bus.
  - True atomic access to the same memory address is achieved via an atomic compare-and-swap instruction provided by the ISA.
- Cross-core interrupts that allow one core to trigger an interrupt on the other core. This allows cores to signal events to each other (such as requesting a context switch on the other core).

Note

Within ESP-IDF, Core 0 and Core 1 are sometimes referred to as `PRO_CPU` and `APP_CPU` respectively. The aliases exist in ESP-IDF as they reflect how typical ESP-IDF applications utilize the two cores. Typically, the tasks responsible for handling protocol related processing such as Wi-Fi or Bluetooth are pinned to Core 0 (thus the name `PRO_CPU`), where as the tasks handling the remainder of the application are pinned to Core 1, (thus the name `APP_CPU`).

Tasks

Creation

Vanilla FreeRTOS provides the following functions to create a task:

- `xTaskCreate()` creates a task. The task's memory is dynamically allocated.
- `xTaskCreateStatic()` creates a task. The task's memory is statically allocated, i.e., provided by the user.

However, in an SMP system, tasks need to be assigned a particular affinity. Therefore, ESP-IDF provides a `...PinnedToCore()` version of Vanilla FreeRTOS's task creation functions:

- `xTaskCreatePinnedToCore()` creates a task with a particular core affinity. The task's memory is dynamically allocated.
- `xTaskCreateStaticPinnedToCore()` creates a task with a particular core affinity. The task's memory is statically allocated, i.e., provided by the user.

The `...PinnedToCore()` versions of the task creation function API differ from their vanilla counterparts by having an extra `xCoreID` parameter that is used to specify the created task's core affinity. The valid values for core affinity are:

- `0`, which pins the created task to Core 0
- `1`, which pins the created task to Core 1
- `tskNO_AFFINITY`, which allows the task to be run on both cores

Note that IDF FreeRTOS still supports the vanilla versions of the task creation functions. However, these standard functions have been modified to essentially invoke their respective `...PinnedToCore()` counterparts while setting the core affinity to `tskNO_AFFINITY`.

Note

IDF FreeRTOS also changes the units of `ulStackDepth` in the task creation functions. Task stack sizes in Vanilla FreeRTOS are specified in a number of words, whereas in IDF FreeRTOS, the task stack sizes are specified in bytes.

Execution

The anatomy of a task in IDF FreeRTOS is the same as in Vanilla FreeRTOS. More specifically, IDF FreeRTOS tasks:

- Can only be in one of the following states: Running, Ready, Blocked, or Suspended.
- Task functions are typically implemented as an infinite loop.
- Task functions should never return.

Deletion

Task deletion in Vanilla FreeRTOS is called via **`vTaskDelete()`**. The function allows deletion of another task or the currently running task if the provided task handle is `NULL`. The actual freeing of the task's memory is sometimes delegated to the idle task if the task being deleted is the currently running task.

IDF FreeRTOS provides the same **`vTaskDelete()`** function. However, due to the dual-core nature, there are some behavioral differences when calling **`vTaskDelete()`** in IDF FreeRTOS:

- When deleting a task that is currently running on the other core, a yield is triggered on the other core, and the task's memory is freed by one of the idle tasks.
- A deleted task's memory is freed immediately if it is not running on either core.

Please avoid deleting a task that is running on another core as it is difficult to determine what the task is performing, which may lead to unpredictable behavior such as:

- Deleting a task that is holding a mutex.
- Deleting a task that has yet to free memory it previously allocated.

Where possible, please design your own application so that when calling **`vTaskDelete()`**, the deleted task is in a known state. For example:

- Tasks self-deleting via `vTaskDelete(NULL)` when their execution is complete and have also cleaned up all resources used within the task.
- Tasks placing themselves in the suspend state via **`vTaskSuspend()`** before being deleted by another task.

SMP Scheduler

The Vanilla FreeRTOS scheduler is best described as a **fixed priority preemptive scheduler with time slicing** meaning that:

- Each task is given a constant priority upon creation. The scheduler executes the highest priority ready-state task.
- The scheduler can switch execution to another task without the cooperation of the currently running task.
- The scheduler periodically switches execution between ready-state tasks of the same priority in a round-robin fashion. Time slicing is governed by a tick interrupt.

The IDF FreeRTOS scheduler supports the same scheduling features, i.e., Fixed Priority, Preemption, and Time Slicing, albeit with some small behavioral differences.

Fixed Priority

In Vanilla FreeRTOS, when the scheduler selects a new task to run, it always selects the current highest priority ready-state task. In IDF FreeRTOS, each core independently schedules tasks to run. When a particular core selects a task, the core will select the highest priority ready-state task that can be run by the core. A task can be run by the core if:

- The task has a compatible affinity, i.e., is either pinned to that core or is unpinned.
- The task is not currently being run by another core.

However, please do not assume that the two highest priority ready-state tasks are always run by the scheduler, as a task's core affinity must also be accounted for. For example, given the following tasks:

- Task A of priority 10 pinned to Core 0
- Task B of priority 9 pinned to Core 0
- Task C of priority 8 pinned to Core 1

The resulting schedule will have Task A running on Core 0 and Task C running on Core 1. Task B is not run even though it is the second-highest priority task.

Preemption

In Vanilla FreeRTOS, the scheduler can preempt the currently running task if a higher priority task becomes ready to execute. Likewise in IDF FreeRTOS, each core can be individually preempted by the scheduler if the scheduler determines that a higher-priority task can run on that core.

However, there are some instances where a higher-priority task that becomes ready can be run on multiple cores. In this case, the scheduler only preempts one core. The scheduler always gives preference to the current core when multiple cores can be preempted. In other words, if the higher priority ready task is unpinned and has a higher priority than the current priority of both cores, the scheduler will always choose to preempt the current core. For example, given the following tasks:

- Task A of priority 8 currently running on Core 0
- Task B of priority 9 currently running on Core 1
- Task C of priority 10 that is unpinned and was unblocked by Task B

The resulting schedule will have Task A running on Core 0 and Task C preempting Task B given that the scheduler always gives preference to the current core.

Time Slicing

The Vanilla FreeRTOS scheduler implements time slicing, which means that if the current highest ready priority contains multiple ready tasks, the scheduler will switch between those tasks periodically in a round-robin fashion.

However, in IDF FreeRTOS, it is not possible to implement perfect Round Robin time slicing due to the fact that a particular task may not be able to run on a particular core due to the following reasons:

- The task is pinned to another core.
- For unpinned tasks, the task is already being run by another core.

Therefore, when a core searches the ready-state task list for a task to run, the core may need to skip over a few tasks in the same priority list or drop to a lower priority in order to find a ready-state task that the core can run.

The IDF FreeRTOS scheduler implements a Best Effort Round Robin time slicing for ready-state tasks of the same priority by ensuring that tasks that have been selected to run are placed at the back of the list, thus giving unselected tasks a higher priority on the next scheduling iteration (i.e., the next tick interrupt or yield).

The following example demonstrates the Best Effort Round Robin time slicing in action. Assume that:

- There are four ready-state tasks of the same priority `AX`, `B0`, `C1`, and `D1` where:
  - The priority is the current highest priority with ready-state .
  - The first character represents the task's name, i.e., `A`, `B`, `C`, `D`.
  - The second character represents the task's core pinning, and `X` means unpinned.
- The task list is always searched from the head.

1. Starting state. None of the ready-state tasks have been selected to run.

```
Head [ AX , B0 , C1 , D0 ] Tail
```

2. Core 0 has a tick interrupt and searches for a task to run. Task A is selected and moved to the back of the list.

```
Core 0 ─┐
        ▼
Head [ AX , B0 , C1 , D0 ] Tail

                      [0]
Head [ B0 , C1 , D0 , AX ] Tail
```

3. Core 1 has a tick interrupt and searches for a task to run. Task B cannot be run due to incompatible affinity, so Core 1 skips to Task C. Task C is selected and moved to the back of the list.

```
Core 1 ──────┐
             ▼        [0]
Head [ B0 , C1 , D0 , AX ] Tail

                [0]  [1]
Head [ B0 , D0 , AX , C1 ] Tail
```

4. Core 0 has another tick interrupt and searches for a task to run. Task B is selected and moved to the back of the list.

```
Core 0 ─┐
        ▼             [1]
Head [ B0 , D0 , AX , C1 ] Tail

                [1]  [0]
Head [ D0 , AX , C1 , B0 ] Tail
```

5. Core 1 has another tick and searches for a task to run. Task D cannot be run due to incompatible affinity, so Core 1 skips to Task A. Task A is selected and moved to the back of the list.

```
Core 1 ──────┐
             ▼        [0]
Head [ D0 , AX , C1 , B0 ] Tail

                 [0]  [1]
Head [ D0 , C1 , B0 , AX ] Tail
```

The implications to users regarding the Best Effort Round Robin time slicing:

- Users cannot expect multiple ready-state tasks of the same priority to run sequentially as is the case in Vanilla FreeRTOS. As demonstrated in the example above, a core may need to skip over tasks.
- However, given enough ticks, a task will eventually be given some processing time.
- If a core cannot find a task runnable task at the highest ready-state priority, it will drop to a lower priority to search for tasks.
- To achieve ideal round-robin time slicing, users should ensure that all tasks of a particular priority are pinned to the same core.

Tick Interrupts

Vanilla FreeRTOS requires that a periodic tick interrupt occurs. The tick interrupt is responsible for:

- Incrementing the scheduler's tick count
- Unblocking any blocked tasks that have timed out
- Checking if time slicing is required, i.e., triggering a context switch
- Executing the application tick hook

In IDF FreeRTOS, each core receives a periodic interrupt and independently runs the tick interrupt. The tick interrupts on each core are of the same period but can be out of phase. However, the tick responsibilities listed above are not run by all cores:

- Core 0 executes all of the tick interrupt responsibilities listed above
- Core 1 only checks for time slicing and executes the application tick hook

Note

Core 0 is solely responsible for keeping time in IDF FreeRTOS. Therefore, anything that prevents Core 0 from incrementing the tick count, such as suspending the scheduler on Core 0, will cause the entire scheduler's timekeeping to lag behind.

Idle Tasks

Vanilla FreeRTOS will implicitly create an idle task of priority 0 when the scheduler is started. The idle task runs when no other task is ready to run, and it has the following responsibilities:

- Freeing the memory of deleted tasks
- Executing the application idle hook

In IDF FreeRTOS, a separate pinned idle task is created for each core. The idle tasks on each core have the same responsibilities as their vanilla counterparts.

Scheduler Suspension

Vanilla FreeRTOS allows the scheduler to be suspended/resumed by calling `vTaskSuspendAll()` and `xTaskResumeAll()` respectively. While the scheduler is suspended:

- Task switching is disabled but interrupts are left enabled.
- Calling any blocking/yielding function is forbidden, and time slicing is disabled.
- The tick count is frozen, but the tick interrupt still occurs to execute the application tick hook.

On scheduler resumption, `xTaskResumeAll()` catches up all of the lost ticks and unblock any timed-out tasks.

In IDF FreeRTOS, suspending the scheduler across multiple cores is not possible. Therefore when `vTaskSuspendAll()` is called on a particular core (e.g., core A):

- Task switching is disabled only on core A but interrupts for core A are left enabled.
- Calling any blocking/yielding function on core A is forbidden. Time slicing is disabled on core A.
- If an interrupt on core A unblocks any tasks, tasks with affinity to core A will go into core A's own pending ready task list. Unpinned tasks or tasks with affinity to other cores can be scheduled on cores with the scheduler running.
- If the scheduler is suspended on all cores, tasks unblocked by an interrupt will be directed to the pending ready task lists of their pinned cores. For unpinned tasks, they will be placed in the pending ready list of the core where the interrupt occurred.
- If core A is on Core 0, the tick count is frozen, and a pended tick count is incremented instead. However, the tick interrupt will still occur in order to execute the application tick hook.

When `xTaskResumeAll()` is called on a particular core (e.g., core A):

- Any tasks added to core A's pending ready task list will be resumed.
- If core A is Core 0, the pended tick count is unwound to catch up with the lost ticks.

Warning

Given that scheduler suspension on IDF FreeRTOS only suspends scheduling on a particular core, scheduler suspension is **NOT** a valid method of ensuring mutual exclusion between tasks when accessing shared data. Users should use proper locking primitives such as mutexes or spinlocks if they require mutual exclusion.

Critical Sections

Disabling Interrupts

Vanilla FreeRTOS allows interrupts to be disabled and enabled by calling `taskDISABLE_INTERRUPTS` and `taskENABLE_INTERRUPTS` respectively. IDF FreeRTOS provides the same API. However, interrupts are only disabled or enabled on the current core.

Disabling interrupts is a valid method of achieving mutual exclusion in Vanilla FreeRTOS (and single-core systems in general). **However, in an SMP system, disabling interrupts is not a valid method of ensuring mutual exclusion**. Critical sections that utilize a spinlock should be used instead.

API Changes

Vanilla FreeRTOS implements critical sections by disabling interrupts, which prevents preemptive context switches and the servicing of ISRs during a critical section. Thus a task/ISR that enters a critical section is guaranteed to be the sole entity to access a shared resource. Critical sections in Vanilla FreeRTOS have the following API:

- `taskENTER_CRITICAL()` enters a critical section by disabling interrupts
- `taskEXIT_CRITICAL()` exits a critical section by reenabling interrupts
- `taskENTER_CRITICAL_FROM_ISR()` enters a critical section from an ISR by disabling interrupt nesting
- `taskEXIT_CRITICAL_FROM_ISR()` exits a critical section from an ISR by reenabling interrupt nesting

However, in an SMP system, merely disabling interrupts does not constitute a critical section as the presence of other cores means that a shared resource can still be concurrently accessed. Therefore, critical sections in IDF FreeRTOS are implemented using spinlocks. To accommodate the spinlocks, the IDF FreeRTOS critical section APIs contain an additional spinlock parameter as shown below:

- Spinlocks are of `portMUX_TYPE` (**not to be confused to FreeRTOS mutexes**)
- `taskENTER_CRITICAL(&spinlock)` enters a critical from a task context
- `taskEXIT_CRITICAL(&spinlock)` exits a critical section from a task context

- `taskENTER_CRITICAL_ISR(&spinlock)` enters a critical section from an interrupt context
- `taskEXIT_CRITICAL_ISR(&spinlock)` exits a critical section from an interrupt context

> **Note**
>
> The critical section API can be called recursively, i.e., nested critical sections. Entering a critical section multiple times recursively is valid so long as the critical section is exited the same number of times it was entered. However, given that critical sections can target different spinlocks, users should take care to avoid deadlocking when entering critical sections recursively.

Spinlocks can be allocated statically or dynamically. As such, macros are provided for both static and dynamic initialization of spinlocks, as demonstrated by the following code snippets.

- Allocating a static spinlock and initializing it using `portMUX_INITIALIZER_UNLOCKED`:

```
// Statically allocate and initialize the spinlock
static portMUX_TYPE my_spinlock = portMUX_INITIALIZER_UNLOCKED;

void some_function(void)
{
    taskENTER_CRITICAL(&my_spinlock);
    // We are now in a critical section
    taskEXIT_CRITICAL(&my_spinlock);
}
```

- Allocating a dynamic spinlock and initializing it using `portMUX_INITIALIZE()`:

```
// Allocate the spinlock dynamically
portMUX_TYPE *my_spinlock = malloc(sizeof(portMUX_TYPE));
// Initialize the spinlock dynamically
portMUX_INITIALIZE(my_spinlock);

...

taskENTER_CRITICAL(my_spinlock);
// Access the resource
taskEXIT_CRITICAL(my_spinlock);
```

Implementation

In IDF FreeRTOS, the process of a particular core entering and exiting a critical section is as follows:

- For `taskENTER_CRITICAL(&spinlock)` or `taskENTER_CRITICAL_ISR(&spinlock)`

    1. The core disables its interrupts or interrupt nesting up to `configMAX_SYSCALL_INTERRUPT_PRIORITY`.
    2. The core then spins on the spinlock using an atomic compare-and-set instruction until it acquires the lock. A lock is acquired when the core is able to set the lock's owner value to the core's ID.
    3. Once the spinlock is acquired, the function returns. The remainder of the critical section runs with interrupts or interrupt nesting disabled.
- For `taskEXIT_CRITICAL(&spinlock)` or `taskEXIT_CRITICAL_ISR(&spinlock)`
    1. The core releases the spinlock by clearing the spinlock's owner value.
    2. The core re-enables interrupts or interrupt nesting.

Restrictions and Considerations

Given that interrupts (or interrupt nesting) are disabled during a critical section, there are multiple restrictions regarding what can be done within critical sections. During a critical section, users should keep the following restrictions and considerations in mind:

- Critical sections should be kept as short as possible
    - The longer the critical section lasts, the longer a pending interrupt can be delayed.
    - A typical critical section should only access a few data structures and/or hardware registers.
    - If possible, defer as much processing and/or event handling to the outside of critical sections.
- FreeRTOS API should not be called from within a critical section
- Users should never call any blocking or yielding functions within a critical section

Misc

Floating Point Usage

Usually, when a context switch occurs:

- the current state of a core's registers are saved to the stack of the task being switched out
- the previously saved state of the core's registers is loaded from the stack of the task being switched in

However, IDF FreeRTOS implements Lazy Context Switching for the Floating Point Unit (FPU) registers of a core. In other words, when a context switch occurs on a particular core (e.g., Core 0), the state of the core's FPU registers is not immediately saved to the stack of the task getting switched out (e.g., Task A). The FPU registers are left untouched until:

- A different task (e.g., Task B) runs on the same core and uses FPU. This will trigger an exception that saves the FPU registers to Task A's stack.

- Task A gets scheduled to the same core and continues execution. Saving and restoring the FPU registers is not necessary in this case.

However, given that tasks can be unpinned and thus can be scheduled on different cores (e.g., Task A switches to Core 1), it is unfeasible to copy and restore the FPU registers across cores. Therefore, when a task utilizes FPU by using a `float` type in its call flow, IDF FreeRTOS will automatically pin the task to the current core it is running on. This ensures that all tasks that use FPU are always pinned to a particular core.

Furthermore, IDF FreeRTOS by default does not support the usage of FPU within an interrupt context given that the FPU register state is tied to a particular task.

Note

Users that require the use of the `float` type in an ISR routine should refer to the CONFIG_FREERTOS_FPU_IN_ISR configuration option.

Note

ESP targets that contain an FPU do not support hardware acceleration for double precision floating point arithmetic (`double`). Instead, `double` is implemented via software, hence the behavioral restrictions regarding the `float` type do not apply to `double`. Note that due to the lack of hardware acceleration, `double` operations may consume significantly more CPU time in comparison to `float`.

Single-Core Mode

Although IDF FreeRTOS is modified for dual-core SMP, IDF FreeRTOS can also be built for single-core by enabling the CONFIG_FREERTOS_UNICORE option.

For single-core targets (such as ESP32-S2 and ESP32-C3), the CONFIG_FREERTOS_UNICORE option is always enabled. For multi-core targets (such as ESP32 and ESP32-S3), CONFIG_FREERTOS_UNICORE can also be set, but will result in the application only running Core 0.

When building in single-core mode, IDF FreeRTOS is designed to be identical to Vanilla FreeRTOS, thus all aforementioned SMP changes to kernel behavior are removed. As a result, building IDF FreeRTOS in single-core mode has the following characteristics:

- All operations performed by the kernel inside critical sections are now deterministic (i.e., no walking of linked lists inside critical sections).
- Vanilla FreeRTOS scheduling algorithm is restored (including perfect Round Robin time slicing).

- All SMP specific data is removed from single-core builds.

SMP APIs can still be called in single-core mode. These APIs remain exposed to allow source code to be built for single-core and multi-core, without needing to call a different set of APIs. However, SMP APIs will not exhibit any SMP behavior in single-core mode, thus becoming equivalent to their single-core counterparts. For example:

- any `...ForCore(..., BaseType_t xCoreID)` SMP API will only accept `0` as a valid value for `xCoreID`.
- `...PinnedToCore()` task creation APIs will simply ignore the `xCoreID` core affinity argument.
- Critical section APIs will still require a spinlock argument, but no spinlock will be taken and critical sections revert to simply disabling/enabling interrupts.

API Reference

This section introduces FreeRTOS types, functions, and macros. It is automatically generated from FreeRTOS header files.

Task API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/task.h
- This header file can be included with:

```
#include "freertos/task.h"
```

Functions

---

static inline **BaseType_t xTaskCreate(TaskFunction_t pxTaskCode, *const* char **const* pcName, *const* configSTACK_DEPTH_TYPE usStackDepth, void **const* pvParameters, UBaseType_t uxPriority, TaskHandle_t **const* pxCreatedTask)**

Create a new task and add it to the list of tasks that are ready to run.

Internally, within the FreeRTOS implementation, tasks use two blocks of memory. The first block is used to hold the task's data structures. The second block is used by the task as its stack. If a task is created using xTaskCreate() then both blocks of memory are automatically dynamically allocated inside the xTaskCreate() function. (see https://www.FreeRTOS.org/a00111.html). If a task is created using xTaskCreateStatic() then the application writer must provide the required memory. xTaskCreateStatic() therefore allows a task to be created without using any dynamic memory allocation.

See xTaskCreateStatic() for a version that does not use any dynamic memory allocation.

xTaskCreate() can only be used to create a task that has unrestricted access to the entire microcontroller memory map. Systems that include MPU support can alternatively create an MPU constrained task using xTaskCreateRestricted().

Example usage:

```
// Task to be created.
void vTaskCode( void * pvParameters )
{
for( ;; )
  {
// Task code goes here.
  }
}

// Function that creates a task.
void vOtherFunction( void )
{
static uint8_t ucParameterToPass;
TaskHandle_t xHandle = NULL;

// Create the task, storing the handle.  Note that the passed parameter
ucParameterToPass
// must exist for the lifetime of the task, so in this case is declared static.  If
it was just an
// an automatic stack variable it might no longer exist, or at least have been
corrupted, by the time
// the new task attempts to access it.
  xTaskCreate( vTaskCode, "NAME", STACK_SIZE, &ucParameterToPass, tskIDLE_PRIORITY,
&xHandle );
  configASSERT( xHandle );

// Use the handle to delete the task.
if( xHandle != NULL )
  {
     vTaskDelete( xHandle );
  }
}
```

Note

If configNUMBER_OF_CORES > 1, this function will create an unpinned task (see tskNO_AFFINITY for more details).

Note

If program uses thread local variables (ones specified with "__thread" keyword) then storage for them will be allocated on the task's stack.

Parameters:

- **pxTaskCode** -- Pointer to the task entry function. Tasks must be implemented to never return (i.e. continuous loop).

- **pcName** -- A descriptive name for the task. This is mainly used to facilitate debugging. Max length defined by configMAX_TASK_NAME_LEN - default is 16.
- **usStackDepth** -- The size of the task stack specified as the NUMBER OF BYTES. Note that this differs from vanilla FreeRTOS.
- **pvParameters** -- Pointer that will be used as the parameter for the task being created.
- **uxPriority** -- The priority at which the task should run. Systems that include MPU support can optionally create tasks in a privileged (system) mode by setting bit portPRIVILEGE_BIT of the priority parameter. For example, to create a privileged task at priority 2 the uxPriority parameter should be set to (2 | portPRIVILEGE_BIT).
- **pxCreatedTask** -- Used to pass back a handle by which the created task can be referenced.

Returns: pdPASS if the task was successfully created and added to a ready list, otherwise an error code defined in the file projdefs.h

---

static inline TaskHandle_t **`xTaskCreateStatic`**(TaskFunction_t pxTaskCode, ***const*** char ****const*** pcName, ***const*** uint32_t ulStackDepth, void ****const*** pvParameters, UBaseType_t uxPriority, StackType_t ****const*** puxStackBuffer, StaticTask_t ****const*** pxTaskBuffer)

Create a new task and add it to the list of tasks that are ready to run.

Internally, within the FreeRTOS implementation, tasks use two blocks of memory. The first block is used to hold the task's data structures. The second block is used by the task as its stack. If a task is created using xTaskCreate() then both blocks of memory are automatically dynamically allocated inside the xTaskCreate() function. (see https://www.FreeRTOS.org/a00111.html). If a task is created using xTaskCreateStatic() then the application writer must provide the required memory. xTaskCreateStatic() therefore allows a task to be created without using any dynamic memory allocation.

Example usage:

```
// Dimensions the buffer that the task being created will use as its stack.
// NOTE:  This is the number of bytes the stack will hold, not the number of
// words as found in vanilla FreeRTOS.
#define STACK_SIZE 200

// Structure that will hold the TCB of the task being created.
   StaticTask_t xTaskBuffer;

// Buffer that the task being created will use as its stack.  Note this is
// an array of StackType_t variables.  The size of StackType_t is dependent on
// the RTOS port.
   StackType_t xStack[ STACK_SIZE ];

// Function that implements the task being created.
void vTaskCode( void * pvParameters )
   {
// The parameter value is expected to be 1 as 1 is passed in the
// pvParameters value in the call to xTaskCreateStatic().
       configASSERT( ( uint32_t ) pvParameters == 1UL );

for( ;; )
       {
// Task code goes here.
       }
   }

// Function that creates a task.
void vOtherFunction( void )
   {
       TaskHandle_t xHandle = NULL;

// Create the task without using any dynamic memory allocation.
       xHandle = xTaskCreateStatic(
                     vTaskCode,       // Function that implements the task.
"NAME",          // Text name for the task.
                     STACK_SIZE,      // Stack size in bytes.
                     ( void * ) 1,    // Parameter passed into the task.
                     tskIDLE_PRIORITY,// Priority at which the task is created.
                     xStack,          // Array to use as the task's stack.
                     &xTaskBuffer );  // Variable to hold the task's data
structure.

// puxStackBuffer and pxTaskBuffer were not NULL, so the task will have
// been created, and xHandle will be the task's handle.  Use the handle
// to suspend the task.
       vTaskSuspend( xHandle );
   }
```

Note

If configNUMBER_OF_CORES > 1, this function will create an unpinned task (see tskNO_AFFINITY for more details).

Note

If program uses thread local variables (ones specified with "__thread" keyword) then storage for them will be allocated on the task's stack.

Parameters:

- **pxTaskCode** -- Pointer to the task entry function. Tasks must be implemented to never return (i.e. continuous loop).
- **pcName** -- A descriptive name for the task. This is mainly used to facilitate debugging. The maximum length of the string is defined by configMAX_TASK_NAME_LEN in FreeRTOSConfig.h.
- **ulStackDepth** -- The size of the task stack specified as the NUMBER OF BYTES. Note that this differs from vanilla FreeRTOS.
- **pvParameters** -- Pointer that will be used as the parameter for the task being created.
- **uxPriority** -- The priority at which the task will run.
- **puxStackBuffer** -- Must point to a StackType_t array that has at least ulStackDepth indexes - the array will then be used as the task's stack, removing the need for the stack to be allocated dynamically.
- **pxTaskBuffer** -- Must point to a variable of type StaticTask_t, which will then be used to hold the task's data structures, removing the need for the memory to be allocated dynamically.

Returns: If neither puxStackBuffer nor pxTaskBuffer are NULL, then the task will be created and a handle to the created task is returned. If either puxStackBuffer or pxTaskBuffer are NULL then the task will not be created and NULL is returned.

---

void **vTaskAllocateMPURegions**(TaskHandle_t xTask, *const* MemoryRegion_t **const* pxRegions)

Memory regions are assigned to a restricted task when the task is created by a call to xTaskCreateRestricted(). These regions can be redefined using vTaskAllocateMPURegions().

Example usage:

```
// Define an array of MemoryRegion_t structures that configures an MPU region
// allowing read/write access for 1024 bytes starting at the beginning of the
// ucOneKByte array.  The other two of the maximum 3 definable regions are
// unused so set to zero.
static const MemoryRegion_t xAltRegions[ portNUM_CONFIGURABLE_REGIONS ] =
{
 // Base address     Length      Parameters
 { ucOneKByte,       1024,       portMPU_REGION_READ_WRITE },
 { 0,                0,          0 },
 { 0,                0,          0 }
};

void vATask( void *pvParameters )
{
 // This task was created such that it has access to certain regions of
 // memory as defined by the MPU configuration.  At some point it is
 // desired that these MPU regions are replaced with that defined in the
 // xAltRegions const struct above.  Use a call to vTaskAllocateMPURegions()
 // for this purpose.  NULL is used as the task handle to indicate that this
 // function should modify the MPU regions of the calling task.
 vTaskAllocateMPURegions( NULL, xAltRegions );

 // Now the task can continue its function, but from this point on can only
 // access its stack and the ucOneKByte array (unless any other statically
 // defined or shared regions have been declared elsewhere).
}
```

Parameters:

- **xTask** -- The handle of the task being updated.
- **pxRegions** -- A pointer to a MemoryRegion_t structure that contains the new memory region definitions.

void **vTaskDelete**(TaskHandle_t xTaskToDelete)

INCLUDE_vTaskDelete must be defined as 1 for this function to be available. See the configuration section for more information.

Remove a task from the RTOS real time kernel's management. The task being deleted will be removed from all ready, blocked, suspended and event lists.

NOTE: The idle task is responsible for freeing the kernel allocated memory from tasks that have been deleted. It is therefore important that the idle task is not starved of microcontroller processing time if your application makes any calls to vTaskDelete (). Memory allocated by the task code is not automatically freed, and should be freed before the task is deleted.

See the demo application file death.c for sample code that utilises vTaskDelete ().

Example usage:

```
void vOtherFunction( void )
{
TaskHandle_t xHandle;

// Create the task, storing the handle.
  xTaskCreate( vTaskCode, "NAME", STACK_SIZE, NULL, tskIDLE_PRIORITY, &xHandle );

// Use the handle to delete the task.
  vTaskDelete( xHandle );
}
```

Parameters: **xTaskToDelete** -- The handle of the task to be deleted. Passing NULL will cause the calling task to be deleted.

void **vTaskDelay**(*const* TickType_t xTicksToDelay)

Delay a task for a given number of ticks. The actual time that the task remains blocked depends on the tick rate. The constant portTICK_PERIOD_MS can be used to calculate real time from the tick rate - with the resolution of one tick period.

INCLUDE_vTaskDelay must be defined as 1 for this function to be available. See the configuration section for more information.

vTaskDelay() specifies a time at which the task wishes to unblock relative to the time at which vTaskDelay() is called. For example, specifying a block period of 100 ticks will cause the task to unblock 100 ticks after vTaskDelay() is called. vTaskDelay() does not therefore provide a good method of controlling the frequency of a periodic task as the path taken through the code, as well as other task and interrupt activity, will affect the frequency at which vTaskDelay() gets called and therefore the time at which the task next executes. See xTaskDelayUntil() for an alternative API function designed to facilitate fixed frequency execution. It does this by specifying an absolute time (rather than a relative time) at which the calling task should unblock.

Example usage:

```
void vTaskFunction( void * pvParameters )
{
// Block for 500ms.
const TickType_t xDelay = 500 / portTICK_PERIOD_MS;

for( ;; )
  {
// Simply toggle the LED every 500ms, blocking between each toggle.
      vToggleLED();
      vTaskDelay( xDelay );
  }
}
```

Parameters: **xTicksToDelay** -- The amount of time, in tick periods, that the calling task should block.

BaseType_t **xTaskDelayUntil**(TickType_t *const* pxPreviousWakeTime, *const* TickType_t xTimeIncrement)

INCLUDE_xTaskDelayUntil must be defined as 1 for this function to be available. See the configuration section for more information.

Delay a task until a specified time. This function can be used by periodic tasks to ensure a constant execution frequency.

This function differs from vTaskDelay () in one important aspect: vTaskDelay () will cause a task to block for the specified number of ticks from the time vTaskDelay () is called. It is therefore difficult to use vTaskDelay () by itself to generate a fixed execution frequency as the time between a task starting to execute and that task calling vTaskDelay () may not be fixed [the task may take a different path though the code between calls, or may get interrupted or preempted a different number of times each time it executes].

Whereas vTaskDelay () specifies a wake time relative to the time at which the function is called, xTaskDelayUntil () specifies the absolute (exact) time at which it wishes to unblock.

The macro pdMS_TO_TICKS() can be used to calculate the number of ticks from a time specified in milliseconds with a resolution of one tick period.

Example usage:

```
// Perform an action every 10 ticks.
void vTaskFunction( void * pvParameters )
{
TickType_t xLastWakeTime;
const TickType_t xFrequency = 10;
BaseType_t xWasDelayed;

// Initialise the xLastWakeTime variable with the current time.
    xLastWakeTime = xTaskGetTickCount ();
for( ;; )
    {
// Wait for the next cycle.
        xWasDelayed = xTaskDelayUntil( &xLastWakeTime, xFrequency );

// Perform action here. xWasDelayed value can be used to determine
// whether a deadline was missed if the code here took too long.
    }
}
```

Parameters:

- **pxPreviousWakeTime** -- Pointer to a variable that holds the time at which the task was last unblocked. The variable must be initialised with the current time prior to its first use (see the example below). Following this the variable is automatically updated within xTaskDelayUntil ().
- **xTimeIncrement** -- The cycle time period. The task will be unblocked at time *pxPreviousWakeTime + xTimeIncrement. Calling xTaskDelayUntil with the same xTimeIncrement parameter value will cause the task to execute with a fixed interface period.

Returns: Value which can be used to check whether the task was actually delayed. Will be pdTRUE if the task way delayed and pdFALSE otherwise. A task will not be delayed if the next expected wake time is in the past.

BaseType_t **xTaskAbortDelay**(TaskHandle_t xTask)

INCLUDE_xTaskAbortDelay must be defined as 1 in FreeRTOSConfig.h for this function to be available.

A task will enter the Blocked state when it is waiting for an event. The event it is waiting for can be a temporal event (waiting for a time), such as when vTaskDelay() is called, or an event on an object, such as when xQueueReceive() or ulTaskNotifyTake() is called. If the handle of a task that is in the Blocked state is used in a call to xTaskAbortDelay() then the task will leave the Blocked state, and return from whichever function call placed the task into the Blocked state.

There is no 'FromISR' version of this function as an interrupt would need to know which object a task was blocked on in order to know which actions to take. For example, if the task was blocked on a queue the interrupt handler would then need to know if the queue was locked.

Parameters: **xTask** -- The handle of the task to remove from the Blocked state.
Returns: If the task referenced by xTask was not in the Blocked state then pdFAIL is returned. Otherwise pdPASS is returned.

UBaseType_t **uxTaskPriorityGet**(*const* TaskHandle_t xTask)

INCLUDE_uxTaskPriorityGet must be defined as 1 for this function to be available. See the configuration section for more information.

Obtain the priority of any task.

Example usage:

```
void vAFunction( void )
{
TaskHandle_t xHandle;

// Create a task, storing the handle.
  xTaskCreate( vTaskCode, "NAME", STACK_SIZE, NULL, tskIDLE_PRIORITY, &xHandle );

// ...

// Use the handle to obtain the priority of the created task.
// It was created with tskIDLE_PRIORITY, but may have changed
// it itself.
if( uxTaskPriorityGet( xHandle ) != tskIDLE_PRIORITY )
  {
// The task has changed it's priority.
  }

// ...

// Is our priority higher than the created task?
if( uxTaskPriorityGet( xHandle ) < uxTaskPriorityGet( NULL ) )
  {
// Our priority (obtained using NULL handle) is higher.
  }
}
```

Parameters: **xTask** -- Handle of the task to be queried. Passing a NULL handle results in the priority of the calling task being returned.

Returns: The priority of xTask.

UBaseType_t **uxTaskPriorityGetFromISR**(*const* TaskHandle_t xTask)

A version of uxTaskPriorityGet() that can be used from an ISR.

---

eTaskState **eTaskGetState**(TaskHandle_t xTask)

INCLUDE_eTaskGetState must be defined as 1 for this function to be available. See the configuration section for more information.

Obtain the state of any task. States are encoded by the eTaskState enumerated type.

Parameters: **xTask** -- Handle of the task to be queried.

Returns: The state of xTask at the time the function was called. Note the state of the task might change between the function being called, and the functions return value being tested by the calling task.

---

void **vTaskGetInfo**(TaskHandle_t xTask, TaskStatus_t *pxTaskStatus, BaseType_t xGetFreeStackSpace, eTaskState eState)

configUSE_TRACE_FACILITY must be defined as 1 for this function to be available. See the configuration section for more information.

Populates a TaskStatus_t structure with information about a task.

Example usage:

```
void vAFunction( void )
{
TaskHandle_t xHandle;
TaskStatus_t xTaskDetails;

 // Obtain the handle of a task from its name.
 xHandle = xTaskGetHandle( "Task_Name" );

 // Check the handle is not NULL.
 configASSERT( xHandle );

 // Use the handle to obtain further information about the task.
 vTaskGetInfo( xHandle,
               &xTaskDetails,
               pdTRUE, // Include the high water mark in xTaskDetails.
               eInvalid ); // Include the task state in xTaskDetails.
}
```

Parameters:

- **xTask** -- Handle of the task being queried. If xTask is NULL then information will be returned about the calling task.
- **pxTaskStatus** -- A pointer to the TaskStatus_t structure that will be filled with information about the task referenced by the handle passed using the xTask parameter.
- **xGetFreeStackSpace** -- The TaskStatus_t structure contains a member to report the stack high water mark of the task being queried. Calculating the stack high water mark takes a relatively long time, and can make the system temporarily unresponsive - so the xGetFreeStackSpace parameter is provided to allow the high water mark checking to be skipped. The high watermark value will only be written to the TaskStatus_t structure if xGetFreeStackSpace is not set to pdFALSE;
- **eState** -- The TaskStatus_t structure contains a member to report the state of the task being queried. Obtaining the task state is not as fast as a simple assignment - so the eState parameter is provided to allow the state information to be omitted from the TaskStatus_t structure. To obtain state information then set eState to eInvalid - otherwise the value passed in eState will be reported as the task state in the TaskStatus_t structure.

void **vTaskPrioritySet**(TaskHandle_t xTask, UBaseType_t uxNewPriority)

INCLUDE_vTaskPrioritySet must be defined as 1 for this function to be available. See the configuration section for more information.

Set the priority of any task.

A context switch will occur before the function returns if the priority being set is higher than the currently executing task.

Example usage:

```
void vAFunction( void )
{
TaskHandle_t xHandle;

// Create a task, storing the handle.
  xTaskCreate( vTaskCode, "NAME", STACK_SIZE, NULL, tskIDLE_PRIORITY, &xHandle );

// ...

// Use the handle to raise the priority of the created task.
  vTaskPrioritySet( xHandle, tskIDLE_PRIORITY + 1 );

// ...

// Use a NULL handle to raise our priority to the same value.
  vTaskPrioritySet( NULL, tskIDLE_PRIORITY + 1 );
}
```

Parameters:

- **xTask** -- Handle to the task for which the priority is being set. Passing a NULL handle results in the priority of the calling task being set.
- **uxNewPriority** -- The priority to which the task will be set.

void **vTaskSuspend**(TaskHandle_t xTaskToSuspend)

INCLUDE_vTaskSuspend must be defined as 1 for this function to be available. See the configuration section for more information.

Suspend any task. When suspended a task will never get any microcontroller processing time, no matter what its priority.

Calls to vTaskSuspend are not accumulative - i.e. calling vTaskSuspend () twice on the same task still only requires one call to vTaskResume () to ready the suspended task.

Example usage:

```
void vAFunction( void )
{
TaskHandle_t xHandle;

// Create a task, storing the handle.
  xTaskCreate( vTaskCode, "NAME", STACK_SIZE, NULL, tskIDLE_PRIORITY, &xHandle );

// ...

// Use the handle to suspend the created task.
  vTaskSuspend( xHandle );

// ...

// The created task will not run during this period, unless
// another task calls vTaskResume( xHandle ).

//...


// Suspend ourselves.
  vTaskSuspend( NULL );

// We cannot get here unless another task calls vTaskResume
// with our handle as the parameter.
}
```

Parameters: **xTaskToSuspend** -- Handle to the task being suspended. Passing a NULL handle will cause the calling task to be suspended.

void **vTaskResume**(TaskHandle_t xTaskToResume)

INCLUDE_vTaskSuspend must be defined as 1 for this function to be available. See the configuration section for more information.

Resumes a suspended task.

A task that has been suspended by one or more calls to vTaskSuspend () will be made available for running again by a single call to vTaskResume ().

Example usage:

```
void vAFunction( void )
{
TaskHandle_t xHandle;

// Create a task, storing the handle.
  xTaskCreate( vTaskCode, "NAME", STACK_SIZE, NULL, tskIDLE_PRIORITY, &xHandle );

// ...

// Use the handle to suspend the created task.
  vTaskSuspend( xHandle );

// ...

// The created task will not run during this period, unless
// another task calls vTaskResume( xHandle ).

//...


// Resume the suspended task ourselves.
  vTaskResume( xHandle );

// The created task will once again get microcontroller processing
// time in accordance with its priority within the system.
}
```

Parameters: **xTaskToResume** -- Handle to the task being readied.

BaseType_t **xTaskResumeFromISR**(TaskHandle_t xTaskToResume)

INCLUDE_xTaskResumeFromISR must be defined as 1 for this function to be available. See the configuration section for more information.

An implementation of vTaskResume() that can be called from within an ISR.

A task that has been suspended by one or more calls to vTaskSuspend () will be made available for running again by a single call to xTaskResumeFromISR ().

xTaskResumeFromISR() should not be used to synchronise a task with an interrupt if there is a chance that the interrupt could arrive prior to the task being suspended - as this can lead to interrupts being missed. Use of a semaphore as a synchronisation mechanism would avoid this eventuality.

Parameters: **xTaskToResume** -- Handle to the task being readied.

Returns: pdTRUE if resuming the task should result in a context switch, otherwise pdFALSE. This is used by the ISR to determine if a context switch may be required following the ISR.

void **vTaskSuspendAll**(void)

Suspends the scheduler without disabling interrupts. Context switches will not occur while the scheduler is suspended.

After calling vTaskSuspendAll () the calling task will continue to execute without risk of being swapped out until a call to xTaskResumeAll () has been made.

API functions that have the potential to cause a context switch (for example, xTaskDelayUntil(), xQueueSend(), etc.) must not be called while the scheduler is suspended.

Example usage:

```
void vTask1( void * pvParameters )
{
for( ;; )
  {
// Task code goes here.

// ...

// At some point the task wants to perform a long operation during
// which it does not want to get swapped out.  It cannot use
// taskENTER_CRITICAL ()/taskEXIT_CRITICAL () as the length of the
// operation may cause interrupts to be missed - including the
// ticks.

// Prevent the real time kernel swapping out the task.
      vTaskSuspendAll ();

// Perform the operation here.  There is no need to use critical
// sections as we have all the microcontroller processing time.
// During this time interrupts will still operate and the kernel
// tick count will be maintained.

// ...

// The operation is complete.  Restart the kernel.
      xTaskResumeAll ();
  }
}
```

BaseType_t **xTaskResumeAll**(void)

Resumes scheduler activity after it was suspended by a call to vTaskSuspendAll().

xTaskResumeAll() only resumes the scheduler. It does not unsuspend tasks that were previously suspended by a call to vTaskSuspend().

Example usage:

```
void vTask1( void * pvParameters )
{
for( ;; )
  {
// Task code goes here.

// ...

// At some point the task wants to perform a long operation during
// which it does not want to get swapped out.  It cannot use
// taskENTER_CRITICAL ()/taskEXIT_CRITICAL () as the length of the
// operation may cause interrupts to be missed - including the
// ticks.

// Prevent the real time kernel swapping out the task.
      vTaskSuspendAll ();

// Perform the operation here.  There is no need to use critical
// sections as we have all the microcontroller processing time.
// During this time interrupts will still operate and the real
// time kernel tick count will be maintained.

// ...

// The operation is complete.  Restart the kernel.  We want to force
// a context switch - but there is no point if resuming the scheduler
// caused a context switch already.
if( !xTaskResumeAll () )
      {
           taskYIELD ();
      }
  }
}
```

Returns: If resuming the scheduler caused a context switch then pdTRUE is returned, otherwise pdFALSE is returned.

TickType_t **xTaskGetTickCount**(void)

Returns: The count of ticks since vTaskStartScheduler was called.

TickType_t **xTaskGetTickCountFromISR**(void)

This is a version of xTaskGetTickCount() that is safe to be called from an ISR - provided that TickType_t is the natural word size of the microcontroller being used or interrupt nesting is either not supported or not being used.

Returns: The count of ticks since vTaskStartScheduler was called.

UBaseType_t **uxTaskGetNumberOfTasks**(void)

Returns: The number of tasks that the real time kernel is currently managing. This includes all ready, blocked and suspended tasks. A task that has been deleted but not yet freed by the idle task will also be included in the count.

char *pcTaskGetName(TaskHandle_t xTaskToQuery)

Returns: The text (human readable) name of the task referenced by the handle xTaskToQuery. A task can query its own name by either passing in its own handle, or by setting xTaskToQuery to NULL.

TaskHandle_t xTaskGetHandle(*const* char *pcNameToQuery)

NOTE: This function takes a relatively long time to complete and should be used sparingly.

Returns: The handle of the task that has the human readable name pcNameToQuery. NULL is returned if no matching name is found. INCLUDE_xTaskGetHandle must be set to 1 in FreeRTOSConfig.h for pcTaskGetHandle() to be available.

BaseType_t xTaskGetStaticBuffers(TaskHandle_t xTask, StackType_t **ppuxStackBuffer, StaticTask_t **ppxTaskBuffer)

Retrieve pointers to a statically created task's data structure buffer and stack buffer. These are the same buffers that are supplied at the time of creation.

Parameters:
- **xTask** -- The task for which to retrieve the buffers.
- **ppuxStackBuffer** -- Used to return a pointer to the task's stack buffer.
- **ppxTaskBuffer** -- Used to return a pointer to the task's data structure buffer.

Returns: pdTRUE if buffers were retrieved, pdFALSE otherwise.

UBaseType_t uxTaskGetStackHighWaterMark(TaskHandle_t xTask)

INCLUDE_uxTaskGetStackHighWaterMark must be set to 1 in FreeRTOSConfig.h for this function to be available.

Returns the high water mark of the stack associated with xTask. That is, the minimum free stack space there has been in bytes (as opposed to words in the standard FreeRTOS documentation) since the task started. The smaller the returned number the closer the task has come to overflowing its stack.

uxTaskGetStackHighWaterMark() and uxTaskGetStackHighWaterMark2() are the same except for their return type. Using configSTACK_DEPTH_TYPE allows the user to determine the return type. It gets around the problem of the value overflowing on 8-bit types without breaking backward compatibility for applications that expect an 8-bit return type.

Parameters: **xTask** -- Handle of the task associated with the stack to be checked. Set xTask to NULL to check the stack of the calling task.

Returns: The smallest amount of free stack space there has been in bytes (as opposed to words in the standard FreeRTOS documentation) since the task referenced by xTask was created.

---

configSTACK_DEPTH_TYPE **uxTaskGetStackHighWaterMark2**(TaskHandle_t xTask)

INCLUDE_uxTaskGetStackHighWaterMark2 must be set to 1 in FreeRTOSConfig.h for this function to be available.

Returns the high water mark of the stack associated with xTask. That is, the minimum free stack space there has been in bytes (as opposed to words in the standard FreeRTOS documentation) since the task started. The smaller the returned number the closer the task has come to overflowing its stack.

uxTaskGetStackHighWaterMark() and uxTaskGetStackHighWaterMark2() are the same except for their return type. Using configSTACK_DEPTH_TYPE allows the user to determine the return type. It gets around the problem of the value overflowing on 8-bit types without breaking backward compatibility for applications that expect an 8-bit return type.

Parameters: **xTask** -- Handle of the task associated with the stack to be checked. Set xTask to NULL to check the stack of the calling task.

Returns: The smallest amount of free stack space there has been in bytes (as opposed to words in the standard FreeRTOS documentation) since the task referenced by xTask was created.

---

void **vTaskSetApplicationTaskTag**(TaskHandle_t xTask, TaskHookFunction_t pxHookFunction)

Sets pxHookFunction to be the task hook function used by the task xTask. Passing xTask as NULL has the effect of setting the calling tasks hook function.

TaskHookFunction_t **xTaskGetApplicationTaskTag**(TaskHandle_t xTask)

Returns the pxHookFunction value assigned to the task xTask. Do not call from an interrupt service routine - call xTaskGetApplicationTaskTagFromISR() instead.

TaskHookFunction_t **xTaskGetApplicationTaskTagFromISR**(TaskHandle_t xTask)

Returns the pxHookFunction value assigned to the task xTask. Can be called from an interrupt service routine.

---

void **vTaskSetThreadLocalStoragePointer**(TaskHandle_t xTaskToSet, BaseType_t xIndex, void *pvValue)

Each task contains an array of pointers that is dimensioned by the configNUM_THREAD_LOCAL_STORAGE_POINTERS setting in FreeRTOSConfig.h. The kernel does not use the pointers itself, so the application writer can use the pointers for

any purpose they wish. The following two functions are used to set and query a pointer respectively.

---

void *pvTaskGetThreadLocalStoragePointer(TaskHandle_t xTaskToQuery, BaseType_t xIndex)

---

void vApplicationGetIdleTaskMemory(StaticTask_t **ppxIdleTaskTCBBuffer, StackType_t **ppxIdleTaskStackBuffer, uint32_t *pulIdleTaskStackSize)

This function is used to provide a statically allocated block of memory to FreeRTOS to hold the Idle Task TCB. This function is required when configSUPPORT_STATIC_ALLOCATION is set. For more information see this URI: https://www.FreeRTOS.org/a00110.html#configSUPPORT_STATIC_ALLOCATION

Parameters:

- **ppxIdleTaskTCBBuffer** -- A handle to a statically allocated TCB buffer
- **ppxIdleTaskStackBuffer** -- A handle to a statically allocated Stack buffer for the idle task
- **pulIdleTaskStackSize** -- A pointer to the number of elements that will fit in the allocated stack buffer

---

BaseType_t xTaskCallApplicationTaskHook(TaskHandle_t xTask, void *pvParameter)

Calls the hook function associated with xTask. Passing xTask as NULL has the effect of calling the Running tasks (the calling task) hook function.

pvParameter is passed to the hook function for the task to interpret as it wants. The return value is the value returned by the task hook function registered by the user.

TaskHandle_t xTaskGetIdleTaskHandle(void)

xTaskGetIdleTaskHandle() is only available if INCLUDE_xTaskGetIdleTaskHandle is set to 1 in FreeRTOSConfig.h.

Simply returns the handle of the idle task of the current core. It is not valid to call xTaskGetIdleTaskHandle() before the scheduler has been started.

UBaseType_t uxTaskGetSystemState(TaskStatus_t *const pxTaskStatusArray, const UBaseType_t uxArraySize, configRUN_TIME_COUNTER_TYPE *const pulTotalRunTime)

configUSE_TRACE_FACILITY must be defined as 1 in FreeRTOSConfig.h for uxTaskGetSystemState() to be available.

uxTaskGetSystemState() populates an TaskStatus_t structure for each task in the system. TaskStatus_t structures contain, among other things, members for the task handle, task name, task priority, task state, and total amount of run time consumed by the task. See the TaskStatus_t structure definition in this file for the full member list.

NOTE: This function is intended for debugging use only as its use results in the scheduler remaining suspended for an extended period.

Example usage:

```
 // This example demonstrates how a human readable table of run time stats
 // information is generated from raw data provided by uxTaskGetSystemState().
 // The human readable table is written to pcWriteBuffer
 void vTaskGetRunTimeStats( char *pcWriteBuffer )
 {
 TaskStatus_t *pxTaskStatusArray;
 volatile UBaseType_t uxArraySize, x;
 configRUN_TIME_COUNTER_TYPE ulTotalRunTime, ulStatsAsPercentage;

// Make sure the write buffer does not contain a string.
pcWriteBuffer = 0x00;

// Take a snapshot of the number of tasks in case it changes while this
// function is executing.
     uxArraySize = uxTaskGetNumberOfTasks();

// Allocate a TaskStatus_t structure for each task.  An array could be
// allocated statically at compile time.
     pxTaskStatusArray = pvPortMalloc( uxArraySize * sizeof( TaskStatus_t ) );

if( pxTaskStatusArray != NULL )
     {
// Generate raw status information about each task.
         uxArraySize = uxTaskGetSystemState( pxTaskStatusArray, uxArraySize,
&ulTotalRunTime );

// For percentage calculations.
         ulTotalRunTime /= 100UL;

// Avoid divide by zero errors.
if( ulTotalRunTime > 0 )
         {
// For each populated position in the pxTaskStatusArray array,
// format the raw data as human readable ASCII data
for( x = 0; x < uxArraySize; x++ )
             {
// What percentage of the total run time has the task used?
// This will always be rounded down to the nearest integer.
// ulTotalRunTimeDiv100 has already been divided by 100.
                 ulStatsAsPercentage = pxTaskStatusArray[ x ].ulRunTimeCounter /
ulTotalRunTime;

if( ulStatsAsPercentage > 0UL )
                 {
                     sprintf( pcWriteBuffer, "%s\t\t%lu\t\t%lu%%\r\n",
pxTaskStatusArray[ x ].pcTaskName, pxTaskStatusArray[ x ].ulRunTimeCounter,
ulStatsAsPercentage );
                 }
else
                 {
// If the percentage is zero here then the task has
// consumed less than 1% of the total run time.
                     sprintf( pcWriteBuffer, "%s\t\t%lu\t\t<1%%\r\n",
pxTaskStatusArray[ x ].pcTaskName, pxTaskStatusArray[ x ].ulRunTimeCounter );
                 }

                 pcWriteBuffer += strlen( ( char * ) pcWriteBuffer );
             }
         }

// The array is no longer needed, free the memory it consumes.
         vPortFree( pxTaskStatusArray );
```

```
    }
}
```

Parameters:

- **pxTaskStatusArray** -- A pointer to an array of TaskStatus_t structures. The array must contain at least one TaskStatus_t structure for each task that is under the control of the RTOS. The number of tasks under the control of the RTOS can be determined using the uxTaskGetNumberOfTasks() API function.
- **uxArraySize** -- The size of the array pointed to by the pxTaskStatusArray parameter. The size is specified as the number of indexes in the array, or the number of TaskStatus_t structures contained in the array, not by the number of bytes in the array.
- **pulTotalRunTime** -- If configGENERATE_RUN_TIME_STATS is set to 1 in FreeRTOSConfig.h then *pulTotalRunTime is set by uxTaskGetSystemState() to the total run time (as defined by the run time stats clock, see https://www.FreeRTOS.org/rtos-run-time-stats.html) since the target booted. pulTotalRunTime can be set to NULL to omit the total run time information.

Returns: The number of TaskStatus_t structures that were populated by uxTaskGetSystemState(). This should equal the number returned by the uxTaskGetNumberOfTasks() API function, but will be zero if the value passed in the uxArraySize parameter was too small.

void `vTaskList`(char *pcWriteBuffer)

configUSE_TRACE_FACILITY and configUSE_STATS_FORMATTING_FUNCTIONS must both be defined as 1 for this function to be available. See the configuration section of the FreeRTOS.org website for more information.

NOTE 1: This function will disable interrupts for its duration. It is not intended for normal application runtime use but as a debug aid.

Lists all the current tasks, along with their current state and stack usage high water mark.

Tasks are reported as blocked ('B'), ready ('R'), deleted ('D') or suspended ('S').

PLEASE NOTE:

This function is provided for convenience only, and is used by many of the demo applications. Do not consider it to be part of the scheduler.

vTaskList() calls uxTaskGetSystemState(), then formats part of the uxTaskGetSystemState() output into a human readable table that displays task: names, states, priority, stack usage and task number. Stack usage specified as the number of unused StackType_t words stack can hold on top of stack - not the number of bytes.

vTaskList() has a dependency on the sprintf() C library function that might bloat the code size, use a lot of stack, and provide different results on different platforms. An alternative, tiny, third party, and limited functionality implementation of sprintf() is provided in many of the FreeRTOS/Demo sub-directories in a file called printf-stdarg.c (note printf-stdarg.c does not provide a full snprintf() implementation!).

It is recommended that production systems call uxTaskGetSystemState() directly to get access to raw stats data, rather than indirectly through a call to vTaskList().

Parameters: **pcWriteBuffer** -- A buffer into which the above mentioned details will be written, in ASCII form. This buffer is assumed to be large enough to contain the generated report. Approximately 40 bytes per task should be sufficient.

---

void `vTaskGetRunTimeStats`(char *pcWriteBuffer)

configGENERATE_RUN_TIME_STATS and configUSE_STATS_FORMATTING_FUNCTIONS must both be defined as 1 for this function to be available. The application must also then provide definitions for portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() and portGET_RUN_TIME_COUNTER_VALUE() to configure a peripheral timer/counter and return the timers current count value respectively. The counter should be at least 10 times the frequency of the tick count.

NOTE 1: This function will disable interrupts for its duration. It is not intended for normal application runtime use but as a debug aid.

Setting configGENERATE_RUN_TIME_STATS to 1 will result in a total accumulated execution time being stored for each task. The resolution of the accumulated time value depends on the frequency of the timer configured by the portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() macro. Calling vTaskGetRunTimeStats() writes the total execution time of each task into a buffer, both as an absolute count value and as a percentage of the total system execution time.

NOTE 2:

This function is provided for convenience only, and is used by many of the demo applications. Do not consider it to be part of the scheduler.

vTaskGetRunTimeStats() calls uxTaskGetSystemState(), then formats part of the uxTaskGetSystemState() output into a human readable table that displays the amount of time each task has spent in the Running state in both absolute and percentage terms.

vTaskGetRunTimeStats() has a dependency on the sprintf() C library function that might bloat the code size, use a lot of stack, and provide different results on different platforms. An alternative, tiny, third party, and limited functionality implementation of sprintf() is provided in many of the FreeRTOS/Demo sub-directories in a file called printf-stdarg.c (note printf-stdarg.c does not provide a full snprintf() implementation!).

It is recommended that production systems call uxTaskGetSystemState() directly to get access to raw stats data, rather than indirectly through a call to vTaskGetRunTimeStats().

Parameters: **pcWriteBuffer** -- A buffer into which the execution times will be written, in ASCII form. This buffer is assumed to be large enough to contain the generated report. Approximately 40 bytes per task should be sufficient.

---

configRUN_TIME_COUNTER_TYPE **ulTaskGetIdleRunTimeCounter**(void)

configGENERATE_RUN_TIME_STATS, configUSE_STATS_FORMATTING_FUNCTIONS and INCLUDE_xTaskGetIdleTaskHandle must all be defined as 1 for these functions to be available. The application must also then provide definitions for portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() and portGET_RUN_TIME_COUNTER_VALUE() to configure a peripheral timer/counter and return the timers current count value respectively. The counter should be at least 10 times the frequency of the tick count.

Setting configGENERATE_RUN_TIME_STATS to 1 will result in a total accumulated execution time being stored for each task. The resolution of the accumulated time value depends on the frequency of the timer configured by the portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() macro. While uxTaskGetSystemState() and vTaskGetRunTimeStats() writes the total execution time of each task into a buffer, ulTaskGetIdleRunTimeCounter() returns the total execution time of just the idle task and ulTaskGetIdleRunTimePercent() returns the percentage of the CPU time used by just the idle task.

Note the amount of idle time is only a good measure of the slack time in a system if there are no other tasks executing at the idle priority, tickless idle is not used, and configIDLE_SHOULD_YIELD is set to 0.

Note

If configNUMBER_OF_CORES > 1, calling this function will query the idle task of the current core.

Returns: The total run time of the idle task or the percentage of the total run time consumed by the idle task. This is the amount of time the idle task has actually been executing. The unit of time is dependent on the frequency configured using the portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() and portGET_RUN_TIME_COUNTER_VALUE() macros.

---

configRUN_TIME_COUNTER_TYPE **ulTaskGetIdleRunTimePercent**(void)

BaseType_t **xTaskGenericNotifyWait**(UBaseType_t uxIndexToWaitOn, uint32_t ulBitsToClearOnEntry, uint32_t ulBitsToClearOnExit, uint32_t *pulNotificationValue, TickType_t xTicksToWait)

Waits for a direct to task notification to be pending at a given index within an array of direct to task notifications.

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for this function to be available.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

Events can be sent to a task using an intermediary object. Examples of such objects are queues, semaphores, mutexes and event groups. Task notifications are a method of sending an event directly to a task without the need for such an intermediary object.

A notification sent to a task can optionally perform an action, such as update, overwrite or increment one of the task's notification values. In that way task notifications can be used to send data to a task, or be used as light weight and fast binary or counting semaphores.

A notification sent to a task will remain pending until it is cleared by the task calling xTaskNotifyWaitIndexed() or ulTaskNotifyTakeIndexed() (or their un-indexed equivalents). If the task was already in the Blocked state to wait for a notification when the notification arrives then the task will automatically be removed from the Blocked state (unblocked) and the notification cleared.

A task can use xTaskNotifyWaitIndexed() to [optionally] block to wait for a notification to be pending, or ulTaskNotifyTakeIndexed() to [optionally] block to wait for a notification value to have a non-zero value. The task does not consume any CPU time while it is in the Blocked state.

NOTE Each notification within the array operates independently - a task can only block on one notification within the array at a time and will not be unblocked by a notification sent to any other array index.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. xTaskNotifyWait() is the original API function, and remains backward compatible by always operating on the notification value at index 0 in the array. Calling xTaskNotifyWait() is equivalent to calling xTaskNotifyWaitIndexed() with the uxIndexToWaitOn parameter set to 0.

Parameters:

- **uxIndexToWaitOn** -- The index within the calling task's array of notification values on which the calling task will wait for a notification to be received. uxIndexToWaitOn must be less than

configTASK_NOTIFICATION_ARRAY_ENTRIES. xTaskNotifyWait() does not have this parameter and always waits for notifications on index 0.

- **ulBitsToClearOnEntry** -- Bits that are set in ulBitsToClearOnEntry value will be cleared in the calling task's notification value before the task checks to see if any notifications are pending, and optionally blocks if no notifications are pending. Setting ulBitsToClearOnEntry to ULONG_MAX (if limits.h is included) or 0xffffffffUL (if limits.h is not included) will have the effect of resetting the task's notification value to 0. Setting ulBitsToClearOnEntry to 0 will leave the task's notification value unchanged.
- **ulBitsToClearOnExit** -- If a notification is pending or received before the calling task exits the xTaskNotifyWait() function then the task's notification value (see the xTaskNotify() API function) is passed out using the pulNotificationValue parameter. Then any bits that are set in ulBitsToClearOnExit will be cleared in the task's notification value (note *pulNotificationValue is set before any bits are cleared). Setting ulBitsToClearOnExit to ULONG_MAX (if limits.h is included) or 0xffffffffUL (if limits.h is not included) will have the effect of resetting the task's notification value to 0 before the function exits. Setting ulBitsToClearOnExit to 0 will leave the task's notification value unchanged when the function exits (in which case the value passed out in pulNotificationValue will match the task's notification value).
- **pulNotificationValue** -- Used to pass the task's notification value out of the function. Note the value passed out will not be effected by the clearing of any bits caused by ulBitsToClearOnExit being non-zero.
- **xTicksToWait** -- The maximum amount of time that the task should wait in the Blocked state for a notification to be received, should a notification not already be pending when xTaskNotifyWait() was called. The task will not consume any processing time while it is in the Blocked state. This is specified in kernel ticks, the macro pdMS_TO_TICKS(value_in_ms) can be used to convert a time specified in milliseconds to a time specified in ticks.

Returns: If a notification was received (including notifications that were already pending when xTaskNotifyWait was called) then pdPASS is returned. Otherwise pdFAIL is returned.

---

void **vTaskGenericNotifyGiveFromISR**(TaskHandle_t xTaskToNotify, UBaseType_t uxIndexToNotify, BaseType_t *pxHigherPriorityTaskWoken)

A version of xTaskNotifyGiveIndexed() that can be called from an interrupt service routine (ISR).

See https://www.FreeRTOS.org/RTOS-task-notifications.html for more details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for this macro to be available.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

Events can be sent to a task using an intermediary object. Examples of such objects are queues, semaphores, mutexes and event groups. Task notifications are a method of sending an event directly to a task without the need for such an intermediary object.

A notification sent to a task can optionally perform an action, such as update, overwrite or increment one of the task's notification values. In that way task notifications can be used to send data to a task, or be used as light weight and fast binary or counting semaphores.

vTaskNotifyGiveIndexedFromISR() is intended for use when task notifications are used as light weight and faster binary or counting semaphore equivalents. Actual FreeRTOS semaphores are given from an ISR using the xSemaphoreGiveFromISR() API function, the equivalent action that instead uses a task notification is vTaskNotifyGiveIndexedFromISR().

When task notifications are being used as a binary or counting semaphore equivalent then the task being notified should wait for the notification using the ulTaskNotifyTakeIndexed() API function rather than the xTaskNotifyWaitIndexed() API function.

NOTE Each notification within the array operates independently - a task can only block on one notification within the array at a time and will not be unblocked by a notification sent to any other array index.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. xTaskNotifyFromISR() is the original API function, and remains backward compatible by always operating on the notification value at index 0 within the array. Calling xTaskNotifyGiveFromISR() is equivalent to calling xTaskNotifyGiveIndexedFromISR() with the uxIndexToNotify parameter set to 0.

Parameters:

- **xTaskToNotify** -- The handle of the task being notified. The handle to a task can be returned from the xTaskCreate() API function used to create the task, and the handle of the currently running task can be obtained by calling xTaskGetCurrentTaskHandle().

- **uxIndexToNotify** -- The index within the target task's array of notification values to which the notification is to be sent. uxIndexToNotify must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. xTaskNotifyGiveFromISR() does not have this parameter and always sends notifications to index 0.
- **pxHigherPriorityTaskWoken** -- vTaskNotifyGiveFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending the notification caused the task to which the notification was sent to leave the Blocked state, and the unblocked task has a priority higher than the currently running task. If vTaskNotifyGiveFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited. How a context switch is requested from an ISR is dependent on the port - see the documentation page for the port in use.

BaseType_t **xTaskGenericNotifyStateClear**(TaskHandle_t xTask, UBaseType_t uxIndexToClear)

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for these functions to be available.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

If a notification is sent to an index within the array of notifications then the notification at that index is said to be 'pending' until it is read or explicitly cleared by the receiving task. xTaskNotifyStateClearIndexed() is the function that clears a pending notification without reading the notification value. The notification value at the same array index is not altered. Set xTask to NULL to clear the notification state of the calling task.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. xTaskNotifyStateClear() is the original API function, and remains backward compatible by always operating on the notification value at index 0 within the array. Calling xTaskNotifyStateClear() is equivalent to calling xTaskNotifyStateClearIndexed() with the uxIndexToNotify parameter set to 0.

Parameters:

- **xTask** -- The handle of the RTOS task that will have a notification state cleared. Set xTask to NULL to clear a notification state in the calling task. To obtain a task's handle create the task using

xTaskCreate() and make use of the pxCreatedTask parameter, or create the task using xTaskCreateStatic() and store the returned value, or use the task's name in a call to xTaskGetHandle().

- **uxIndexToClear** -- The index within the target task's array of notification values to act upon. For example, setting uxIndexToClear to 1 will clear the state of the notification at index 1 within the array. uxIndexToClear must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. ulTaskNotifyStateClear() does not have this parameter and always acts on the notification at index 0.

Returns: pdTRUE if the task's notification state was set to eNotWaitingNotification, otherwise pdFALSE.

---

uint32_t **ulTaskGenericNotifyValueClear**(TaskHandle_t xTask, UBaseType_t uxIndexToClear, uint32_t ulBitsToClear)

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for these functions to be available.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

ulTaskNotifyValueClearIndexed() clears the bits specified by the ulBitsToClear bit mask in the notification value at array index uxIndexToClear of the task referenced by xTask.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. ulTaskNotifyValueClear() is the original API function, and remains backward compatible by always operating on the notification value at index 0 within the array. Calling ulTaskNotifyValueClear() is equivalent to calling ulTaskNotifyValueClearIndexed() with the uxIndexToClear parameter set to 0.

Parameters:

- **xTask** -- The handle of the RTOS task that will have bits in one of its notification values cleared. Set xTask to NULL to clear bits in a notification value of the calling task. To obtain a task's handle create the task using xTaskCreate() and make use of the pxCreatedTask parameter, or create the task using xTaskCreateStatic() and store the returned value, or use the task's name in a call to xTaskGetHandle().
- **uxIndexToClear** -- The index within the target task's array of notification values in which to clear the bits. uxIndexToClear must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES.

ulTaskNotifyValueClear() does not have this parameter and always clears bits in the notification value at index 0.

- **ulBitsToClear** -- Bit mask of the bits to clear in the notification value of xTask. Set a bit to 1 to clear the corresponding bits in the task's notification value. Set ulBitsToClear to 0xffffffff (UINT_MAX on 32-bit architectures) to clear the notification value to 0. Set ulBitsToClear to 0 to query the task's notification value without clearing any bits.

Returns: The value of the target task's notification value before the bits specified by ulBitsToClear were cleared.

---

void **vTaskSetTimeOutState**(TimeOut_t *const pxTimeOut)

Capture the current time for future use with xTaskCheckForTimeOut().

Parameters: **pxTimeOut** -- Pointer to a timeout object into which the current time is to be captured. The captured time includes the tick count and the number of times the tick count has overflowed since the system first booted.

BaseType_t **xTaskCheckForTimeOut**(TimeOut_t *const pxTimeOut, TickType_t *const pxTicksToWait)

Determines if pxTicksToWait ticks has passed since a time was captured using a call to vTaskSetTimeOutState(). The captured time includes the tick count and the number of times the tick count has overflowed.

Example Usage:

```
// Driver library function used to receive uxWantedBytes from an Rx buffer
// that is filled by a UART interrupt. If there are not enough bytes in the
// Rx buffer then the task enters the Blocked state until it is notified that
// more data has been placed into the buffer. If there is still not enough
// data then the task re-enters the Blocked state, and xTaskCheckForTimeOut()
// is used to re-calculate the Block time to ensure the total amount of time
// spent in the Blocked state does not exceed MAX_TIME_TO_WAIT. This
// continues until either the buffer contains at least uxWantedBytes bytes,
// or the total amount of time spent in the Blocked state reaches
// MAX_TIME_TO_WAIT - at which point the task reads however many bytes are
// available up to a maximum of uxWantedBytes.

size_t xUART_Receive( uint8_t *pucBuffer, size_t uxWantedBytes )
{
size_t uxReceived = 0;
TickType_t xTicksToWait = MAX_TIME_TO_WAIT;
TimeOut_t xTimeOut;

// Initialize xTimeOut.  This records the time at which this function
// was entered.
    vTaskSetTimeOutState( &xTimeOut );

// Loop until the buffer contains the wanted number of bytes, or a
// timeout occurs.
while( UART_bytes_in_rx_buffer( pxUARTInstance ) < uxWantedBytes )
    {
// The buffer didn't contain enough data so this task is going to
// enter the Blocked state. Adjusting xTicksToWait to account for
// any time that has been spent in the Blocked state within this
// function so far to ensure the total amount of time spent in the
// Blocked state does not exceed MAX_TIME_TO_WAIT.
if( xTaskCheckForTimeOut( &xTimeOut, &xTicksToWait ) != pdFALSE )
        {
//Timed out before the wanted number of bytes were available,
// exit the loop.
break;
        }

// Wait for a maximum of xTicksToWait ticks to be notified that the
// receive interrupt has placed more data into the buffer.
        ulTaskNotifyTake( pdTRUE, xTicksToWait );
    }

// Attempt to read uxWantedBytes from the receive buffer into pucBuffer.
// The actual number of bytes read (which might be less than
// uxWantedBytes) is returned.
    uxReceived = UART_read_from_receive_buffer( pxUARTInstance,
                                                pucBuffer,
                                                uxWantedBytes );

return uxReceived;
}
```

See also

https://www.FreeRTOS.org/xTaskCheckForTimeOut.html

Parameters:

- **pxTimeOut** -- The time status as captured previously using vTaskSetTimeOutState. If the timeout has not yet occurred, it is updated to reflect the current time status.
- **pxTicksToWait** -- The number of ticks to check for timeout i.e. if pxTicksToWait ticks have passed since pxTimeOut was last updated (either by vTaskSetTimeOutState() or xTaskCheckForTimeOut()), the timeout has occurred. If the timeout has not occurred, pxTicksToWait is updated to reflect the number of remaining ticks.

Returns: If timeout has occurred, pdTRUE is returned. Otherwise pdFALSE is returned and pxTicksToWait is updated to reflect the number of remaining ticks.

BaseType_t **xTaskCatchUpTicks**(TickType_t xTicksToCatchUp)

This function corrects the tick count value after the application code has held interrupts disabled for an extended period resulting in tick interrupts having been missed.

This function is similar to vTaskStepTick(), however, unlike vTaskStepTick(), xTaskCatchUpTicks() may move the tick count forward past a time at which a task should be removed from the blocked state. That means tasks may have to be removed from the blocked state as the tick count is moved.

Parameters: **xTicksToCatchUp** -- The number of tick interrupts that have been missed due to interrupts being disabled. Its value is not computed automatically, so must be computed by the application writer.

Returns: pdTRUE if moving the tick count forward resulted in a task leaving the blocked state and a context switch being performed. Otherwise pdFALSE.

Structures

---

struct **xTASK_STATUS**

Used with the uxTaskGetSystemState() function to return the state of each task in the system.

Public Members

TaskHandle_t **xHandle**

The handle of the task to which the rest of the information in the structure relates.

const char ***pcTaskName**

A pointer to the task's name. This value will be invalid if the task was deleted since the structure was populated!

UBaseType_t **xTaskNumber**

A number unique to the task.

eTaskState **eCurrentState**

The state in which the task existed when the structure was populated.

UBaseType_t **uxCurrentPriority**

The priority at which the task was running (may be inherited) when the structure was populated.

UBaseType_t **uxBasePriority**

The priority to which the task will return if the task's current priority has been inherited to avoid unbounded priority inversion when obtaining a mutex. Only valid if configUSE_MUTEXES is defined as 1 in FreeRTOSConfig.h.

configRUN_TIME_COUNTER_TYPE **ulRunTimeCounter**

The total run time allocated to the task so far, as defined by the run time stats clock. See https://www.FreeRTOS.org/rtos-run-time-stats.html. Only valid when configGENERATE_RUN_TIME_STATS is defined as 1 in FreeRTOSConfig.h.

StackType_t ***pxStackBase**

Points to the lowest address of the task's stack area.

configSTACK_DEPTH_TYPE **usStackHighWaterMark**

The minimum amount of stack space that has remained for the task since the task was created. The closer this value is to zero the closer the task has come to overflowing its stack.

BaseType_t **xCoreID**

Core this task is pinned to (0, 1, or tskNO_AFFINITY). If configNUMBER_OF_CORES == 1, this will always be 0.

Macros

tskIDLE_PRIORITY

Defines the priority used by the idle task. This must not be modified.

tskNO_AFFINITY

Macro representing and unpinned (i.e., "no affinity") task in xCoreID parameters

taskVALID_CORE_ID(xCoreID)

Macro to check if an xCoreID value is valid

Returns: pdTRUE if valid, pdFALSE otherwise.

---

taskYIELD()

Macro for forcing a context switch.

---

taskENTER_CRITICAL(x)

Macro to mark the start of a critical code region. Preemptive context switches cannot occur when in a critical region.

NOTE: This may alter the stack (depending on the portable implementation) so must be used with care!

---

taskENTER_CRITICAL_FROM_ISR()

---

taskENTER_CRITICAL_ISR(x)

---

taskEXIT_CRITICAL(x)

Macro to mark the end of a critical code region. Preemptive context switches cannot occur when in a critical region.

NOTE: This may alter the stack (depending on the portable implementation) so must be used with care!

---

taskEXIT_CRITICAL_FROM_ISR(x)

---

taskEXIT_CRITICAL_ISR(x)

---

taskDISABLE_INTERRUPTS()

Macro to disable all maskable interrupts.

---

taskENABLE_INTERRUPTS()

Macro to enable microcontroller interrupts.

---

taskSCHEDULER_SUSPENDED

Definitions returned by xTaskGetSchedulerState(). taskSCHEDULER_SUSPENDED is 0 to generate more optimal code when configASSERT() is defined as the constant is used in assert() statements.

---

taskSCHEDULER_NOT_STARTED

`taskSCHEDULER_RUNNING`

`xTaskNotifyIndexed`(xTaskToNotify, uxIndexToNotify, ulValue, eAction)

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for these functions to be available.

Sends a direct to task notification to a task, with an optional value and action.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

Events can be sent to a task using an intermediary object. Examples of such objects are queues, semaphores, mutexes and event groups. Task notifications are a method of sending an event directly to a task without the need for such an intermediary object.

A notification sent to a task can optionally perform an action, such as update, overwrite or increment one of the task's notification values. In that way task notifications can be used to send data to a task, or be used as light weight and fast binary or counting semaphores.

A task can use xTaskNotifyWaitIndexed() or ulTaskNotifyTakeIndexed() to [optionally] block to wait for a notification to be pending. The task does not consume any CPU time while it is in the Blocked state.

A notification sent to a task will remain pending until it is cleared by the task calling xTaskNotifyWaitIndexed() or ulTaskNotifyTakeIndexed() (or their un-indexed equivalents). If the task was already in the Blocked state to wait for a notification when the notification arrives then the task will automatically be removed from the Blocked state (unblocked) and the notification cleared.

NOTE Each notification within the array operates independently - a task can only block on one notification within the array at a time and will not be unblocked by a notification sent to any other array index.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. xTaskNotify() is the original API function, and remains backward compatible by always operating on the notification value at index 0 in the array. Calling xTaskNotify() is equivalent to calling xTaskNotifyIndexed() with the uxIndexToNotify parameter set to 0.

eSetBits - The target notification value is bitwise ORed with ulValue. xTaskNotifyIndexed() always returns pdPASS in this case.

eIncrement - The target notification value is incremented. ulValue is not used and xTaskNotifyIndexed() always returns pdPASS in this case.

eSetValueWithOverwrite - The target notification value is set to the value of ulValue, even if the task being notified had not yet processed the previous notification at the same array index (the task already had a notification pending at that index). xTaskNotifyIndexed() always returns pdPASS in this case.

eSetValueWithoutOverwrite - If the task being notified did not already have a notification pending at the same array index then the target notification value is set to ulValue and xTaskNotifyIndexed() will return pdPASS. If the task being notified already had a notification pending at the same array index then no action is performed and pdFAIL is returned.

eNoAction - The task receives a notification at the specified array index without the notification value at that index being updated. ulValue is not used and xTaskNotifyIndexed() always returns pdPASS in this case.

pulPreviousNotificationValue - Can be used to pass out the subject task's notification value before any bits are modified by the notify function.

Parameters:

- **xTaskToNotify** -- The handle of the task being notified. The handle to a task can be returned from the xTaskCreate() API function used to create the task, and the handle of the currently running task can be obtained by calling xTaskGetCurrentTaskHandle().
- **uxIndexToNotify** -- The index within the target task's array of notification values to which the notification is to be sent. uxIndexToNotify must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. xTaskNotify() does not have this parameter and always sends notifications to index 0.
- **ulValue** -- Data that can be sent with the notification. How the data is used depends on the value of the eAction parameter.
- **eAction** -- Specifies how the notification updates the task's notification value, if at all. Valid values for eAction are as follows:

Returns: Dependent on the value of eAction. See the description of the eAction parameter.

`xTaskNotifyAndQueryIndexed`(xTaskToNotify, uxIndexToNotify, ulValue, eAction, pulPreviousNotifyValue)

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

xTaskNotifyAndQueryIndexed() performs the same operation as xTaskNotifyIndexed() with the addition that it also returns the subject task's prior notification value (the notification value at the time the function is called rather than when the function returns) in the additional pulPreviousNotifyValue parameter.

xTaskNotifyAndQuery() performs the same operation as xTaskNotify() with the addition that it also returns the subject task's prior notification value (the notification value as it was at the time the function is called, rather than when the function returns) in the additional pulPreviousNotifyValue parameter.

`xTaskNotifyIndexedFromISR`(xTaskToNotify, uxIndexToNotify, ulValue, eAction, pxHigherPriorityTaskWoken)

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for these functions to be available.

A version of xTaskNotifyIndexed() that can be used from an interrupt service routine (ISR).

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

Events can be sent to a task using an intermediary object. Examples of such objects are queues, semaphores, mutexes and event groups. Task notifications are a method of sending an event directly to a task without the need for such an intermediary object.

A notification sent to a task can optionally perform an action, such as update, overwrite or increment one of the task's notification values. In that way task notifications can be used to send data to a task, or be used as light weight and fast binary or counting semaphores.

A task can use xTaskNotifyWaitIndexed() to [optionally] block to wait for a notification to be pending, or ulTaskNotifyTakeIndexed() to [optionally] block to wait for a notification value to have a non-zero value. The task does not consume any CPU time while it is in the Blocked state.

A notification sent to a task will remain pending until it is cleared by the task calling xTaskNotifyWaitIndexed() or ulTaskNotifyTakeIndexed() (or their un-indexed equivalents). If the task was already in the Blocked state to wait for a notification when the notification arrives then the task will automatically be removed from the Blocked state (unblocked) and the notification cleared.

NOTE Each notification within the array operates independently - a task can only block on one notification within the array at a time and will not be unblocked by a notification sent to any other array index.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. xTaskNotifyFromISR() is the original API function, and remains backward compatible by

always operating on the notification value at index 0 within the array. Calling xTaskNotifyFromISR() is equivalent to calling xTaskNotifyIndexedFromISR() with the uxIndexToNotify parameter set to 0.

eSetBits - The task's notification value is bitwise ORed with ulValue. xTaskNotify() always returns pdPASS in this case.

eIncrement - The task's notification value is incremented. ulValue is not used and xTaskNotify() always returns pdPASS in this case.

eSetValueWithOverwrite - The task's notification value is set to the value of ulValue, even if the task being notified had not yet processed the previous notification (the task already had a notification pending). xTaskNotify() always returns pdPASS in this case.

eSetValueWithoutOverwrite - If the task being notified did not already have a notification pending then the task's notification value is set to ulValue and xTaskNotify() will return pdPASS. If the task being notified already had a notification pending then no action is performed and pdFAIL is returned.

eNoAction - The task receives a notification without its notification value being updated. ulValue is not used and xTaskNotify() always returns pdPASS in this case.

Parameters:

- **uxIndexToNotify** -- The index within the target task's array of notification values to which the notification is to be sent. uxIndexToNotify must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. xTaskNotifyFromISR() does not have this parameter and always sends notifications to index 0.
- **xTaskToNotify** -- The handle of the task being notified. The handle to a task can be returned from the xTaskCreate() API function used to create the task, and the handle of the currently running task can be obtained by calling xTaskGetCurrentTaskHandle().
- **ulValue** -- Data that can be sent with the notification. How the data is used depends on the value of the eAction parameter.
- **eAction** -- Specifies how the notification updates the task's notification value, if at all. Valid values for eAction are as follows:
- **pxHigherPriorityTaskWoken** -- xTaskNotifyFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending the notification caused the task to which the notification was sent to leave the Blocked state, and the unblocked task has a priority higher than the currently running task. If xTaskNotifyFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited. How a context switch is requested from an ISR is dependent on the port - see the documentation page for the port in use.

Returns: Dependent on the value of eAction. See the description of the eAction parameter.

xTaskNotifyAndQueryIndexedFromISR(xTaskToNotify, uxIndexToNotify, ulValue, eAction, pulPreviousNotificationValue, pxHigherPriorityTaskWoken)

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

xTaskNotifyAndQueryIndexedFromISR() performs the same operation as xTaskNotifyIndexedFromISR() with the addition that it also returns the subject task's prior notification value (the notification value at the time the function is called rather than at the time the function returns) in the additional pulPreviousNotifyValue parameter.

xTaskNotifyAndQueryFromISR() performs the same operation as xTaskNotifyFromISR() with the addition that it also returns the subject task's prior notification value (the notification value at the time the function is called rather than at the time the function returns) in the additional pulPreviousNotifyValue parameter.

xTaskNotifyWait(ulBitsToClearOnEntry, ulBitsToClearOnExit, pulNotificationValue, xTicksToWait)

xTaskNotifyWaitIndexed(uxIndexToWaitOn, ulBitsToClearOnEntry, ulBitsToClearOnExit, pulNotificationValue, xTicksToWait)

xTaskNotifyGiveIndexed(xTaskToNotify, uxIndexToNotify)

Sends a direct to task notification to a particular index in the target task's notification array in a manner similar to giving a counting semaphore.

See https://www.FreeRTOS.org/RTOS-task-notifications.html for more details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for these macros to be available.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

Events can be sent to a task using an intermediary object. Examples of such objects are queues, semaphores, mutexes and event groups. Task notifications are a method of sending an event directly to a task without the need for such an intermediary object.

A notification sent to a task can optionally perform an action, such as update, overwrite or increment one of the task's notification values. In that way task notifications can be used to send data to a task, or be used as light weight and fast binary or counting semaphores.

xTaskNotifyGiveIndexed() is a helper macro intended for use when task notifications are used as light weight and faster binary or counting semaphore equivalents. Actual FreeRTOS semaphores are given using the xSemaphoreGive() API function, the equivalent

action that instead uses a task notification is xTaskNotifyGiveIndexed().

When task notifications are being used as a binary or counting semaphore equivalent then the task being notified should wait for the notification using the ulTaskNotifyTakeIndexed() API function rather than the xTaskNotifyWaitIndexed() API function.

NOTE Each notification within the array operates independently - a task can only block on one notification within the array at a time and will not be unblocked by a notification sent to any other array index.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. xTaskNotifyGive() is the original API function, and remains backward compatible by always operating on the notification value at index 0 in the array. Calling xTaskNotifyGive() is equivalent to calling xTaskNotifyGiveIndexed() with the uxIndexToNotify parameter set to 0.

Parameters:

- **xTaskToNotify** -- The handle of the task being notified. The handle to a task can be returned from the xTaskCreate() API function used to create the task, and the handle of the currently running task can be obtained by calling xTaskGetCurrentTaskHandle().
- **uxIndexToNotify** -- The index within the target task's array of notification values to which the notification is to be sent. uxIndexToNotify must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. xTaskNotifyGive() does not have this parameter and always sends notifications to index 0.

Returns: xTaskNotifyGive() is a macro that calls xTaskNotify() with the eAction parameter set to eIncrement - so pdPASS is always returned.

`vTaskNotifyGiveFromISR`(xTaskToNotify, pxHigherPriorityTaskWoken)

`vTaskNotifyGiveIndexedFromISR`(xTaskToNotify, uxIndexToNotify, pxHigherPriorityTaskWoken)

`ulTaskNotifyTakeIndexed`(uxIndexToWaitOn, xClearCountOnExit, xTicksToWait)

Waits for a direct to task notification on a particular index in the calling task's notification array in a manner similar to taking a counting semaphore.

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for this function to be available.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

Events can be sent to a task using an intermediary object. Examples of such objects are queues, semaphores, mutexes and event groups. Task notifications are a method of sending an event directly to a task without the need for such an intermediary object.

A notification sent to a task can optionally perform an action, such as update, overwrite or increment one of the task's notification values. In that way task notifications can be used to send data to a task, or be used as light weight and fast binary or counting semaphores.

ulTaskNotifyTakeIndexed() is intended for use when a task notification is used as a faster and lighter weight binary or counting semaphore alternative. Actual FreeRTOS semaphores are taken using the xSemaphoreTake() API function, the equivalent action that instead uses a task notification is ulTaskNotifyTakeIndexed().

When a task is using its notification value as a binary or counting semaphore other tasks should send notifications to it using the xTaskNotifyGiveIndexed() macro, or xTaskNotifyIndex() function with the eAction parameter set to eIncrement.

ulTaskNotifyTakeIndexed() can either clear the task's notification value at the array index specified by the uxIndexToWaitOn parameter to zero on exit, in which case the notification value acts like a binary semaphore, or decrement the notification value on exit, in which case the notification value acts like a counting semaphore.

A task can use ulTaskNotifyTakeIndexed() to [optionally] block to wait for a notification. The task does not consume any CPU time while it is in the Blocked state.

Where as xTaskNotifyWaitIndexed() will return when a notification is pending, ulTaskNotifyTakeIndexed() will return when the task's notification value is not zero.

NOTE Each notification within the array operates independently - a task can only block on one notification within the array at a time and will not be unblocked by a notification sent to any other array index.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. ulTaskNotifyTake() is the original API function, and remains backward compatible by always operating on the notification value at index 0 in the array. Calling ulTaskNotifyTake() is equivalent to calling ulTaskNotifyTakeIndexed() with the uxIndexToWaitOn parameter set to 0.

Parameters:

- **uxIndexToWaitOn** -- The index within the calling task's array of notification values on which the calling task will wait for a notification to be non-zero. uxIndexToWaitOn must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. xTaskNotifyTake() does not have this parameter and always waits for notifications on index 0.
- **xClearCountOnExit** -- if xClearCountOnExit is pdFALSE then the task's notification value is decremented when the function exits. In this way the notification value acts like a counting semaphore. If xClearCountOnExit is not pdFALSE then the task's notification value is cleared to zero when the function exits. In this way the notification value acts like a binary semaphore.
- **xTicksToWait** -- The maximum amount of time that the task should wait in the Blocked state for the task's notification value to be greater than zero, should the count not already be greater than zero when ulTaskNotifyTake() was called. The task will not consume any processing time while it is in the Blocked state. This is specified in kernel ticks, the macro pdMS_TO_TICKS(value_in_ms) can be used to convert a time specified in milliseconds to a time specified in ticks.

Returns: The task's notification count before it is either cleared to zero or decremented (see the xClearCountOnExit parameter).

`xTaskNotifyStateClear`(xTask)

`xTaskNotifyStateClearIndexed`(xTask, uxIndexToClear)

`ulTaskNotifyValueClear`(xTask, ulBitsToClear)

`ulTaskNotifyValueClearIndexed`(xTask, uxIndexToClear, ulBitsToClear)

Type Definitions

---

typedef struct tskTaskControlBlock ***`TaskHandle_t`**

---

typedef BaseType_t (***`TaskHookFunction_t`**)(void*)

Defines the prototype to which the application task hook function must conform.

---

typedef struct xTASK_STATUS **`TaskStatus_t`**

Used with the uxTaskGetSystemState() function to return the state of each task in the system.

Enumerations

enum **eTaskState**

Task states returned by eTaskGetState.

Values:

enumerator **eRunning**

A task is querying the state of itself, so must be running.

enumerator **eReady**

The task being queried is in a ready or pending ready list.

enumerator **eBlocked**

The task being queried is in the Blocked state.

enumerator **eSuspended**

The task being queried is in the Suspended state, or is in the Blocked state with an infinite time out.

enumerator **eDeleted**

The task being queried has been deleted, but its TCB has not yet been freed.

enumerator **eInvalid**

Used as an 'invalid state' value.

enum **eNotifyAction**

Actions that can be performed when vTaskNotify() is called.

Values:

enumerator **eNoAction**

Notify the task without updating its notify value.

enumerator **eSetBits**

Set bits in the task's notification value.

enumerator **eIncrement**

Increment the task's notification value.

enumerator **eSetValueWithOverwrite**

Set the task's notification value to a specific value even if the previous value has not yet been read by the task.

enumerator **eSetValueWithoutOverwrite**

Set the task's notification value if the previous value has been read by the task.

enum **eSleepModeStatus**

Possible return values for eTaskConfirmSleepModeStatus().

Values:

enumerator **eAbortSleep**

A task has been made ready or a context switch pended since portSUPPRESS_TICKS_AND_SLEEP() was called - abort entering a sleep mode.

enumerator **eStandardSleep**

Enter a sleep mode that will not last any longer than the expected idle time.

Queue API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/queue.h
- This header file can be included with:

```
#include "freertos/queue.h"
```

Functions

BaseType_t **xQueueGenericSend**(QueueHandle_t xQueue, *const* void **const* pvItemToQueue, TickType_t xTicksToWait, *const* BaseType_t xCopyPosition)

It is preferred that the macros xQueueSend(), xQueueSendToFront() and xQueueSendToBack() are used in place of calling this function directly.

Post an item on a queue. The item is queued by copy, not by reference. This function must not be called from an interrupt service routine. See xQueueSendFromISR () for an alternative which may be used in an ISR.

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
} xMessage;

uint32_t ulVar = 10UL;

void vATask( void *pvParameters )
{
QueueHandle_t xQueue1, xQueue2;
struct AMessage *pxMessage;

 // Create a queue capable of containing 10 uint32_t values.
 xQueue1 = xQueueCreate( 10, sizeof( uint32_t ) );

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue2 = xQueueCreate( 10, sizeof( struct AMessage * ) );

 // ...

 if( xQueue1 != 0 )
 {
// Send an uint32_t.  Wait for 10 ticks for space to become
// available if necessary.
if( xQueueGenericSend( xQueue1, ( void * ) &ulVar, ( TickType_t ) 10,
queueSEND_TO_BACK ) != pdPASS )
     {
// Failed to post the message, even after 10 ticks.
     }
 }

 if( xQueue2 != 0 )
 {
// Send a pointer to a struct AMessage object.  Don't block if the
// queue is already full.
     pxMessage = & xMessage;
     xQueueGenericSend( xQueue2, ( void * ) &pxMessage, ( TickType_t ) 0,
queueSEND_TO_BACK );
 }

 // ... Rest of task code.
}
```

Parameters:

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **xTicksToWait** -- The maximum amount of time the task should block waiting for space to become available on the queue, should it already be full. The call will return immediately if this is set to 0 and

the queue is full. The time is defined in tick periods so the constant portTICK_PERIOD_MS should be used to convert to real time if this is required.

- **xCopyPosition** -- Can take the value queueSEND_TO_BACK to place the item at the back of the queue, or queueSEND_TO_FRONT to place the item at the front of the queue (for high priority messages).

Returns: pdTRUE if the item was successfully posted, otherwise errQUEUE_FULL.

BaseType_t **xQueuePeek**(QueueHandle_t xQueue, void *const pvBuffer, TickType_t xTicksToWait)

Receive an item from a queue without removing the item from the queue. The item is received by copy so a buffer of adequate size must be provided. The number of bytes copied into the buffer was defined when the queue was created.

Successfully received items remain on the queue so will be returned again by the next call, or a call to xQueueReceive().

This macro must not be used in an interrupt service routine. See xQueuePeekFromISR() for an alternative that can be called from an interrupt service routine.

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
} xMessage;

QueueHandle_t xQueue;

// Task to create a queue and post a value.
void vATask( void *pvParameters )
{
struct AMessage *pxMessage;

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue = xQueueCreate( 10, sizeof( struct AMessage * ) );
 if( xQueue == 0 )
 {
// Failed to create the queue.
 }

 // ...

 // Send a pointer to a struct AMessage object.  Don't block if the
 // queue is already full.
 pxMessage = & xMessage;
 xQueueSend( xQueue, ( void * ) &pxMessage, ( TickType_t ) 0 );

 // ... Rest of task code.
}

// Task to peek the data from the queue.
void vADifferentTask( void *pvParameters )
{
struct AMessage *pxRxedMessage;

 if( xQueue != 0 )
 {
// Peek a message on the created queue.  Block for 10 ticks if a
// message is not immediately available.
if( xQueuePeek( xQueue, &( pxRxedMessage ), ( TickType_t ) 10 ) )
     {
// pcRxedMessage now points to the struct AMessage variable posted
// by vATask, but the item still remains on the queue.
     }
 }

 // ... Rest of task code.
}
```

Parameters:

- **xQueue** -- The handle to the queue from which the item is to be received.
- **pvBuffer** -- Pointer to the buffer into which the received item will be copied.
- **xTicksToWait** -- The maximum amount of time the task should block waiting for an item to receive should the queue be empty at the time of the call. The time is defined in tick periods so the constant

portTICK_PERIOD_MS should be used to convert to real time if this is required. xQueuePeek() will return immediately if xTicksToWait is 0 and the queue is empty.

Returns: pdTRUE if an item was successfully received from the queue, otherwise pdFALSE.

BaseType_t **xQueuePeekFromISR**(QueueHandle_t xQueue, void *const pvBuffer)

A version of xQueuePeek() that can be called from an interrupt service routine (ISR).

Receive an item from a queue without removing the item from the queue. The item is received by copy so a buffer of adequate size must be provided. The number of bytes copied into the buffer was defined when the queue was created.

Successfully received items remain on the queue so will be returned again by the next call, or a call to xQueueReceive().

Parameters:
- **xQueue** -- The handle to the queue from which the item is to be received.
- **pvBuffer** -- Pointer to the buffer into which the received item will be copied.

Returns: pdTRUE if an item was successfully received from the queue, otherwise pdFALSE.

BaseType_t **xQueueReceive**(QueueHandle_t xQueue, void *const pvBuffer, TickType_t xTicksToWait)

Receive an item from a queue. The item is received by copy so a buffer of adequate size must be provided. The number of bytes copied into the buffer was defined when the queue was created.

Successfully received items are removed from the queue.

This function must not be used in an interrupt service routine. See xQueueReceiveFromISR for an alternative that can.

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
} xMessage;

QueueHandle_t xQueue;

// Task to create a queue and post a value.
void vATask( void *pvParameters )
{
struct AMessage *pxMessage;

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue = xQueueCreate( 10, sizeof( struct AMessage * ) );
 if( xQueue == 0 )
 {
// Failed to create the queue.
 }

 // ...

 // Send a pointer to a struct AMessage object.  Don't block if the
 // queue is already full.
 pxMessage = & xMessage;
 xQueueSend( xQueue, ( void * ) &pxMessage, ( TickType_t ) 0 );

 // ... Rest of task code.
}

// Task to receive from the queue.
void vADifferentTask( void *pvParameters )
{
struct AMessage *pxRxedMessage;

 if( xQueue != 0 )
 {
// Receive a message on the created queue.  Block for 10 ticks if a
// message is not immediately available.
if( xQueueReceive( xQueue, &( pxRxedMessage ), ( TickType_t ) 10 ) )
     {
// pcRxedMessage now points to the struct AMessage variable posted
// by vATask.
     }
 }

 // ... Rest of task code.
}
```

Parameters:

- **xQueue** -- The handle to the queue from which the item is to be received.
- **pvBuffer** -- Pointer to the buffer into which the received item will be copied.
- **xTicksToWait** -- The maximum amount of time the task should block waiting for an item to receive should the queue be empty at the time of the call. xQueueReceive() will return immediately if

xTicksToWait is zero and the queue is empty. The time is defined in tick periods so the constant portTICK_PERIOD_MS should be used to convert to real time if this is required.

Returns: pdTRUE if an item was successfully received from the queue, otherwise pdFALSE.

UBaseType_t **uxQueueMessagesWaiting**(*const* QueueHandle_t xQueue)

Return the number of messages stored in a queue.

Parameters: **xQueue** -- A handle to the queue being queried.

Returns: The number of messages available in the queue.

UBaseType_t **uxQueueSpacesAvailable**(*const* QueueHandle_t xQueue)

Return the number of free spaces available in a queue. This is equal to the number of items that can be sent to the queue before the queue becomes full if no items are removed.

Parameters: **xQueue** -- A handle to the queue being queried.

Returns: The number of spaces available in the queue.

---

void **vQueueDelete**(QueueHandle_t xQueue)

Delete a queue - freeing all the memory allocated for storing of items placed on the queue.

Parameters: **xQueue** -- A handle to the queue to be deleted.

BaseType_t **xQueueGenericSendFromISR**(QueueHandle_t xQueue, *const* void **const* pvItemToQueue, BaseType_t **const* pxHigherPriorityTaskWoken, *const* BaseType_t xCopyPosition)

It is preferred that the macros xQueueSendFromISR(), xQueueSendToFrontFromISR() and xQueueSendToBackFromISR() be used in place of calling this function directly. xQueueGiveFromISR() is an equivalent for use by semaphores that don't actually copy any data.

Post an item on a queue. It is safe to use this function from within an interrupt service routine.

Items are queued by copy not reference so it is preferable to only queue small items, especially when called from an ISR. In most cases it would be preferable to store a pointer to the item being queued.

Example usage for buffered IO (where the ISR can obtain more than one value per call):

```
void vBufferISR( void )
{
char cIn;
BaseType_t xHigherPriorityTaskWokenByPost;

 // We have not woken a task at the start of the ISR.
 xHigherPriorityTaskWokenByPost = pdFALSE;

 // Loop until the buffer is empty.
 do
 {
// Obtain a byte from the buffer.
     cIn = portINPUT_BYTE( RX_REGISTER_ADDRESS );

// Post each byte.
     xQueueGenericSendFromISR( xRxQueue, &cIn, &xHigherPriorityTaskWokenByPost,
queueSEND_TO_BACK );

 } while( portINPUT_BYTE( BUFFER_COUNT ) );

 // Now the buffer is empty we can switch context if necessary.  Note that the
 // name of the yield function required is port specific.
 if( xHigherPriorityTaskWokenByPost )
 {
     portYIELD_FROM_ISR();
 }
}
```

Parameters:

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **pxHigherPriorityTaskWoken** -- xQueueGenericSendFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending to the queue caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xQueueGenericSendFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.
- **xCopyPosition** -- Can take the value queueSEND_TO_BACK to place the item at the back of the queue, or queueSEND_TO_FRONT to place the item at the front of the queue (for high priority messages).

Returns: pdTRUE if the data was successfully sent to the queue, otherwise errQUEUE_FULL.

BaseType_t **xQueueGiveFromISR**(QueueHandle_t xQueue, BaseType_t *const pxHigherPriorityTaskWoken)

BaseType_t **xQueueReceiveFromISR**(QueueHandle_t xQueue, void *const pvBuffer, BaseType_t *const pxHigherPriorityTaskWoken)

Receive an item from a queue. It is safe to use this function from within an interrupt service routine.

Example usage:

```
QueueHandle_t xQueue;

// Function to create a queue and post some values.
void vAFunction( void *pvParameters )
{
char cValueToPost;
const TickType_t xTicksToWait = ( TickType_t )0xff;

 // Create a queue capable of containing 10 characters.
 xQueue = xQueueCreate( 10, sizeof( char ) );
 if( xQueue == 0 )
 {
// Failed to create the queue.
 }

 // ...

 // Post some characters that will be used within an ISR.  If the queue
 // is full then this task will block for xTicksToWait ticks.
 cValueToPost = 'a';
 xQueueSend( xQueue, ( void * ) &cValueToPost, xTicksToWait );
 cValueToPost = 'b';
 xQueueSend( xQueue, ( void * ) &cValueToPost, xTicksToWait );

 // ... keep posting characters ... this task may block when the queue
 // becomes full.

 cValueToPost = 'c';
 xQueueSend( xQueue, ( void * ) &cValueToPost, xTicksToWait );
}

// ISR that outputs all the characters received on the queue.
void vISR_Routine( void )
{
BaseType_t xTaskWokenByReceive = pdFALSE;
char cRxedChar;

 while( xQueueReceiveFromISR( xQueue, ( void * ) &cRxedChar, &xTaskWokenByReceive)
)
 {
// A character was received.  Output the character now.
     vOutputCharacter( cRxedChar );

// If removing the character from the queue woke the task that was
// posting onto the queue xTaskWokenByReceive will have been set to
// pdTRUE.  No matter how many times this loop iterates only one
// task will be woken.
 }

 if( xTaskWokenByReceive != ( char ) pdFALSE;
 {
     taskYIELD ();
 }
}
```

Parameters:

- **xQueue** -- The handle to the queue from which the item is to be received.
- **pvBuffer** -- Pointer to the buffer into which the received item will be copied.

- **pxHigherPriorityTaskWoken** -- A task may be blocked waiting for space to become available on the queue. If xQueueReceiveFromISR causes such a task to unblock *pxTaskWoken will get set to pdTRUE, otherwise *pxTaskWoken will remain unchanged.

Returns: pdTRUE if an item was successfully received from the queue, otherwise pdFALSE.

BaseType_t **xQueueIsQueueEmptyFromISR**(*const* QueueHandle_t xQueue)

Queries a queue to determine if the queue is empty. This function should only be used in an ISR.

Parameters: **xQueue** -- The handle of the queue being queried
Returns: pdFALSE if the queue is not empty, or pdTRUE if the queue is empty.

BaseType_t **xQueueIsQueueFullFromISR**(*const* QueueHandle_t xQueue)

Queries a queue to determine if the queue is full. This function should only be used in an ISR.

Parameters: **xQueue** -- The handle of the queue being queried
Returns: pdFALSE if the queue is not full, or pdTRUE if the queue is full.

UBaseType_t **uxQueueMessagesWaitingFromISR**(*const* QueueHandle_t xQueue)

A version of uxQueueMessagesWaiting() that can be called from an ISR. Return the number of messages stored in a queue.

Parameters: **xQueue** -- A handle to the queue being queried.
Returns: The number of messages available in the queue.

---

void **vQueueAddToRegistry**(QueueHandle_t xQueue, *const* char *pcQueueName)

The registry is provided as a means for kernel aware debuggers to locate queues, semaphores and mutexes. Call vQueueAddToRegistry() add a queue, semaphore or mutex handle to the registry if you want the handle to be available to a kernel aware debugger. If you are not using a kernel aware debugger then this function can be ignored.

configQUEUE_REGISTRY_SIZE defines the maximum number of handles the registry can hold. configQUEUE_REGISTRY_SIZE must be greater than 0 within FreeRTOSConfig.h for the registry to be available. Its value does not affect the number of queues, semaphores and mutexes that can be created - just the number that the registry can hold.

If vQueueAddToRegistry is called more than once with the same xQueue parameter, the registry will store the pcQueueName parameter from the most recent call to vQueueAddToRegistry.

Parameters:

- **xQueue** -- The handle of the queue being added to the registry. This is the handle returned by a call to xQueueCreate(). Semaphore and mutex handles can also be passed in here.
- **pcQueueName** -- The name to be associated with the handle. This is the name that the kernel aware debugger will display. The queue registry only stores a pointer to the string - so the string must be persistent (global or preferably in ROM/Flash), not on the stack.

---

void **vQueueUnregisterQueue**(QueueHandle_t xQueue)

The registry is provided as a means for kernel aware debuggers to locate queues, semaphores and mutexes. Call vQueueAddToRegistry() add a queue, semaphore or mutex handle to the registry if you want the handle to be available to a kernel aware debugger, and vQueueUnregisterQueue() to remove the queue, semaphore or mutex from the register. If you are not using a kernel aware debugger then this function can be ignored.

Parameters: **xQueue** -- The handle of the queue being removed from the registry.

---

const char ***pcQueueGetName**(QueueHandle_t xQueue)

The queue registry is provided as a means for kernel aware debuggers to locate queues, semaphores and mutexes. Call pcQueueGetName() to look up and return the name of a queue in the queue registry from the queue's handle.

Parameters: **xQueue** -- The handle of the queue the name of which will be returned.
Returns: If the queue is in the registry then a pointer to the name of the queue is returned. If the queue is not in the registry then NULL is returned.

QueueSetHandle_t **xQueueCreateSet**(*const* UBaseType_t uxEventQueueLength)

Queue sets provide a mechanism to allow a task to block (pend) on a read operation from multiple queues or semaphores simultaneously.

See FreeRTOS/Source/Demo/Common/Minimal/QueueSet.c for an example using this function.

A queue set must be explicitly created using a call to xQueueCreateSet() before it can be used. Once created, standard FreeRTOS queues and semaphores can be added to the set using calls to xQueueAddToSet(). xQueueSelectFromSet() is then used to determine which, if any, of the queues or semaphores contained in the set is in a state where a queue read or semaphore take operation would be successful.

Note 1: See the documentation on https://www.freertos.org/Documentation/02-Kernel/04-API-references/07-Queue-sets/00-RTOS-queue-sets for reasons why queue sets are very rarely needed in practice as there are simpler methods of blocking on multiple objects.

Note 2: Blocking on a queue set that contains a mutex will not cause the mutex holder to inherit the priority of the blocked task.

Note 3: An additional 4 bytes of RAM is required for each space in a every queue added to a queue set. Therefore counting semaphores that have a high maximum count value should not be added to a queue set.

Note 4: A receive (in the case of a queue) or take (in the case of a semaphore) operation must not be performed on a member of a queue set unless a call to xQueueSelectFromSet() has first returned a handle to that set member.

Parameters: **uxEventQueueLength** -- Queue sets store events that occur on the queues and semaphores contained in the set. uxEventQueueLength specifies the maximum number of events that can be queued at once. To be absolutely certain that events are not lost uxEventQueueLength should be set to the total sum of the length of the queues added to the set, where binary semaphores and mutexes have a length of 1, and counting semaphores have a length set by their maximum count value. Examples:

- If a queue set is to hold a queue of length 5, another queue of length 12, and a binary semaphore, then uxEventQueueLength should be set to (5 + 12 + 1), or 18.
- If a queue set is to hold three binary semaphores then uxEventQueueLength should be set to (1 + 1 + 1), or 3.
- If a queue set is to hold a counting semaphore that has a maximum count of 5, and a counting semaphore that has a maximum count of 3, then uxEventQueueLength should be set to (5 + 3), or 8.

Returns: If the queue set is created successfully then a handle to the created queue set is returned. Otherwise NULL is returned.

BaseType_t **xQueueAddToSet**(QueueSetMemberHandle_t xQueueOrSemaphore, QueueSetHandle_t xQueueSet)

Adds a queue or semaphore to a queue set that was previously created by a call to xQueueCreateSet().

See FreeRTOS/Source/Demo/Common/Minimal/QueueSet.c for an example using this function.

Note 1: A receive (in the case of a queue) or take (in the case of a semaphore) operation must not be performed on a member of a queue set unless a call to xQueueSelectFromSet() has first returned a handle to that set member.

Parameters:

- **xQueueOrSemaphore** -- The handle of the queue or semaphore being added to the queue set (cast to an QueueSetMemberHandle_t type).
- **xQueueSet** -- The handle of the queue set to which the queue or semaphore is being added.

Returns: If the queue or semaphore was successfully added to the queue set then pdPASS is returned. If the queue could not be successfully added to the queue set because it is already a member of a different queue set then pdFAIL is returned.

BaseType_t **xQueueRemoveFromSet**(QueueSetMemberHandle_t xQueueOrSemaphore, QueueSetHandle_t xQueueSet)

Removes a queue or semaphore from a queue set. A queue or semaphore can only be removed from a set if the queue or semaphore is empty.

See FreeRTOS/Source/Demo/Common/Minimal/QueueSet.c for an example using this function.

Parameters:
- **xQueueOrSemaphore** -- The handle of the queue or semaphore being removed from the queue set (cast to an QueueSetMemberHandle_t type).
- **xQueueSet** -- The handle of the queue set in which the queue or semaphore is included.

Returns: If the queue or semaphore was successfully removed from the queue set then pdPASS is returned. If the queue was not in the queue set, or the queue (or semaphore) was not empty, then pdFAIL is returned.

QueueSetMemberHandle_t **xQueueSelectFromSet**(QueueSetHandle_t xQueueSet, *const* TickType_t xTicksToWait)

xQueueSelectFromSet() selects from the members of a queue set a queue or semaphore that either contains data (in the case of a queue) or is available to take (in the case of a semaphore). xQueueSelectFromSet() effectively allows a task to block (pend) on a read operation on all the queues and semaphores in a queue set simultaneously.

See FreeRTOS/Source/Demo/Common/Minimal/QueueSet.c for an example using this function.

Note 1: See the documentation on https://www.freertos.org/Documentation/02-Kernel/04-API-references/07-Queue-sets/00-RTOS-queue-sets for reasons why queue sets are very rarely needed in practice as there are simpler methods of blocking on multiple objects.

Note 2: Blocking on a queue set that contains a mutex will not cause the mutex holder to inherit the priority of the blocked task.

Note 3: A receive (in the case of a queue) or take (in the case of a semaphore) operation must not be performed on a member of a queue set unless a call to xQueueSelectFromSet() has first returned a handle to that set member.

Parameters:
- **xQueueSet** -- The queue set on which the task will (potentially) block.

- **xTicksToWait** -- The maximum time, in ticks, that the calling task will remain in the Blocked state (with other tasks executing) to wait for a member of the queue set to be ready for a successful queue read or semaphore take operation.

Returns: xQueueSelectFromSet() will return the handle of a queue (cast to a QueueSetMemberHandle_t type) contained in the queue set that contains data, or the handle of a semaphore (cast to a QueueSetMemberHandle_t type) contained in the queue set that is available, or NULL if no such queue or semaphore exists before before the specified block time expires.

QueueSetMemberHandle_t **`xQueueSelectFromSetFromISR`**(QueueSetHandle_t xQueueSet)

A version of xQueueSelectFromSet() that can be used from an ISR.

Macros

`xQueueCreate`(uxQueueLength, uxItemSize)

Creates a new queue instance, and returns a handle by which the new queue can be referenced.

Internally, within the FreeRTOS implementation, queues use two blocks of memory. The first block is used to hold the queue's data structures. The second block is used to hold items placed into the queue. If a queue is created using xQueueCreate() then both blocks of memory are automatically dynamically allocated inside the xQueueCreate() function. (see https://www.FreeRTOS.org/a00111.html). If a queue is created using xQueueCreateStatic() then the application writer must provide the memory that will get used by the queue. xQueueCreateStatic() therefore allows a queue to be created without using any dynamic memory allocation.

https://www.FreeRTOS.org/Embedded-RTOS-Queues.html

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
};

void vATask( void *pvParameters )
{
QueueHandle_t xQueue1, xQueue2;

 // Create a queue capable of containing 10 uint32_t values.
 xQueue1 = xQueueCreate( 10, sizeof( uint32_t ) );
 if( xQueue1 == 0 )
 {
// Queue was not created and must not be used.
 }

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue2 = xQueueCreate( 10, sizeof( struct AMessage * ) );
 if( xQueue2 == 0 )
 {
// Queue was not created and must not be used.
 }

 // ... Rest of task code.
}
```

Parameters:

- **uxQueueLength** -- The maximum number of items that the queue can contain.
- **uxItemSize** -- The number of bytes each item in the queue will require. Items are queued by copy, not by reference, so this is the number of bytes that will be copied for each posted item. Each item on the queue must be the same size.

Returns: If the queue is successfully create then a handle to the newly created queue is returned. If the queue cannot be created then 0 is returned.

`xQueueCreateStatic`(uxQueueLength, uxItemSize, pucQueueStorage, pxQueueBuffer)

Creates a new queue instance, and returns a handle by which the new queue can be referenced.

Internally, within the FreeRTOS implementation, queues use two blocks of memory. The first block is used to hold the queue's data structures. The second block is used to hold items placed into the queue. If a queue is created using xQueueCreate() then both blocks of memory are automatically dynamically allocated inside the xQueueCreate() function. (see https://www.FreeRTOS.org/a00111.html). If a queue is created using xQueueCreateStatic() then the application writer must provide the memory that will get used by the queue. xQueueCreateStatic() therefore allows a queue to be created without using any dynamic memory allocation.

https://www.FreeRTOS.org/Embedded-RTOS-Queues.html

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
};

#define QUEUE_LENGTH 10
#define ITEM_SIZE sizeof( uint32_t )

// xQueueBuffer will hold the queue structure.
StaticQueue_t xQueueBuffer;

// ucQueueStorage will hold the items posted to the queue.  Must be at least
// [(queue length) * ( queue item size)] bytes long.
uint8_t ucQueueStorage[ QUEUE_LENGTH * ITEM_SIZE ];

void vATask( void *pvParameters )
{
 QueueHandle_t xQueue1;

 // Create a queue capable of containing 10 uint32_t values.
 xQueue1 = xQueueCreate( QUEUE_LENGTH, // The number of items the queue can hold.
                         ITEM_SIZE     // The size of each item in the queue
                         &( ucQueueStorage[ 0 ] ), // The buffer that will hold the
items in the queue.
                         &xQueueBuffer ); // The buffer that will hold the queue
structure.

 // The queue is guaranteed to be created successfully as no dynamic memory
 // allocation is used.  Therefore xQueue1 is now a handle to a valid queue.

 // ... Rest of task code.
}
```

Parameters:

- **uxQueueLength** -- The maximum number of items that the queue can contain.
- **uxItemSize** -- The number of bytes each item in the queue will require. Items are queued by copy, not by reference, so this is the number of bytes that will be copied for each posted item. Each item on the queue must be the same size.
- **pucQueueStorage** -- If uxItemSize is not zero then pucQueueStorage must point to a uint8_t array that is at least large enough to hold the maximum number of items that can be in the queue at any one time - which is (uxQueueLength * uxItemsSize) bytes. If uxItemSize is zero then pucQueueStorage can be NULL.
- **pxQueueBuffer** -- Must point to a variable of type StaticQueue_t, which will be used to hold the queue's data structure.

Returns: If the queue is created then a handle to the created queue is returned. If pxQueueBuffer is NULL then NULL is returned.

`xQueueGetStaticBuffers`(xQueue, ppucQueueStorage, ppxStaticQueue)

Retrieve pointers to a statically created queue's data structure buffer and storage area buffer. These are the same buffers that are supplied at the time of creation.

Parameters:
- **xQueue** -- The queue for which to retrieve the buffers.
- **ppucQueueStorage** -- Used to return a pointer to the queue's storage area buffer.
- **ppxStaticQueue** -- Used to return a pointer to the queue's data structure buffer.

Returns: pdTRUE if buffers were retrieved, pdFALSE otherwise.

`xQueueSendToFront`(xQueue, pvItemToQueue, xTicksToWait)

Post an item to the front of a queue. The item is queued by copy, not by reference. This function must not be called from an interrupt service routine. See xQueueSendFromISR () for an alternative which may be used in an ISR.

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
} xMessage;

uint32_t ulVar = 10UL;

void vATask( void *pvParameters )
{
QueueHandle_t xQueue1, xQueue2;
struct AMessage *pxMessage;

 // Create a queue capable of containing 10 uint32_t values.
 xQueue1 = xQueueCreate( 10, sizeof( uint32_t ) );

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue2 = xQueueCreate( 10, sizeof( struct AMessage * ) );

 // ...

 if( xQueue1 != 0 )
 {
// Send an uint32_t.  Wait for 10 ticks for space to become
// available if necessary.
if( xQueueSendToFront( xQueue1, ( void * ) &ulVar, ( TickType_t ) 10 ) != pdPASS )
     {
// Failed to post the message, even after 10 ticks.
     }
 }

 if( xQueue2 != 0 )
 {
// Send a pointer to a struct AMessage object.  Don't block if the
// queue is already full.
     pxMessage = & xMessage;
     xQueueSendToFront( xQueue2, ( void * ) &pxMessage, ( TickType_t ) 0 );
 }

 // ... Rest of task code.
}
```

Parameters:

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **xTicksToWait** -- The maximum amount of time the task should block waiting for space to become available on the queue, should it already be full. The call will return immediately if this is set to 0 and the queue is full. The time is defined in tick periods so the constant portTICK_PERIOD_MS should be used to convert to real time if this is required.

Returns: pdTRUE if the item was successfully posted, otherwise errQUEUE_FULL.

`xQueueSendToBack`(xQueue, pvItemToQueue, xTicksToWait)

This is a macro that calls xQueueGenericSend().

Post an item to the back of a queue. The item is queued by copy, not by reference. This function must not be called from an interrupt service routine. See xQueueSendFromISR () for an alternative which may be used in an ISR.

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
} xMessage;

uint32_t ulVar = 10UL;

void vATask( void *pvParameters )
{
QueueHandle_t xQueue1, xQueue2;
struct AMessage *pxMessage;

 // Create a queue capable of containing 10 uint32_t values.
 xQueue1 = xQueueCreate( 10, sizeof( uint32_t ) );

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue2 = xQueueCreate( 10, sizeof( struct AMessage * ) );

 // ...

 if( xQueue1 != 0 )
 {
// Send an uint32_t.  Wait for 10 ticks for space to become
// available if necessary.
if( xQueueSendToBack( xQueue1, ( void * ) &ulVar, ( TickType_t ) 10 ) != pdPASS )
     {
// Failed to post the message, even after 10 ticks.
     }
 }

 if( xQueue2 != 0 )
 {
// Send a pointer to a struct AMessage object.  Don't block if the
// queue is already full.
     pxMessage = & xMessage;
     xQueueSendToBack( xQueue2, ( void * ) &pxMessage, ( TickType_t ) 0 );
 }

 // ... Rest of task code.
}
```

Parameters:

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **xTicksToWait** -- The maximum amount of time the task should block waiting for space to become available on the queue, should it already be full. The call will return immediately if this is set to 0 and the queue is full. The time is defined in tick periods so the constant portTICK_PERIOD_MS should be used to convert to real time if this is required.

Returns: pdTRUE if the item was successfully posted, otherwise errQUEUE_FULL.

---

xQueueSend(xQueue, pvItemToQueue, xTicksToWait)

This is a macro that calls xQueueGenericSend(). It is included for backward compatibility with versions of FreeRTOS.org that did not include the xQueueSendToFront() and xQueueSendToBack() macros. It is equivalent to xQueueSendToBack().

Post an item on a queue. The item is queued by copy, not by reference. This function must not be called from an interrupt service routine. See xQueueSendFromISR () for an alternative which may be used in an ISR.

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
} xMessage;

uint32_t ulVar = 10UL;

void vATask( void *pvParameters )
{
QueueHandle_t xQueue1, xQueue2;
struct AMessage *pxMessage;

 // Create a queue capable of containing 10 uint32_t values.
 xQueue1 = xQueueCreate( 10, sizeof( uint32_t ) );

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue2 = xQueueCreate( 10, sizeof( struct AMessage * ) );

 // ...

 if( xQueue1 != 0 )
 {
// Send an uint32_t.  Wait for 10 ticks for space to become
// available if necessary.
if( xQueueSend( xQueue1, ( void * ) &ulVar, ( TickType_t ) 10 ) != pdPASS )
     {
// Failed to post the message, even after 10 ticks.
     }
 }

 if( xQueue2 != 0 )
 {
// Send a pointer to a struct AMessage object.  Don't block if the
// queue is already full.
     pxMessage = & xMessage;
     xQueueSend( xQueue2, ( void * ) &pxMessage, ( TickType_t ) 0 );
 }

 // ... Rest of task code.
}
```

Parameters:

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **xTicksToWait** -- The maximum amount of time the task should block waiting for space to become available on the queue, should it already be full. The call will return immediately if this is set to 0 and the queue is full. The time is defined in tick periods so the constant portTICK_PERIOD_MS should be used to convert to real time if this is required.

Returns: pdTRUE if the item was successfully posted, otherwise errQUEUE_FULL.

`xQueueOverwrite`(xQueue, pvItemToQueue)

Only for use with queues that have a length of one - so the queue is either empty or full.

Post an item on a queue. If the queue is already full then overwrite the value held in the queue. The item is queued by copy, not by reference.

This function must not be called from an interrupt service routine. See xQueueOverwriteFromISR () for an alternative which may be used in an ISR.

Example usage:

```
void vFunction( void *pvParameters )
{
QueueHandle_t xQueue;
uint32_t ulVarToSend, ulValReceived;

 // Create a queue to hold one uint32_t value.  It is strongly
 // recommended *not* to use xQueueOverwrite() on queues that can
 // contain more than one value, and doing so will trigger an assertion
 // if configASSERT() is defined.
 xQueue = xQueueCreate( 1, sizeof( uint32_t ) );

 // Write the value 10 to the queue using xQueueOverwrite().
 ulVarToSend = 10;
 xQueueOverwrite( xQueue, &ulVarToSend );

 // Peeking the queue should now return 10, but leave the value 10 in
 // the queue.  A block time of zero is used as it is known that the
 // queue holds a value.
 ulValReceived = 0;
 xQueuePeek( xQueue, &ulValReceived, 0 );

 if( ulValReceived != 10 )
 {
// Error unless the item was removed by a different task.
 }

 // The queue is still full.  Use xQueueOverwrite() to overwrite the
 // value held in the queue with 100.
 ulVarToSend = 100;
 xQueueOverwrite( xQueue, &ulVarToSend );

 // This time read from the queue, leaving the queue empty once more.
 // A block time of 0 is used again.
 xQueueReceive( xQueue, &ulValReceived, 0 );

 // The value read should be the last value written, even though the
 // queue was already full when the value was written.
 if( ulValReceived != 100 )
 {
// Error!
 }

 // ...
}
```

Parameters:

- **xQueue** -- The handle of the queue to which the data is being sent.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.

Returns: xQueueOverwrite() is a macro that calls xQueueGenericSend(), and therefore has the same return values as xQueueSendToFront(). However, pdPASS is the only value that can be returned because xQueueOverwrite() will write to the queue even when the queue is already full.

`xQueueSendToFrontFromISR`(xQueue, pvItemToQueue, pxHigherPriorityTaskWoken)

This is a macro that calls xQueueGenericSendFromISR().

Post an item to the front of a queue. It is safe to use this macro from within an interrupt service routine.

Items are queued by copy not reference so it is preferable to only queue small items, especially when called from an ISR. In most cases it would be preferable to store a pointer to the item being queued.

Example usage for buffered IO (where the ISR can obtain more than one value per call):

```
void vBufferISR( void )
{
char cIn;
BaseType_t xHigherPriorityTaskWoken;

 // We have not woken a task at the start of the ISR.
 xHigherPriorityTaskWoken = pdFALSE;

 // Loop until the buffer is empty.
 do
 {
// Obtain a byte from the buffer.
     cIn = portINPUT_BYTE( RX_REGISTER_ADDRESS );

// Post the byte.
     xQueueSendToFrontFromISR( xRxQueue, &cIn, &xHigherPriorityTaskWoken );

 } while( portINPUT_BYTE( BUFFER_COUNT ) );

 // Now the buffer is empty we can switch context if necessary.
 if( xHigherPriorityTaskWoken )
 {
     taskYIELD ();
 }
}
```

Parameters:

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **pxHigherPriorityTaskWoken** -- xQueueSendToFrontFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending to the queue caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xQueueSendToFromFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.

Returns: pdTRUE if the data was successfully sent to the queue, otherwise errQUEUE_FULL.

`xQueueSendToBackFromISR`(xQueue, pvItemToQueue, pxHigherPriorityTaskWoken)

This is a macro that calls xQueueGenericSendFromISR().

Post an item to the back of a queue. It is safe to use this macro from within an interrupt service routine.

Items are queued by copy not reference so it is preferable to only queue small items, especially when called from an ISR. In most cases it would be preferable to store a pointer to the item being queued.

Example usage for buffered IO (where the ISR can obtain more than one value per call):

```
void vBufferISR( void )
{
char cIn;
BaseType_t xHigherPriorityTaskWoken;

 // We have not woken a task at the start of the ISR.
 xHigherPriorityTaskWoken = pdFALSE;

 // Loop until the buffer is empty.
 do
 {
// Obtain a byte from the buffer.
     cIn = portINPUT_BYTE( RX_REGISTER_ADDRESS );

// Post the byte.
     xQueueSendToBackFromISR( xRxQueue, &cIn, &xHigherPriorityTaskWoken );

 } while( portINPUT_BYTE( BUFFER_COUNT ) );

 // Now the buffer is empty we can switch context if necessary.
 if( xHigherPriorityTaskWoken )
 {
     taskYIELD ();
 }
}
```

Parameters:

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **pxHigherPriorityTaskWoken** -- xQueueSendToBackFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending to the queue caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xQueueSendToBackFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.

Returns: pdTRUE if the data was successfully sent to the queue, otherwise errQUEUE_FULL.

`xQueueOverwriteFromISR`(xQueue, pvItemToQueue, pxHigherPriorityTaskWoken)

A version of xQueueOverwrite() that can be used in an interrupt service routine (ISR).

Only for use with queues that can hold a single item - so the queue is either empty or full.

Post an item on a queue. If the queue is already full then overwrite the value held in the queue. The item is queued by copy, not by reference.

Example usage:

```
QueueHandle_t xQueue;

void vFunction( void *pvParameters )
{
 // Create a queue to hold one uint32_t value.  It is strongly
 // recommended *not* to use xQueueOverwriteFromISR() on queues that can
 // contain more than one value, and doing so will trigger an assertion
 // if configASSERT() is defined.
 xQueue = xQueueCreate( 1, sizeof( uint32_t ) );
}

void vAnInterruptHandler( void )
{
// xHigherPriorityTaskWoken must be set to pdFALSE before it is used.
BaseType_t xHigherPriorityTaskWoken = pdFALSE;
uint32_t ulVarToSend, ulValReceived;

 // Write the value 10 to the queue using xQueueOverwriteFromISR().
 ulVarToSend = 10;
 xQueueOverwriteFromISR( xQueue, &ulVarToSend, &xHigherPriorityTaskWoken );

 // The queue is full, but calling xQueueOverwriteFromISR() again will still
 // pass because the value held in the queue will be overwritten with the
 // new value.
 ulVarToSend = 100;
 xQueueOverwriteFromISR( xQueue, &ulVarToSend, &xHigherPriorityTaskWoken );

 // Reading from the queue will now return 100.

 // ...

 if( xHigherPrioritytaskWoken == pdTRUE )
 {
// Writing to the queue caused a task to unblock and the unblocked task
// has a priority higher than or equal to the priority of the currently
// executing task (the task this interrupt interrupted).  Perform a context
// switch so this interrupt returns directly to the unblocked task.
     portYIELD_FROM_ISR(); // or portEND_SWITCHING_ISR() depending on the port.
 }
}
```

Parameters:

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **pxHigherPriorityTaskWoken** -- xQueueOverwriteFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending to the queue caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xQueueOverwriteFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.

Returns: xQueueOverwriteFromISR() is a macro that calls xQueueGenericSendFromISR(), and therefore has the same return values as xQueueSendToFrontFromISR(). However, pdPASS is the only value that can be returned because xQueueOverwriteFromISR() will write to the queue even when the queue is already full.

`xQueueSendFromISR`(xQueue, pvItemToQueue, pxHigherPriorityTaskWoken)

This is a macro that calls xQueueGenericSendFromISR(). It is included for backward compatibility with versions of FreeRTOS.org that did not include the xQueueSendToBackFromISR() and xQueueSendToFrontFromISR() macros.

Post an item to the back of a queue. It is safe to use this function from within an interrupt service routine.

Items are queued by copy not reference so it is preferable to only queue small items, especially when called from an ISR. In most cases it would be preferable to store a pointer to the item being queued.

Example usage for buffered IO (where the ISR can obtain more than one value per call):

```
void vBufferISR( void )
{
char cIn;
BaseType_t xHigherPriorityTaskWoken;

 // We have not woken a task at the start of the ISR.
 xHigherPriorityTaskWoken = pdFALSE;

 // Loop until the buffer is empty.
 do
 {
// Obtain a byte from the buffer.
     cIn = portINPUT_BYTE( RX_REGISTER_ADDRESS );

// Post the byte.
     xQueueSendFromISR( xRxQueue, &cIn, &xHigherPriorityTaskWoken );

 } while( portINPUT_BYTE( BUFFER_COUNT ) );

 // Now the buffer is empty we can switch context if necessary.
 if( xHigherPriorityTaskWoken )
 {
// Actual macro used here is port specific.
     portYIELD_FROM_ISR ();
 }
}
```

Parameters:

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from

pvItemToQueue into the queue storage area.

- **pxHigherPriorityTaskWoken** -- xQueueSendFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending to the queue caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xQueueSendFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.

Returns: pdTRUE if the data was successfully sent to the queue, otherwise errQUEUE_FULL.

---

xQueueReset(xQueue)

Reset a queue back to its original empty state. The return value is now obsolete and is always set to pdPASS.

Type Definitions

---

typedef struct QueueDefinition ***QueueHandle_t**

---

typedef struct QueueDefinition ***QueueSetHandle_t**

Type by which queue sets are referenced. For example, a call to xQueueCreateSet() returns an xQueueSet variable that can then be used as a parameter to xQueueSelectFromSet(), xQueueAddToSet(), etc.

---

typedef struct QueueDefinition ***QueueSetMemberHandle_t**

Queue sets can contain both queues and semaphores, so the QueueSetMemberHandle_t is defined as a type to be used where a parameter or return value can be either an QueueHandle_t or an SemaphoreHandle_t.

Semaphore API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/semphr.h
- This header file can be included with:

```
#include "freertos/semphr.h"
```

Macros

---

semBINARY_SEMAPHORE_QUEUE_LENGTH

semSEMAPHORE_QUEUE_ITEM_LENGTH

semGIVE_BLOCK_TIME

vSemaphoreCreateBinary(xSemaphore)

In many usage scenarios it is faster and more memory efficient to use a direct to task notification in place of a binary semaphore! https://www.FreeRTOS.org/RTOS-task-notifications.html

This old vSemaphoreCreateBinary() macro is now deprecated in favour of the xSemaphoreCreateBinary() function. Note that binary semaphores created using the vSemaphoreCreateBinary() macro are created in a state such that the first call to 'take' the semaphore would pass, whereas binary semaphores created using xSemaphoreCreateBinary() are created in a state such that the the semaphore must first be 'given' before it can be 'taken'.

Macro that implements a semaphore by using the existing queue mechanism. The queue length is 1 as this is a binary semaphore. The data size is 0 as we don't want to actually store any data - we just want to know if the queue is empty or full.

This type of semaphore can be used for pure synchronisation between tasks or between an interrupt and a task. The semaphore need not be given back once obtained, so one task/interrupt can continuously 'give' the semaphore while another continuously 'takes' the semaphore. For this reason this type of semaphore does not use a priority inheritance mechanism. For an alternative that does use priority inheritance see xSemaphoreCreateMutex().

Example usage:

```
SemaphoreHandle_t xSemaphore = NULL;

void vATask( void * pvParameters )
{
 // Semaphore cannot be used before a call to vSemaphoreCreateBinary ().
 // This is a macro so pass the variable in directly.
 vSemaphoreCreateBinary( xSemaphore );

 if( xSemaphore != NULL )
 {
// The semaphore was created successfully.
// The semaphore can now be used.
 }
}
```

Parameters:

- **xSemaphore** -- Handle to the created semaphore. Should be of type SemaphoreHandle_t.

xSemaphoreCreateBinary()

Creates a new binary semaphore instance, and returns a handle by which the new semaphore can be referenced.

In many usage scenarios it is faster and more memory efficient to use a direct to task notification in place of a binary semaphore! https://www.FreeRTOS.org/RTOS-task-notifications.html

Internally, within the FreeRTOS implementation, binary semaphores use a block of memory, in which the semaphore structure is stored. If a binary semaphore is created using xSemaphoreCreateBinary() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateBinary() function. (see https://www.FreeRTOS.org/a00111.html). If a binary semaphore is created using xSemaphoreCreateBinaryStatic() then the application writer must provide the memory. xSemaphoreCreateBinaryStatic() therefore allows a binary semaphore to be created without using any dynamic memory allocation.

The old vSemaphoreCreateBinary() macro is now deprecated in favour of this xSemaphoreCreateBinary() function. Note that binary semaphores created using the vSemaphoreCreateBinary() macro are created in a state such that the first call to 'take' the semaphore would pass, whereas binary semaphores created using xSemaphoreCreateBinary() are created in a state such that the the semaphore must first be 'given' before it can be 'taken'.

This type of semaphore can be used for pure synchronisation between tasks or between an interrupt and a task. The semaphore need not be given back once obtained, so one task/interrupt can continuously 'give' the semaphore while another continuously 'takes' the semaphore. For this reason this type of semaphore does not use a priority inheritance mechanism. For an alternative that does use priority inheritance see xSemaphoreCreateMutex().

Example usage:

```
SemaphoreHandle_t xSemaphore = NULL;

void vATask( void * pvParameters )
{
 // Semaphore cannot be used before a call to xSemaphoreCreateBinary().
 // This is a macro so pass the variable in directly.
 xSemaphore = xSemaphoreCreateBinary();

 if( xSemaphore != NULL )
 {
// The semaphore was created successfully.
// The semaphore can now be used.
 }
}
```

Returns: Handle to the created semaphore, or NULL if the memory required to hold the semaphore's data structures could not be allocated.

xSemaphoreCreateBinaryStatic(pxStaticSemaphore)

Creates a new binary semaphore instance, and returns a handle by which the new semaphore can be referenced.

NOTE: In many usage scenarios it is faster and more memory efficient to use a direct to task notification in place of a binary semaphore! https://www.FreeRTOS.org/RTOS-task-notifications.html

Internally, within the FreeRTOS implementation, binary semaphores use a block of memory, in which the semaphore structure is stored. If a binary semaphore is created using xSemaphoreCreateBinary() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateBinary() function. (see https://www.FreeRTOS.org/a00111.html). If a binary semaphore is created using xSemaphoreCreateBinaryStatic() then the application writer must provide the memory. xSemaphoreCreateBinaryStatic() therefore allows a binary semaphore to be created without using any dynamic memory allocation.

This type of semaphore can be used for pure synchronisation between tasks or between an interrupt and a task. The semaphore need not be given back once obtained, so one task/interrupt can continuously 'give' the semaphore while another continuously 'takes' the semaphore. For this reason this type of semaphore does not use a priority inheritance mechanism. For an alternative that does use priority inheritance see xSemaphoreCreateMutex().

Example usage:

```
SemaphoreHandle_t xSemaphore = NULL;
StaticSemaphore_t xSemaphoreBuffer;

void vATask( void * pvParameters )
{
 // Semaphore cannot be used before a call to xSemaphoreCreateBinary().
 // The semaphore's data structures will be placed in the xSemaphoreBuffer
 // variable, the address of which is passed into the function.  The
 // function's parameter is not NULL, so the function will not attempt any
 // dynamic memory allocation, and therefore the function will not return
 // return NULL.
 xSemaphore = xSemaphoreCreateBinary( &xSemaphoreBuffer );

 // Rest of task code goes here.
}
```

Parameters:
- **pxStaticSemaphore** -- Must point to a variable of type StaticSemaphore_t, which will then be used to hold the semaphore's data structure, removing the need for the memory to be allocated dynamically.

Returns: If the semaphore is created then a handle to the created semaphore is returned. If pxSemaphoreBuffer is NULL then NULL is returned.

`xSemaphoreTake`(xSemaphore, xBlockTime)

Macro to obtain a semaphore. The semaphore must have previously been created with a call to xSemaphoreCreateBinary(), xSemaphoreCreateMutex() or xSemaphoreCreateCounting().

Example usage:

```
SemaphoreHandle_t xSemaphore = NULL;

// A task that creates a semaphore.
void vATask( void * pvParameters )
{
 // Create the semaphore to guard a shared resource.
 xSemaphore = xSemaphoreCreateBinary();
}

// A task that uses the semaphore.
void vAnotherTask( void * pvParameters )
{
 // ... Do other things.

 if( xSemaphore != NULL )
 {
// See if we can obtain the semaphore.  If the semaphore is not available
// wait 10 ticks to see if it becomes free.
if( xSemaphoreTake( xSemaphore, ( TickType_t ) 10 ) == pdTRUE )
     {
// We were able to obtain the semaphore and can now access the
// shared resource.

// ...

// We have finished accessing the shared resource.  Release the
// semaphore.
         xSemaphoreGive( xSemaphore );
     }
else
     {
// We could not obtain the semaphore and can therefore not access
// the shared resource safely.
     }
 }
}
```

Parameters:

- **xSemaphore** -- A handle to the semaphore being taken - obtained when the semaphore was created.
- **xBlockTime** -- The time in ticks to wait for the semaphore to become available. The macro portTICK_PERIOD_MS can be used to convert this to a real time. A block time of zero can be used to poll the semaphore. A block time of portMAX_DELAY can be used to block indefinitely (provided INCLUDE_vTaskSuspend is set to 1 in FreeRTOSConfig.h).

Returns: pdTRUE if the semaphore was obtained. pdFALSE if xBlockTime expired without the semaphore becoming available.

`xSemaphoreTakeRecursive`(xMutex, xBlockTime)

Macro to recursively obtain, or 'take', a mutex type semaphore. The mutex must have previously been created using a call to xSemaphoreCreateRecursiveMutex();

configUSE_RECURSIVE_MUTEXES must be set to 1 in FreeRTOSConfig.h for this macro to be available.

This macro must not be used on mutexes created using xSemaphoreCreateMutex().

A mutex used recursively can be 'taken' repeatedly by the owner. The mutex doesn't become available again until the owner has called xSemaphoreGiveRecursive() for each successful 'take' request. For example, if a task successfully 'takes' the same mutex 5 times then the mutex will not be available to any other task until it has also 'given' the mutex back exactly five times.

Example usage:

```
SemaphoreHandle_t xMutex = NULL;

// A task that creates a mutex.
void vATask( void * pvParameters )
{
 // Create the mutex to guard a shared resource.
 xMutex = xSemaphoreCreateRecursiveMutex();
}

// A task that uses the mutex.
void vAnotherTask( void * pvParameters )
{
 // ... Do other things.

 if( xMutex != NULL )
 {
// See if we can obtain the mutex.  If the mutex is not available
// wait 10 ticks to see if it becomes free.
if( xSemaphoreTakeRecursive( xSemaphore, ( TickType_t ) 10 ) == pdTRUE )
     {
// We were able to obtain the mutex and can now access the
// shared resource.

// ...
// For some reason due to the nature of the code further calls to
// xSemaphoreTakeRecursive() are made on the same mutex.  In real
// code these would not be just sequential calls as this would make
// no sense.  Instead the calls are likely to be buried inside
// a more complex call structure.
         xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 );
         xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 );

// The mutex has now been 'taken' three times, so will not be
// available to another task until it has also been given back
// three times.  Again it is unlikely that real code would have
// these calls sequentially, but instead buried in a more complex
// call structure.  This is just for illustrative purposes.
         xSemaphoreGiveRecursive( xMutex );
         xSemaphoreGiveRecursive( xMutex );
         xSemaphoreGiveRecursive( xMutex );

// Now the mutex can be taken by other tasks.
     }
else
     {
// We could not obtain the mutex and can therefore not access
// the shared resource safely.
     }
 }
}
```

Parameters:

- **xMutex** -- A handle to the mutex being obtained. This is the handle returned by xSemaphoreCreateRecursiveMutex();
- **xBlockTime** -- The time in ticks to wait for the semaphore to become available. The macro portTICK_PERIOD_MS can be used to convert this to a real time. A block time of zero can be used to poll

the semaphore. If the task already owns the semaphore then xSemaphoreTakeRecursive() will return immediately no matter what the value of xBlockTime.

Returns: pdTRUE if the semaphore was obtained. pdFALSE if xBlockTime expired without the semaphore becoming available.

`xSemaphoreGive`(xSemaphore)

Macro to release a semaphore. The semaphore must have previously been created with a call to xSemaphoreCreateBinary(), xSemaphoreCreateMutex() or xSemaphoreCreateCounting(). and obtained using sSemaphoreTake().

This macro must not be used from an ISR. See xSemaphoreGiveFromISR () for an alternative which can be used from an ISR.

This macro must also not be used on semaphores created using xSemaphoreCreateRecursiveMutex().

Example usage:

```
SemaphoreHandle_t xSemaphore = NULL;

void vATask( void * pvParameters )
{
 // Create the semaphore to guard a shared resource.
 xSemaphore = vSemaphoreCreateBinary();

 if( xSemaphore != NULL )
 {
if( xSemaphoreGive( xSemaphore ) != pdTRUE )
     {
// We would expect this call to fail because we cannot give
// a semaphore without first "taking" it!
     }

// Obtain the semaphore - don't block if the semaphore is not
// immediately available.
if( xSemaphoreTake( xSemaphore, ( TickType_t ) 0 ) )
     {
// We now have the semaphore and can access the shared resource.

// ...

// We have finished accessing the shared resource so can free the
// semaphore.
if( xSemaphoreGive( xSemaphore ) != pdTRUE )
         {
// We would not expect this call to fail because we must have
// obtained the semaphore to get here.
         }
     }
 }
}
```

Parameters:
- **xSemaphore** -- A handle to the semaphore being released. This is the handle returned when the semaphore was created.

Returns: pdTRUE if the semaphore was released. pdFALSE if an error occurred. Semaphores are implemented using queues. An error can occur if there is no space on the queue to post a message - indicating that the semaphore was not first obtained correctly.

`xSemaphoreGiveRecursive`(xMutex)

Macro to recursively release, or 'give', a mutex type semaphore. The mutex must have previously been created using a call to xSemaphoreCreateRecursiveMutex();

configUSE_RECURSIVE_MUTEXES must be set to 1 in FreeRTOSConfig.h for this macro to be available.

This macro must not be used on mutexes created using xSemaphoreCreateMutex().

A mutex used recursively can be 'taken' repeatedly by the owner. The mutex doesn't become available again until the owner has called xSemaphoreGiveRecursive() for each successful 'take' request. For example, if a task successfully 'takes' the same mutex 5 times then the mutex will not be available to any other task until it has also 'given' the mutex back exactly five times.

Example usage:

```
SemaphoreHandle_t xMutex = NULL;

// A task that creates a mutex.
void vATask( void * pvParameters )
{
 // Create the mutex to guard a shared resource.
 xMutex = xSemaphoreCreateRecursiveMutex();
}

// A task that uses the mutex.
void vAnotherTask( void * pvParameters )
{
 // ... Do other things.

 if( xMutex != NULL )
 {
// See if we can obtain the mutex.  If the mutex is not available
// wait 10 ticks to see if it becomes free.
if( xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 ) == pdTRUE )
     {
// We were able to obtain the mutex and can now access the
// shared resource.

// ...
// For some reason due to the nature of the code further calls to
// xSemaphoreTakeRecursive() are made on the same mutex.  In real
// code these would not be just sequential calls as this would make
// no sense.  Instead the calls are likely to be buried inside
// a more complex call structure.
         xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 );
         xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 );

// The mutex has now been 'taken' three times, so will not be
// available to another task until it has also been given back
// three times.  Again it is unlikely that real code would have
// these calls sequentially, it would be more likely that the calls
// to xSemaphoreGiveRecursive() would be called as a call stack
// unwound.  This is just for demonstrative purposes.
         xSemaphoreGiveRecursive( xMutex );
         xSemaphoreGiveRecursive( xMutex );
         xSemaphoreGiveRecursive( xMutex );

// Now the mutex can be taken by other tasks.
     }
else
     {
// We could not obtain the mutex and can therefore not access
// the shared resource safely.
     }
 }
}
```

Parameters:
- **xMutex** -- A handle to the mutex being released, or 'given'. This is the handle returned by xSemaphoreCreateMutex();

Returns: pdTRUE if the semaphore was given.

---

xSemaphoreGiveFromISR(xSemaphore, pxHigherPriorityTaskWoken)

Macro to release a semaphore. The semaphore must have previously been created with a call to xSemaphoreCreateBinary() or xSemaphoreCreateCounting().

Mutex type semaphores (those created using a call to xSemaphoreCreateMutex()) must not be used with this macro.

This macro can be used from an ISR.

Example usage:

```
#define LONG_TIME 0xffff
#define TICKS_TO_WAIT 10
SemaphoreHandle_t xSemaphore = NULL;

// Repetitive task.
void vATask( void * pvParameters )
{
 for( ;; )
 {
// We want this task to run every 10 ticks of a timer.  The semaphore
// was created before this task was started.

// Block waiting for the semaphore to become available.
if( xSemaphoreTake( xSemaphore, LONG_TIME ) == pdTRUE )
     {
// It is time to execute.

// ...

// We have finished our task.  Return to the top of the loop where
// we will block on the semaphore until it is time to execute
// again.  Note when using the semaphore for synchronisation with an
// ISR in this manner there is no need to 'give' the semaphore back.
     }
 }
}

// Timer ISR
void vTimerISR( void * pvParameters )
{
static uint8_t ucLocalTickCount = 0;
static BaseType_t xHigherPriorityTaskWoken;

 // A timer tick has occurred.

 // ... Do other time functions.

 // Is it time for vATask () to run?
 xHigherPriorityTaskWoken = pdFALSE;
 ucLocalTickCount++;
 if( ucLocalTickCount >= TICKS_TO_WAIT )
 {
// Unblock the task by releasing the semaphore.
     xSemaphoreGiveFromISR( xSemaphore, &xHigherPriorityTaskWoken );

// Reset the count so we release the semaphore again in 10 ticks time.
     ucLocalTickCount = 0;
 }

 if( xHigherPriorityTaskWoken != pdFALSE )
 {
// We can force a context switch here.  Context switching from an
// ISR uses port specific syntax.  Check the demo task for your port
// to find the syntax required.
 }
}
```

Parameters:

- **xSemaphore** -- A handle to the semaphore being released. This is the handle returned when the semaphore was created.

- **pxHigherPriorityTaskWoken** -- xSemaphoreGiveFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if giving the semaphore caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xSemaphoreGiveFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.

Returns: pdTRUE if the semaphore was successfully given, otherwise errQUEUE_FULL.

`xSemaphoreTakeFromISR`(xSemaphore, pxHigherPriorityTaskWoken)

Macro to take a semaphore from an ISR. The semaphore must have previously been created with a call to xSemaphoreCreateBinary() or xSemaphoreCreateCounting().

Mutex type semaphores (those created using a call to xSemaphoreCreateMutex()) must not be used with this macro.

This macro can be used from an ISR, however taking a semaphore from an ISR is not a common operation. It is likely to only be useful when taking a counting semaphore when an interrupt is obtaining an object from a resource pool (when the semaphore count indicates the number of resources available).

Parameters:
- **xSemaphore** -- A handle to the semaphore being taken. This is the handle returned when the semaphore was created.
- **pxHigherPriorityTaskWoken** -- xSemaphoreTakeFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if taking the semaphore caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xSemaphoreTakeFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.

Returns: pdTRUE if the semaphore was successfully taken, otherwise pdFALSE

`xSemaphoreCreateMutex`()

Creates a new mutex type semaphore instance, and returns a handle by which the new mutex can be referenced.

Internally, within the FreeRTOS implementation, mutex semaphores use a block of memory, in which the mutex structure is stored. If a mutex is created using xSemaphoreCreateMutex() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateMutex() function. (see https://www.FreeRTOS.org/a00111.html). If a mutex is created using xSemaphoreCreateMutexStatic() then the application writer must provided the memory. xSemaphoreCreateMutexStatic() therefore allows a mutex to be created without using any dynamic memory allocation.

Mutexes created using this function can be accessed using the xSemaphoreTake() and xSemaphoreGive() macros. The xSemaphoreTakeRecursive() and xSemaphoreGiveRecursive() macros must not be used.

This type of semaphore uses a priority inheritance mechanism so a task 'taking' a semaphore MUST ALWAYS 'give' the semaphore back once the semaphore it is no longer required.

Mutex type semaphores cannot be used from within interrupt service routines.

See xSemaphoreCreateBinary() for an alternative implementation that can be used for pure synchronisation (where one task or interrupt always 'gives' the semaphore and another always 'takes' the semaphore) and from within interrupt service routines.

Example usage:

```
SemaphoreHandle_t xSemaphore;

void vATask( void * pvParameters )
{
 // Semaphore cannot be used before a call to xSemaphoreCreateMutex().
 // This is a macro so pass the variable in directly.
 xSemaphore = xSemaphoreCreateMutex();

 if( xSemaphore != NULL )
 {
// The semaphore was created successfully.
// The semaphore can now be used.
 }
}
```

Returns: If the mutex was successfully created then a handle to the created semaphore is returned. If there was not enough heap to allocate the mutex data structures then NULL is returned.

`xSemaphoreCreateMutexStatic`(pxMutexBuffer)

Creates a new mutex type semaphore instance, and returns a handle by which the new mutex can be referenced.

Internally, within the FreeRTOS implementation, mutex semaphores use a block of memory, in which the mutex structure is stored. If a mutex is created using xSemaphoreCreateMutex() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateMutex() function. (see https://www.FreeRTOS.org/a00111.html). If a mutex is created using xSemaphoreCreateMutexStatic() then the application writer must provided the memory. xSemaphoreCreateMutexStatic() therefore allows a mutex to be created without using any dynamic memory allocation.

Mutexes created using this function can be accessed using the xSemaphoreTake() and xSemaphoreGive() macros. The xSemaphoreTakeRecursive() and xSemaphoreGiveRecursive() macros must not be used.

This type of semaphore uses a priority inheritance mechanism so a task 'taking' a semaphore MUST ALWAYS 'give' the semaphore back once the semaphore it is no longer required.

Mutex type semaphores cannot be used from within interrupt service routines.

See xSemaphoreCreateBinary() for an alternative implementation that can be used for pure synchronisation (where one task or interrupt always 'gives' the semaphore and another always 'takes' the semaphore) and from within interrupt service routines.

Example usage:

```
SemaphoreHandle_t xSemaphore;
StaticSemaphore_t xMutexBuffer;

void vATask( void * pvParameters )
{
 // A mutex cannot be used before it has been created.  xMutexBuffer is
 // into xSemaphoreCreateMutexStatic() so no dynamic memory allocation is
 // attempted.
 xSemaphore = xSemaphoreCreateMutexStatic( &xMutexBuffer );

 // As no dynamic memory allocation was performed, xSemaphore cannot be NULL,
 // so there is no need to check it.
}
```

Parameters:
- **pxMutexBuffer** -- Must point to a variable of type StaticSemaphore_t, which will be used to hold the mutex's data structure, removing the need for the memory to be allocated dynamically.

Returns: If the mutex was successfully created then a handle to the created mutex is returned. If pxMutexBuffer was NULL then NULL is returned.

`xSemaphoreCreateRecursiveMutex()`

Creates a new recursive mutex type semaphore instance, and returns a handle by which the new recursive mutex can be referenced.

Internally, within the FreeRTOS implementation, recursive mutexes use a block of memory, in which the mutex structure is stored. If a recursive mutex is created using xSemaphoreCreateRecursiveMutex() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateRecursiveMutex() function. (see https://www.FreeRTOS.org/a00111.html). If a recursive mutex is created using xSemaphoreCreateRecursiveMutexStatic() then the application writer must provide the

memory that will get used by the mutex. xSemaphoreCreateRecursiveMutexStatic() therefore allows a recursive mutex to be created without using any dynamic memory allocation.

Mutexes created using this macro can be accessed using the xSemaphoreTakeRecursive() and xSemaphoreGiveRecursive() macros. The xSemaphoreTake() and xSemaphoreGive() macros must not be used.

A mutex used recursively can be 'taken' repeatedly by the owner. The mutex doesn't become available again until the owner has called xSemaphoreGiveRecursive() for each successful 'take' request. For example, if a task successfully 'takes' the same mutex 5 times then the mutex will not be available to any other task until it has also 'given' the mutex back exactly five times.

This type of semaphore uses a priority inheritance mechanism so a task 'taking' a semaphore MUST ALWAYS 'give' the semaphore back once the semaphore it is no longer required.

Mutex type semaphores cannot be used from within interrupt service routines.

See xSemaphoreCreateBinary() for an alternative implementation that can be used for pure synchronisation (where one task or interrupt always 'gives' the semaphore and another always 'takes' the semaphore) and from within interrupt service routines.

Example usage:

```
SemaphoreHandle_t xSemaphore;

void vATask( void * pvParameters )
{
 // Semaphore cannot be used before a call to xSemaphoreCreateMutex().
 // This is a macro so pass the variable in directly.
 xSemaphore = xSemaphoreCreateRecursiveMutex();

 if( xSemaphore != NULL )
 {
// The semaphore was created successfully.
// The semaphore can now be used.
 }
}
```

Returns: xSemaphore Handle to the created mutex semaphore. Should be of type SemaphoreHandle_t.

`xSemaphoreCreateRecursiveMutexStatic`(pxStaticSemaphore)

Creates a new recursive mutex type semaphore instance, and returns a handle by which the new recursive mutex can be referenced.

Internally, within the FreeRTOS implementation, recursive mutexes use a block of memory, in which the mutex structure is stored. If a recursive mutex is created using xSemaphoreCreateRecursiveMutex() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateRecursiveMutex() function. (see https://www.FreeRTOS.org/a00111.html). If a recursive mutex is created using xSemaphoreCreateRecursiveMutexStatic() then the application writer must provide the memory that will get used by the mutex. xSemaphoreCreateRecursiveMutexStatic() therefore allows a recursive mutex to be created without using any dynamic memory allocation.

Mutexes created using this macro can be accessed using the xSemaphoreTakeRecursive() and xSemaphoreGiveRecursive() macros. The xSemaphoreTake() and xSemaphoreGive() macros must not be used.

A mutex used recursively can be 'taken' repeatedly by the owner. The mutex doesn't become available again until the owner has called xSemaphoreGiveRecursive() for each successful 'take' request. For example, if a task successfully 'takes' the same mutex 5 times then the mutex will not be available to any other task until it has also 'given' the mutex back exactly five times.

This type of semaphore uses a priority inheritance mechanism so a task 'taking' a semaphore MUST ALWAYS 'give' the semaphore back once the semaphore it is no longer required.

Mutex type semaphores cannot be used from within interrupt service routines.

See xSemaphoreCreateBinary() for an alternative implementation that can be used for pure synchronisation (where one task or interrupt always 'gives' the semaphore and another always 'takes' the semaphore) and from within interrupt service routines.

Example usage:

```
SemaphoreHandle_t xSemaphore;
StaticSemaphore_t xMutexBuffer;

void vATask( void * pvParameters )
{
 // A recursive semaphore cannot be used before it is created.  Here a
 // recursive mutex is created using xSemaphoreCreateRecursiveMutexStatic().
 // The address of xMutexBuffer is passed into the function, and will hold
 // the mutexes data structures - so no dynamic memory allocation will be
 // attempted.
 xSemaphore = xSemaphoreCreateRecursiveMutexStatic( &xMutexBuffer );

 // As no dynamic memory allocation was performed, xSemaphore cannot be NULL,
 // so there is no need to check it.
}
```

Parameters:

- **pxStaticSemaphore** -- Must point to a variable of type StaticSemaphore_t, which will then be used to hold the recursive mutex's data structure, removing the need for the memory to be

allocated dynamically.

Returns: If the recursive mutex was successfully created then a handle to the created recursive mutex is returned. If pxStaticSemaphore was NULL then NULL is returned.

`xSemaphoreCreateCounting`(uxMaxCount, uxInitialCount)

Creates a new counting semaphore instance, and returns a handle by which the new counting semaphore can be referenced.

In many usage scenarios it is faster and more memory efficient to use a direct to task notification in place of a counting semaphore! https://www.FreeRTOS.org/RTOS-task-notifications.html

Internally, within the FreeRTOS implementation, counting semaphores use a block of memory, in which the counting semaphore structure is stored. If a counting semaphore is created using xSemaphoreCreateCounting() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateCounting() function. (see https://www.FreeRTOS.org/a00111.html). If a counting semaphore is created using xSemaphoreCreateCountingStatic() then the application writer can instead optionally provide the memory that will get used by the counting semaphore. xSemaphoreCreateCountingStatic() therefore allows a counting semaphore to be created without using any dynamic memory allocation.

Counting semaphores are typically used for two things:

1) Counting events.

In this usage scenario an event handler will 'give' a semaphore each time an event occurs (incrementing the semaphore count value), and a handler task will 'take' a semaphore each time it processes an event (decrementing the semaphore count value). The count value is therefore the difference between the number of events that have occurred and the number that have been processed. In this case it is desirable for the initial count value to be zero.

2) Resource management.

In this usage scenario the count value indicates the number of resources available. To obtain control of a resource a task must first obtain a semaphore - decrementing the semaphore count value. When the count value reaches zero there are no free resources. When a task finishes with the resource it 'gives' the semaphore back - incrementing the semaphore count value. In this case it is desirable for the initial count value to be equal to the maximum count value, indicating that all resources are free.

Example usage:

```
SemaphoreHandle_t xSemaphore;

void vATask( void * pvParameters )
{
SemaphoreHandle_t xSemaphore = NULL;

 // Semaphore cannot be used before a call to xSemaphoreCreateCounting().
 // The max value to which the semaphore can count should be 10, and the
 // initial value assigned to the count should be 0.
 xSemaphore = xSemaphoreCreateCounting( 10, 0 );

 if( xSemaphore != NULL )
 {
// The semaphore was created successfully.
// The semaphore can now be used.
 }
}
```

Parameters:
- **uxMaxCount** -- The maximum count value that can be reached. When the semaphore reaches this value it can no longer be 'given'.
- **uxInitialCount** -- The count value assigned to the semaphore when it is created.

Returns: Handle to the created semaphore. Null if the semaphore could not be created.

`xSemaphoreCreateCountingStatic`(uxMaxCount, uxInitialCount, pxSemaphoreBuffer)

Creates a new counting semaphore instance, and returns a handle by which the new counting semaphore can be referenced.

In many usage scenarios it is faster and more memory efficient to use a direct to task notification in place of a counting semaphore! https://www.FreeRTOS.org/RTOS-task-notifications.html

Internally, within the FreeRTOS implementation, counting semaphores use a block of memory, in which the counting semaphore structure is stored. If a counting semaphore is created using xSemaphoreCreateCounting() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateCounting() function. (see https://www.FreeRTOS.org/a00111.html). If a counting semaphore is created using xSemaphoreCreateCountingStatic() then the application writer must provide the memory. xSemaphoreCreateCountingStatic() therefore allows a counting semaphore to be created without using any dynamic memory allocation.

Counting semaphores are typically used for two things:

1) Counting events.

In this usage scenario an event handler will 'give' a semaphore each time an event occurs (incrementing the semaphore count value), and a handler task will 'take' a semaphore each time it processes an event (decrementing the semaphore count value). The count value is

therefore the difference between the number of events that have occurred and the number that have been processed. In this case it is desirable for the initial count value to be zero.

2) Resource management.

In this usage scenario the count value indicates the number of resources available. To obtain control of a resource a task must first obtain a semaphore - decrementing the semaphore count value. When the count value reaches zero there are no free resources. When a task finishes with the resource it 'gives' the semaphore back - incrementing the semaphore count value. In this case it is desirable for the initial count value to be equal to the maximum count value, indicating that all resources are free.

Example usage:

```
SemaphoreHandle_t xSemaphore;
StaticSemaphore_t xSemaphoreBuffer;

void vATask( void * pvParameters )
{
SemaphoreHandle_t xSemaphore = NULL;

 // Counting semaphore cannot be used before they have been created.  Create
 // a counting semaphore using xSemaphoreCreateCountingStatic().  The max
 // value to which the semaphore can count is 10, and the initial value
 // assigned to the count will be 0.  The address of xSemaphoreBuffer is
 // passed in and will be used to hold the semaphore structure, so no dynamic
 // memory allocation will be used.
 xSemaphore = xSemaphoreCreateCounting( 10, 0, &xSemaphoreBuffer );

 // No memory allocation was attempted so xSemaphore cannot be NULL, so there
 // is no need to check its value.
}
```

Parameters:

- **uxMaxCount** -- The maximum count value that can be reached. When the semaphore reaches this value it can no longer be 'given'.
- **uxInitialCount** -- The count value assigned to the semaphore when it is created.
- **pxSemaphoreBuffer** -- Must point to a variable of type StaticSemaphore_t, which will then be used to hold the semaphore's data structure, removing the need for the memory to be allocated dynamically.

Returns: If the counting semaphore was successfully created then a handle to the created counting semaphore is returned. If pxSemaphoreBuffer was NULL then NULL is returned.

`vSemaphoreDelete`(xSemaphore)

Delete a semaphore. This function must be used with care. For example, do not delete a mutex type semaphore if the mutex is held by a task.

Parameters:
- **xSemaphore** -- A handle to the semaphore to be deleted.

`xSemaphoreGetMutexHolder`(xSemaphore)

If xMutex is indeed a mutex type semaphore, return the current mutex holder. If xMutex is not a mutex type semaphore, or the mutex is available (not held by a task), return NULL.

Note: This is a good way of determining if the calling task is the mutex holder, but not a good way of determining the identity of the mutex holder as the holder may change between the function exiting and the returned value being tested.

`xSemaphoreGetMutexHolderFromISR`(xSemaphore)

If xMutex is indeed a mutex type semaphore, return the current mutex holder. If xMutex is not a mutex type semaphore, or the mutex is available (not held by a task), return NULL.

`uxSemaphoreGetCount`(xSemaphore)

If the semaphore is a counting semaphore then uxSemaphoreGetCount() returns its current count value. If the semaphore is a binary semaphore then uxSemaphoreGetCount() returns 1 if the semaphore is available, and 0 if the semaphore is not available.

`uxSemaphoreGetCountFromISR`(xSemaphore)

semphr.h

```
UBaseType_t uxSemaphoreGetCountFromISR( SemaphoreHandle_t xSemaphore );
```

If the semaphore is a counting semaphore then uxSemaphoreGetCountFromISR() returns its current count value. If the semaphore is a binary semaphore then uxSemaphoreGetCountFromISR() returns 1 if the semaphore is available, and 0 if the semaphore is not available.

`xSemaphoreGetStaticBuffer`(xSemaphore, ppxSemaphoreBuffer)

Retrieve pointer to a statically created binary semaphore, counting semaphore, or mutex semaphore's data structure buffer. This is the same buffer that is supplied at the time of creation.

Parameters:
- **xSemaphore** -- The semaphore for which to retrieve the buffer.
- **ppxSemaphoreBuffer** -- Used to return a pointer to the semaphore's data structure buffer.

Returns: pdTRUE if buffer was retrieved, pdFALSE otherwise.

Type Definitions

typedef QueueHandle_t **SemaphoreHandle_t**

Timer API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/timers.h
- This header file can be included with:

```
#include "freertos/timers.h"
```

Functions

TimerHandle_t **xTimerCreate**(*const* char **const* pcTimerName, *const* TickType_t xTimerPeriodInTicks, *const* BaseType_t xAutoReload, void **const* pvTimerID, TimerCallbackFunction_t pxCallbackFunction)

Creates a new software timer instance, and returns a handle by which the created software timer can be referenced.

Internally, within the FreeRTOS implementation, software timers use a block of memory, in which the timer data structure is stored. If a software timer is created using xTimerCreate() then the required memory is automatically dynamically allocated inside the xTimerCreate() function. (see https://www.FreeRTOS.org/a00111.html). If a software timer is created using xTimerCreateStatic() then the application writer must provide the memory that will get used by the software timer. xTimerCreateStatic() therefore allows a software timer to be created without using any dynamic memory allocation.

Timers are created in the dormant state. The xTimerStart(), xTimerReset(), xTimerStartFromISR(), xTimerResetFromISR(), xTimerChangePeriod() and xTimerChangePeriodFromISR() API functions can all be used to transition a timer into the active state.

Example usage:

```
* #define NUM_TIMERS 5
*
* // An array to hold handles to the created timers.
* TimerHandle_t xTimers[ NUM_TIMERS ];
*
* // An array to hold a count of the number of times each timer expires.
* int32_t lExpireCounters[ NUM_TIMERS ] = { 0 };
*
* // Define a callback function that will be used by multiple timer instances.
* // The callback function does nothing but count the number of times the
* // associated timer expires, and stop the timer once the timer has expired
* // 10 times.
* void vTimerCallback( TimerHandle_t pxTimer )
* {
* int32_t lArrayIndex;
* const int32_t xMaxExpiryCountBeforeStopping = 10;
*
*     // Optionally do something if the pxTimer parameter is NULL.
*     configASSERT( pxTimer );
*
*     // Which timer expired?
*     lArrayIndex = ( int32_t ) pvTimerGetTimerID( pxTimer );
*
*     // Increment the number of times that pxTimer has expired.
*     lExpireCounters[ lArrayIndex ] += 1;
*
*     // If the timer has expired 10 times then stop it from running.
*     if( lExpireCounters[ lArrayIndex ] == xMaxExpiryCountBeforeStopping )
*     {
*         // Do not use a block time if calling a timer API function from a
*         // timer callback function, as doing so could cause a deadlock!
*         xTimerStop( pxTimer, 0 );
*     }
* }
*
* void main( void )
* {
* int32_t x;
*
*     // Create then start some timers.  Starting the timers before the scheduler
*     // has been started means the timers will start running immediately that
*     // the scheduler starts.
*     for( x = 0; x < NUM_TIMERS; x++ )
*     {
*         xTimers[ x ] = xTimerCreate(    "Timer",             // Just a text name,
not used by the kernel.
*                                         ( 100 * ( x + 1 ) ), // The timer period
in ticks.
*                                         pdTRUE,              // The timers will
auto-reload themselves when they expire.
*                                         ( void * ) x,        // Assign each timer
a unique id equal to its array index.
*                                         vTimerCallback       // Each timer calls
the same callback when it expires.
*                                     );
*
*         if( xTimers[ x ] == NULL )
*         {
*             // The timer was not created.
*         }
*         else
*         {
*             // Start the timer.  No block time is specified, and even if one was
```

```
*             // it would be ignored because the scheduler has not yet been
*             // started.
*             if( xTimerStart( xTimers[ x ], 0 ) != pdPASS )
*             {
*                 // The timer could not be set into the Active state.
*             }
*         }
*     }
*
*     // ...
*     // Create tasks here.
*     // ...
*
*     // Starting the scheduler will start the timers running as they have already
*     // been set into the active state.
*     vTaskStartScheduler();
*
*     // Should not reach here.
*     for( ;; );
* }
*
```

Parameters:

- **pcTimerName** -- A text name that is assigned to the timer. This is done purely to assist debugging. The kernel itself only ever references a timer by its handle, and never by its name.
- **xTimerPeriodInTicks** -- The timer period. The time is defined in tick periods so the constant portTICK_PERIOD_MS can be used to convert a time that has been specified in milliseconds. For example, if the timer must expire after 100 ticks, then xTimerPeriodInTicks should be set to 100. Alternatively, if the timer must expire after 500ms, then xPeriod can be set to (500 / portTICK_PERIOD_MS) provided configTICK_RATE_HZ is less than or equal to 1000. Time timer period must be greater than 0.
- **xAutoReload** -- If xAutoReload is set to pdTRUE then the timer will expire repeatedly with a frequency set by the xTimerPeriodInTicks parameter. If xAutoReload is set to pdFALSE then the timer will be a one-shot timer and enter the dormant state after it expires.
- **pvTimerID** -- An identifier that is assigned to the timer being created. Typically this would be used in the timer callback function to identify which timer expired when the same callback function is assigned to more than one timer.
- **pxCallbackFunction** -- The function to call when the timer expires. Callback functions must have the prototype defined by TimerCallbackFunction_t, which is "void vCallbackFunction(TimerHandle_t xTimer);".

Returns: If the timer is successfully created then a handle to the newly created timer is returned. If the timer cannot be created because there is insufficient FreeRTOS heap remaining to allocate the timer structures

then NULL is returned.

TimerHandle_t `xTimerCreateStatic`(*const* char **const* pcTimerName, *const* TickType_t xTimerPeriodInTicks, *const* BaseType_t xAutoReload, void **const* pvTimerID, TimerCallbackFunction_t pxCallbackFunction, StaticTimer_t *pxTimerBuffer)

Creates a new software timer instance, and returns a handle by which the created software timer can be referenced.

Internally, within the FreeRTOS implementation, software timers use a block of memory, in which the timer data structure is stored. If a software timer is created using xTimerCreate() then the required memory is automatically dynamically allocated inside the xTimerCreate() function. (see https://www.FreeRTOS.org/a00111.html). If a software timer is created using xTimerCreateStatic() then the application writer must provide the memory that will get used by the software timer. xTimerCreateStatic() therefore allows a software timer to be created without using any dynamic memory allocation.

Timers are created in the dormant state. The xTimerStart(), xTimerReset(), xTimerStartFromISR(), xTimerResetFromISR(), xTimerChangePeriod() and xTimerChangePeriodFromISR() API functions can all be used to transition a timer into the active state.

Example usage:

```
*
* // The buffer used to hold the software timer's data structure.
* static StaticTimer_t xTimerBuffer;
*
* // A variable that will be incremented by the software timer's callback
* // function.
* UBaseType_t uxVariableToIncrement = 0;
*
* // A software timer callback function that increments a variable passed to
* // it when the software timer was created.  After the 5th increment the
* // callback function stops the software timer.
* static void prvTimerCallback( TimerHandle_t xExpiredTimer )
* {
* UBaseType_t *puxVariableToIncrement;
* BaseType_t xReturned;
*
*     // Obtain the address of the variable to increment from the timer ID.
*     puxVariableToIncrement = ( UBaseType_t * ) pvTimerGetTimerID( xExpiredTimer
);
*
*     // Increment the variable to show the timer callback has executed.
*     ( *puxVariableToIncrement )++;
*
*     // If this callback has executed the required number of times, stop the
*     // timer.
*     if( *puxVariableToIncrement == 5 )
*     {
*         // This is called from a timer callback so must not block.
*         xTimerStop( xExpiredTimer, staticDONT_BLOCK );
*     }
* }
*
*
* void main( void )
* {
*     // Create the software time.  xTimerCreateStatic() has an extra parameter
*     // than the normal xTimerCreate() API function.  The parameter is a pointer
*     // to the StaticTimer_t structure that will hold the software timer
*     // structure.  If the parameter is passed as NULL then the structure will be
*     // allocated dynamically, just as if xTimerCreate() had been called.
*     xTimer = xTimerCreateStatic( "T1",             // Text name for the task.
Helps debugging only.  Not used by FreeRTOS.
*                                  xTimerPeriod,     // The period of the timer in
ticks.
*                                  pdTRUE,           // This is an auto-reload
timer.
*                                  ( void * ) &uxVariableToIncrement,    // A
variable incremented by the software timer's callback function
*                                  prvTimerCallback, // The function to execute
when the timer expires.
*                                  &xTimerBuffer );  // The buffer that will hold
the software timer structure.
*
*     // The scheduler has not started yet so a block time is not used.
*     xReturned = xTimerStart( xTimer, 0 );
*
*     // ...
*     // Create tasks here.
*     // ...
*
*     // Starting the scheduler will start the timers running as they have already
*     // been set into the active state.
*     vTaskStartScheduler();
```

```
*
*      // Should not reach here.
*      for( ;; );
* }
*
```

Parameters:

- **pcTimerName** -- A text name that is assigned to the timer. This is done purely to assist debugging. The kernel itself only ever references a timer by its handle, and never by its name.
- **xTimerPeriodInTicks** -- The timer period. The time is defined in tick periods so the constant portTICK_PERIOD_MS can be used to convert a time that has been specified in milliseconds. For example, if the timer must expire after 100 ticks, then xTimerPeriodInTicks should be set to 100. Alternatively, if the timer must expire after 500ms, then xPeriod can be set to (500 / portTICK_PERIOD_MS) provided configTICK_RATE_HZ is less than or equal to 1000. The timer period must be greater than 0.
- **xAutoReload** -- If xAutoReload is set to pdTRUE then the timer will expire repeatedly with a frequency set by the xTimerPeriodInTicks parameter. If xAutoReload is set to pdFALSE then the timer will be a one-shot timer and enter the dormant state after it expires.
- **pvTimerID** -- An identifier that is assigned to the timer being created. Typically this would be used in the timer callback function to identify which timer expired when the same callback function is assigned to more than one timer.
- **pxCallbackFunction** -- The function to call when the timer expires. Callback functions must have the prototype defined by TimerCallbackFunction_t, which is "void vCallbackFunction(TimerHandle_t xTimer);".
- **pxTimerBuffer** -- Must point to a variable of type StaticTimer_t, which will be then be used to hold the software timer's data structures, removing the need for the memory to be allocated dynamically.

Returns: If the timer is created then a handle to the created timer is returned. If pxTimerBuffer was NULL then NULL is returned.

---

void *pvTimerGetTimerID(*const* TimerHandle_t xTimer)

Returns the ID assigned to the timer.

IDs are assigned to timers using the pvTimerID parameter of the call to xTimerCreated() that was used to create the timer, and by calling the vTimerSetTimerID() API function.

If the same callback function is assigned to multiple timers then the timer ID can be used as time specific (timer local) storage.

Example usage:

See the xTimerCreate() API function example usage scenario.

Parameters: **xTimer** -- The timer being queried.
Returns: The ID assigned to the timer being queried.

---

void **vTimerSetTimerID**(TimerHandle_t xTimer, void *pvNewID)

Sets the ID assigned to the timer.

IDs are assigned to timers using the pvTimerID parameter of the call to xTimerCreated() that was used to create the timer.

If the same callback function is assigned to multiple timers then the timer ID can be used as time specific (timer local) storage.

Example usage:

See the xTimerCreate() API function example usage scenario.

Parameters:
- **xTimer** -- The timer being updated.
- **pvNewID** -- The ID to assign to the timer.

BaseType_t **xTimerIsTimerActive**(TimerHandle_t xTimer)

Queries a timer to see if it is active or dormant.

A timer will be dormant if: 1) It has been created but not started, or 2) It is an expired one-shot timer that has not been restarted.

Timers are created in the dormant state. The xTimerStart(), xTimerReset(), xTimerStartFromISR(), xTimerResetFromISR(), xTimerChangePeriod() and xTimerChangePeriodFromISR() API functions can all be used to transition a timer into the active state.

Example usage:

```
* // This function assumes xTimer has already been created.
* void vAFunction( TimerHandle_t xTimer )
* {
*     if( xTimerIsTimerActive( xTimer ) != pdFALSE ) // or more simply and
equivalently "if( xTimerIsTimerActive( xTimer ) )"
*     {
*         // xTimer is active, do something.
*     }
*     else
*     {
*         // xTimer is not active, do something else.
*     }
* }
*
```

Parameters: **xTimer** -- The timer being queried.

Returns: pdFALSE will be returned if the timer is dormant. A value other than pdFALSE will be returned if the timer is active.

TaskHandle_t **xTimerGetTimerDaemonTaskHandle**(void)

Simply returns the handle of the timer service/daemon task. It it not valid to call xTimerGetTimerDaemonTaskHandle() before the scheduler has been started.

BaseType_t **xTimerPendFunctionCallFromISR**(PendedFunction_t xFunctionToPend, void *pvParameter1, uint32_t ulParameter2, BaseType_t *pxHigherPriorityTaskWoken)

Used from application interrupt service routines to defer the execution of a function to the RTOS daemon task (the timer service task, hence this function is implemented in timers.c and is prefixed with 'Timer').

Ideally an interrupt service routine (ISR) is kept as short as possible, but sometimes an ISR either has a lot of processing to do, or needs to perform processing that is not deterministic. In these cases xTimerPendFunctionCallFromISR() can be used to defer processing of a function to the RTOS daemon task.

A mechanism is provided that allows the interrupt to return directly to the task that will subsequently execute the pended callback function. This allows the callback function to execute contiguously in time with the interrupt - just as if the callback had executed in the interrupt itself.

Example usage:

```
*
*  // The callback function that will execute in the context of the daemon task.
*  // Note callback functions must all use this same prototype.
*  void vProcessInterface( void *pvParameter1, uint32_t ulParameter2 )
*  {
*      BaseType_t xInterfaceToService;
*
*      // The interface that requires servicing is passed in the second
*      // parameter.  The first parameter is not used in this case.
*      xInterfaceToService = ( BaseType_t ) ulParameter2;
*
*      // ...Perform the processing here...
*  }
*
*  // An ISR that receives data packets from multiple interfaces
*  void vAnISR( void )
*  {
*      BaseType_t xInterfaceToService, xHigherPriorityTaskWoken;
*
*      // Query the hardware to determine which interface needs processing.
*      xInterfaceToService = prvCheckInterfaces();
*
*      // The actual processing is to be deferred to a task.  Request the
*      // vProcessInterface() callback function is executed, passing in the
*      // number of the interface that needs processing.  The interface to
*      // service is passed in the second parameter.  The first parameter is
*      // not used in this case.
*      xHigherPriorityTaskWoken = pdFALSE;
*      xTimerPendFunctionCallFromISR( vProcessInterface, NULL, ( uint32_t )
xInterfaceToService, &xHigherPriorityTaskWoken );
*
*      // If xHigherPriorityTaskWoken is now set to pdTRUE then a context
*      // switch should be requested.  The macro used is port specific and will
*      // be either portYIELD_FROM_ISR() or portEND_SWITCHING_ISR() - refer to
*      // the documentation page for the port being used.
*      portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
*
*  }
*
```

Parameters:

- **xFunctionToPend** -- The function to execute from the timer service/ daemon task. The function must conform to the PendedFunction_t prototype.
- **pvParameter1** -- The value of the callback function's first parameter. The parameter has a void * type to allow it to be used to pass any type. For example, unsigned longs can be cast to a void *, or the void * can be used to point to a structure.
- **ulParameter2** -- The value of the callback function's second parameter.
- **pxHigherPriorityTaskWoken** -- As mentioned above, calling this function will result in a message being sent to the timer daemon task. If the priority of the timer daemon task (which is set using configTIMER_TASK_PRIORITY in FreeRTOSConfig.h) is higher than the priority of the currently running task (the task the interrupt

interrupted) then *pxHigherPriorityTaskWoken will be set to pdTRUE within xTimerPendFunctionCallFromISR(), indicating that a context switch should be requested before the interrupt exits. For that reason *pxHigherPriorityTaskWoken must be initialised to pdFALSE. See the example code below.

Returns: pdPASS is returned if the message was successfully sent to the timer daemon task, otherwise pdFALSE is returned.

BaseType_t **`xTimerPendFunctionCall`**(PendedFunction_t xFunctionToPend, void *pvParameter1, uint32_t ulParameter2, TickType_t xTicksToWait)

Used to defer the execution of a function to the RTOS daemon task (the timer service task, hence this function is implemented in timers.c and is prefixed with 'Timer').

Parameters:

- **xFunctionToPend** -- The function to execute from the timer service/ daemon task. The function must conform to the PendedFunction_t prototype.
- **pvParameter1** -- The value of the callback function's first parameter. The parameter has a void * type to allow it to be used to pass any type. For example, unsigned longs can be cast to a void *, or the void * can be used to point to a structure.
- **ulParameter2** -- The value of the callback function's second parameter.
- **xTicksToWait** -- Calling this function will result in a message being sent to the timer daemon task on a queue. xTicksToWait is the amount of time the calling task should remain in the Blocked state (so not using any processing time) for space to become available on the timer queue if the queue is found to be full.

Returns: pdPASS is returned if the message was successfully sent to the timer daemon task, otherwise pdFALSE is returned.

---

const char ***`pcTimerGetName`**(TimerHandle_t xTimer)

Returns the name that was assigned to a timer when the timer was created.

Parameters: **xTimer** -- The handle of the timer being queried.

Returns: The name assigned to the timer specified by the xTimer parameter.

---

void **`vTimerSetReloadMode`**(TimerHandle_t xTimer, *const* BaseType_t xAutoReload)

Updates a timer to be either an auto-reload timer, in which case the timer automatically resets itself each time it expires, or a one-shot timer, in which case the timer will only expire once unless it is manually restarted.

Parameters:

- **xTimer** -- The handle of the timer being updated.
- **xAutoReload** -- If xAutoReload is set to pdTRUE then the timer will expire repeatedly with a frequency set by the timer's period (see the xTimerPeriodInTicks parameter of the xTimerCreate() API function).

If xAutoReload is set to pdFALSE then the timer will be a one-shot timer and enter the dormant state after it expires.

BaseType_t **xTimerGetReloadMode**(TimerHandle_t xTimer)

Queries a timer to determine if it is an auto-reload timer, in which case the timer automatically resets itself each time it expires, or a one-shot timer, in which case the timer will only expire once unless it is manually restarted.

Parameters: **xTimer** -- The handle of the timer being queried.
Returns: If the timer is an auto-reload timer then pdTRUE is returned, otherwise pdFALSE is returned.

UBaseType_t **uxTimerGetReloadMode**(TimerHandle_t xTimer)

Queries a timer to determine if it is an auto-reload timer, in which case the timer automatically resets itself each time it expires, or a one-shot timer, in which case the timer will only expire once unless it is manually restarted.

Parameters: **xTimer** -- The handle of the timer being queried.
Returns: If the timer is an auto-reload timer then pdTRUE is returned, otherwise pdFALSE is returned.

TickType_t **xTimerGetPeriod**(TimerHandle_t xTimer)

Returns the period of a timer.

Parameters: **xTimer** -- The handle of the timer being queried.
Returns: The period of the timer in ticks.

TickType_t **xTimerGetExpiryTime**(TimerHandle_t xTimer)

Returns the time in ticks at which the timer will expire. If this is less than the current tick count then the expiry time has overflowed from the current time.

Parameters: **xTimer** -- The handle of the timer being queried.
Returns: If the timer is running then the time in ticks at which the timer will next expire is returned. If the timer is not running then the return value is undefined.

BaseType_t **xTimerGetStaticBuffer**(TimerHandle_t xTimer, StaticTimer_t **ppxTimerBuffer)

Retrieve pointer to a statically created timer's data structure buffer. This is the same buffer that is supplied at the time of creation.

Parameters:
- **xTimer** -- The timer for which to retrieve the buffer.
- **ppxTimerBuffer** -- Used to return a pointer to the timers's data structure buffer.

Returns: pdTRUE if the buffer was retrieved, pdFALSE otherwise.

void **vApplicationGetTimerTaskMemory**(StaticTask_t **ppxTimerTaskTCBBuffer, StackType_t **ppxTimerTaskStackBuffer, uint32_t *pulTimerTaskStackSize)

This function is used to provide a statically allocated block of memory to FreeRTOS to hold the Timer Task TCB. This function is required when configSUPPORT_STATIC_ALLOCATION is set. For more information see this URI: https://www.FreeRTOS.org/a00110.html#configSUPPORT_STATIC_ALLOCATION

Parameters:

- **ppxTimerTaskTCBBuffer** -- A handle to a statically allocated TCB buffer
- **ppxTimerTaskStackBuffer** -- A handle to a statically allocated Stack buffer for the idle task
- **pulTimerTaskStackSize** -- A pointer to the number of elements that will fit in the allocated stack buffer

Macros

xTimerStart(xTimer, xTicksToWait)

Timer functionality is provided by a timer service/daemon task. Many of the public FreeRTOS timer API functions send commands to the timer service task through a queue called the timer command queue. The timer command queue is private to the kernel itself and is not directly accessible to application code. The length of the timer command queue is set by the configTIMER_QUEUE_LENGTH configuration constant.

xTimerStart() starts a timer that was previously created using the xTimerCreate() API function. If the timer had already been started and was already in the active state, then xTimerStart() has equivalent functionality to the xTimerReset() API function.

Starting a timer ensures the timer is in the active state. If the timer is not stopped, deleted, or reset in the mean time, the callback function associated with the timer will get called 'n' ticks after xTimerStart() was called, where 'n' is the timers defined period.

It is valid to call xTimerStart() before the scheduler has been started, but when this is done the timer will not actually start until the scheduler is started, and the timers expiry time will be relative to when the scheduler is started, not relative to when xTimerStart() was called.

The configUSE_TIMERS configuration constant must be set to 1 for xTimerStart() to be available.

Example usage:

See the xTimerCreate() API function example usage scenario.

Parameters:

- **xTimer** -- The handle of the timer being started/restarted.
- **xTicksToWait** -- Specifies the time, in ticks, that the calling task should be held in the Blocked state to wait for the start command to be successfully sent to the timer command queue, should the queue

already be full when xTimerStart() was called. xTicksToWait is ignored if xTimerStart() is called before the scheduler is started.

Returns: pdFAIL will be returned if the start command could not be sent to the timer command queue even after xTicksToWait ticks had passed. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system, although the timers expiry time is relative to when xTimerStart() is actually called. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

`xTimerStop`(xTimer, xTicksToWait)

Timer functionality is provided by a timer service/daemon task. Many of the public FreeRTOS timer API functions send commands to the timer service task through a queue called the timer command queue. The timer command queue is private to the kernel itself and is not directly accessible to application code. The length of the timer command queue is set by the configTIMER_QUEUE_LENGTH configuration constant.

xTimerStop() stops a timer that was previously started using either of the The xTimerStart(), xTimerReset(), xTimerStartFromISR(), xTimerResetFromISR(), xTimerChangePeriod() or xTimerChangePeriodFromISR() API functions.

Stopping a timer ensures the timer is not in the active state.

The configUSE_TIMERS configuration constant must be set to 1 for xTimerStop() to be available.

Example usage:

See the xTimerCreate() API function example usage scenario.

Parameters:

- **xTimer** -- The handle of the timer being stopped.
- **xTicksToWait** -- Specifies the time, in ticks, that the calling task should be held in the Blocked state to wait for the stop command to be successfully sent to the timer command queue, should the queue already be full when xTimerStop() was called. xTicksToWait is ignored if xTimerStop() is called before the scheduler is started.

Returns: pdFAIL will be returned if the stop command could not be sent to the timer command queue even after xTicksToWait ticks had passed. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

`xTimerChangePeriod`(xTimer, xNewPeriod, xTicksToWait)

Timer functionality is provided by a timer service/daemon task. Many of the public FreeRTOS timer API functions send commands to the timer service task through a queue called the timer command queue. The timer command queue is private to the kernel itself and is not directly accessible to application code. The length of the timer command queue is set by the configTIMER_QUEUE_LENGTH configuration constant.

xTimerChangePeriod() changes the period of a timer that was previously created using the xTimerCreate() API function.

xTimerChangePeriod() can be called to change the period of an active or dormant state timer.

The configUSE_TIMERS configuration constant must be set to 1 for xTimerChangePeriod() to be available.

Example usage:

```
* // This function assumes xTimer has already been created.  If the timer
* // referenced by xTimer is already active when it is called, then the timer
* // is deleted.  If the timer referenced by xTimer is not active when it is
* // called, then the period of the timer is set to 500ms and the timer is
* // started.
* void vAFunction( TimerHandle_t xTimer )
* {
*     if( xTimerIsTimerActive( xTimer ) != pdFALSE ) // or more simply and
equivalently "if( xTimerIsTimerActive( xTimer ) )"
*     {
*         // xTimer is already active - delete it.
*         xTimerDelete( xTimer );
*     }
*     else
*     {
*         // xTimer is not active, change its period to 500ms.  This will also
*         // cause the timer to start.  Block for a maximum of 100 ticks if the
*         // change period command cannot immediately be sent to the timer
*         // command queue.
*         if( xTimerChangePeriod( xTimer, 500 / portTICK_PERIOD_MS, 100 ) == pdPASS
)
*         {
*             // The command was successfully sent.
*         }
*         else
*         {
*             // The command could not be sent, even after waiting for 100 ticks
*             // to pass.  Take appropriate action here.
*         }
*     }
* }
*
```

Parameters:

- **xTimer** -- The handle of the timer that is having its period changed.
- **xNewPeriod** -- The new period for xTimer. Timer periods are specified in tick periods, so the constant portTICK_PERIOD_MS can be used to convert a time that has been specified in milliseconds. For example, if the timer must expire after 100 ticks, then

xNewPeriod should be set to 100. Alternatively, if the timer must expire after 500ms, then xNewPeriod can be set to (500 / portTICK_PERIOD_MS) provided configTICK_RATE_HZ is less than or equal to 1000.

- **xTicksToWait** -- Specifies the time, in ticks, that the calling task should be held in the Blocked state to wait for the change period command to be successfully sent to the timer command queue, should the queue already be full when xTimerChangePeriod() was called. xTicksToWait is ignored if xTimerChangePeriod() is called before the scheduler is started.

Returns: pdFAIL will be returned if the change period command could not be sent to the timer command queue even after xTicksToWait ticks had passed. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

`xTimerDelete`(xTimer, xTicksToWait)

Timer functionality is provided by a timer service/daemon task. Many of the public FreeRTOS timer API functions send commands to the timer service task through a queue called the timer command queue. The timer command queue is private to the kernel itself and is not directly accessible to application code. The length of the timer command queue is set by the configTIMER_QUEUE_LENGTH configuration constant.

xTimerDelete() deletes a timer that was previously created using the xTimerCreate() API function.

The configUSE_TIMERS configuration constant must be set to 1 for xTimerDelete() to be available.

Example usage:

See the xTimerChangePeriod() API function example usage scenario.

Parameters:

- **xTimer** -- The handle of the timer being deleted.
- **xTicksToWait** -- Specifies the time, in ticks, that the calling task should be held in the Blocked state to wait for the delete command to be successfully sent to the timer command queue, should the queue already be full when xTimerDelete() was called. xTicksToWait is ignored if xTimerDelete() is called before the scheduler is started.

Returns: pdFAIL will be returned if the delete command could not be sent to the timer command queue even after xTicksToWait ticks had passed. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will

depend on the priority of the timer service/daemon task relative to other tasks in the system. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

`xTimerReset`(xTimer, xTicksToWait)

Timer functionality is provided by a timer service/daemon task. Many of the public FreeRTOS timer API functions send commands to the timer service task through a queue called the timer command queue. The timer command queue is private to the kernel itself and is not directly accessible to application code. The length of the timer command queue is set by the configTIMER_QUEUE_LENGTH configuration constant.

xTimerReset() re-starts a timer that was previously created using the xTimerCreate() API function. If the timer had already been started and was already in the active state, then xTimerReset() will cause the timer to re-evaluate its expiry time so that it is relative to when xTimerReset() was called. If the timer was in the dormant state then xTimerReset() has equivalent functionality to the xTimerStart() API function.

Resetting a timer ensures the timer is in the active state. If the timer is not stopped, deleted, or reset in the mean time, the callback function associated with the timer will get called 'n' ticks after xTimerReset() was called, where 'n' is the timers defined period.

It is valid to call xTimerReset() before the scheduler has been started, but when this is done the timer will not actually start until the scheduler is started, and the timers expiry time will be relative to when the scheduler is started, not relative to when xTimerReset() was called.

The configUSE_TIMERS configuration constant must be set to 1 for xTimerReset() to be available.

Example usage:

```
* // When a key is pressed, an LCD back-light is switched on.  If 5 seconds pass
* // without a key being pressed, then the LCD back-light is switched off.  In
* // this case, the timer is a one-shot timer.
*
* TimerHandle_t xBacklightTimer = NULL;
*
* // The callback function assigned to the one-shot timer.  In this case the
* // parameter is not used.
* void vBacklightTimerCallback( TimerHandle_t pxTimer )
* {
*     // The timer expired, therefore 5 seconds must have passed since a key
*     // was pressed.  Switch off the LCD back-light.
*     vSetBacklightState( BACKLIGHT_OFF );
* }
*
* // The key press event handler.
* void vKeyPressEventHandler( char cKey )
* {
*     // Ensure the LCD back-light is on, then reset the timer that is
*     // responsible for turning the back-light off after 5 seconds of
*     // key inactivity.  Wait 10 ticks for the command to be successfully sent
*     // if it cannot be sent immediately.
*     vSetBacklightState( BACKLIGHT_ON );
*     if( xTimerReset( xBacklightTimer, 100 ) != pdPASS )
*     {
*         // The reset command was not executed successfully.  Take appropriate
*         // action here.
*     }
*
*     // Perform the rest of the key processing here.
* }
*
* void main( void )
* {
* int32_t x;
*
*     // Create then start the one-shot timer that is responsible for turning
*     // the back-light off if no keys are pressed within a 5 second period.
*     xBacklightTimer = xTimerCreate( "BacklightTimer",           // Just a text
name, not used by the kernel.
*                                     ( 5000 / portTICK_PERIOD_MS), // The timer
period in ticks.
*                                     pdFALSE,                    // The timer is a
one-shot timer.
*                                     0,                          // The id is not
used by the callback so can take any value.
*                                     vBacklightTimerCallback     // The callback
function that switches the LCD back-light off.
*                                   );
*
*     if( xBacklightTimer == NULL )
*     {
*         // The timer was not created.
*     }
*     else
*     {
*         // Start the timer.  No block time is specified, and even if one was
*         // it would be ignored because the scheduler has not yet been
*         // started.
*         if( xTimerStart( xBacklightTimer, 0 ) != pdPASS )
*         {
*             // The timer could not be set into the Active state.
*         }
```

```
*     }
*
*     // ...
*     // Create tasks here.
*     // ...
*
*     // Starting the scheduler will start the timer running as it has already
*     // been set into the active state.
*     vTaskStartScheduler();
*
*     // Should not reach here.
*     for( ;; );
* }
*
```

Parameters:

- **xTimer** -- The handle of the timer being reset/started/restarted.
- **xTicksToWait** -- Specifies the time, in ticks, that the calling task should be held in the Blocked state to wait for the reset command to be successfully sent to the timer command queue, should the queue already be full when xTimerReset() was called. xTicksToWait is ignored if xTimerReset() is called before the scheduler is started.

Returns: pdFAIL will be returned if the reset command could not be sent to the timer command queue even after xTicksToWait ticks had passed. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system, although the timers expiry time is relative to when xTimerStart() is actually called. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

`xTimerStartFromISR`(xTimer, pxHigherPriorityTaskWoken)

A version of xTimerStart() that can be called from an interrupt service routine.

Example usage:

```
* // This scenario assumes xBacklightTimer has already been created.  When a
* // key is pressed, an LCD back-light is switched on.  If 5 seconds pass
* // without a key being pressed, then the LCD back-light is switched off.  In
* // this case, the timer is a one-shot timer, and unlike the example given for
* // the xTimerReset() function, the key press event handler is an interrupt
* // service routine.
*
* // The callback function assigned to the one-shot timer.  In this case the
* // parameter is not used.
* void vBacklightTimerCallback( TimerHandle_t pxTimer )
* {
*     // The timer expired, therefore 5 seconds must have passed since a key
*     // was pressed.  Switch off the LCD back-light.
*     vSetBacklightState( BACKLIGHT_OFF );
* }
*
* // The key press interrupt service routine.
* void vKeyPressEventInterruptHandler( void )
* {
* BaseType_t xHigherPriorityTaskWoken = pdFALSE;
*
*     // Ensure the LCD back-light is on, then restart the timer that is
*     // responsible for turning the back-light off after 5 seconds of
*     // key inactivity.  This is an interrupt service routine so can only
*     // call FreeRTOS API functions that end in "FromISR".
*     vSetBacklightState( BACKLIGHT_ON );
*
*     // xTimerStartFromISR() or xTimerResetFromISR() could be called here
*     // as both cause the timer to re-calculate its expiry time.
*     // xHigherPriorityTaskWoken was initialised to pdFALSE when it was
*     // declared (in this function).
*     if( xTimerStartFromISR( xBacklightTimer, &xHigherPriorityTaskWoken ) !=
pdPASS )
*     {
*         // The start command was not executed successfully.  Take appropriate
*         // action here.
*     }
*
*     // Perform the rest of the key processing here.
*
*     // If xHigherPriorityTaskWoken equals pdTRUE, then a context switch
*     // should be performed.  The syntax required to perform a context switch
*     // from inside an ISR varies from port to port, and from compiler to
*     // compiler.  Inspect the demos for the port you are using to find the
*     // actual syntax required.
*     if( xHigherPriorityTaskWoken != pdFALSE )
*     {
*         // Call the interrupt safe yield function here (actual function
*         // depends on the FreeRTOS port being used).
*     }
* }
*
```

Parameters:

- **xTimer** -- The handle of the timer being started/restarted.
- **pxHigherPriorityTaskWoken** -- The timer service/daemon task spends most of its time in the Blocked state, waiting for messages to arrive on the timer command queue. Calling xTimerStartFromISR() writes a message to the timer command queue, so has the potential

to transition the timer service/daemon task out of the Blocked state. If calling xTimerStartFromISR() causes the timer service/daemon task to leave the Blocked state, and the timer service/ daemon task has a priority equal to or greater than the currently executing task (the task that was interrupted), then *pxHigherPriorityTaskWoken will get set to pdTRUE internally within the xTimerStartFromISR() function. If xTimerStartFromISR() sets this value to pdTRUE then a context switch should be performed before the interrupt exits.

Returns: pdFAIL will be returned if the start command could not be sent to the timer command queue. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system, although the timers expiry time is relative to when xTimerStartFromISR() is actually called. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

`xTimerStopFromISR``(xTimer, pxHigherPriorityTaskWoken)`

A version of xTimerStop() that can be called from an interrupt service routine.

Example usage:

```
* // This scenario assumes xTimer has already been created and started.  When
* // an interrupt occurs, the timer should be simply stopped.
*
* // The interrupt service routine that stops the timer.
* void vAnExampleInterruptServiceRoutine( void )
* {
* BaseType_t xHigherPriorityTaskWoken = pdFALSE;
*
*     // The interrupt has occurred - simply stop the timer.
*     // xHigherPriorityTaskWoken was set to pdFALSE where it was defined
*     // (within this function).  As this is an interrupt service routine, only
*     // FreeRTOS API functions that end in "FromISR" can be used.
*     if( xTimerStopFromISR( xTimer, &xHigherPriorityTaskWoken ) != pdPASS )
*     {
*         // The stop command was not executed successfully.  Take appropriate
*         // action here.
*     }
*
*     // If xHigherPriorityTaskWoken equals pdTRUE, then a context switch
*     // should be performed.  The syntax required to perform a context switch
*     // from inside an ISR varies from port to port, and from compiler to
*     // compiler.  Inspect the demos for the port you are using to find the
*     // actual syntax required.
*     if( xHigherPriorityTaskWoken != pdFALSE )
*     {
*         // Call the interrupt safe yield function here (actual function
*         // depends on the FreeRTOS port being used).
*     }
* }
*
```

Parameters:

- **xTimer** -- The handle of the timer being stopped.
- **pxHigherPriorityTaskWoken** -- The timer service/daemon task spends most of its time in the Blocked state, waiting for messages to arrive on the timer command queue. Calling xTimerStopFromISR() writes a message to the timer command queue, so has the potential to transition the timer service/daemon task out of the Blocked state. If calling xTimerStopFromISR() causes the timer service/daemon task to leave the Blocked state, and the timer service/ daemon task has a priority equal to or greater than the currently executing task (the task that was interrupted), then *pxHigherPriorityTaskWoken will get set to pdTRUE internally within the xTimerStopFromISR() function. If xTimerStopFromISR() sets this value to pdTRUE then a context switch should be performed before the interrupt exits.

Returns: pdFAIL will be returned if the stop command could not be sent to the timer command queue. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

xTimerChangePeriodFromISR(xTimer, xNewPeriod, pxHigherPriorityTaskWoken)

A version of xTimerChangePeriod() that can be called from an interrupt service routine.

Example usage:

```
* // This scenario assumes xTimer has already been created and started.  When
* // an interrupt occurs, the period of xTimer should be changed to 500ms.
*
* // The interrupt service routine that changes the period of xTimer.
* void vAnExampleInterruptServiceRoutine( void )
* {
* BaseType_t xHigherPriorityTaskWoken = pdFALSE;
*
*     // The interrupt has occurred - change the period of xTimer to 500ms.
*     // xHigherPriorityTaskWoken was set to pdFALSE where it was defined
*     // (within this function).  As this is an interrupt service routine, only
*     // FreeRTOS API functions that end in "FromISR" can be used.
*     if( xTimerChangePeriodFromISR( xTimer, &xHigherPriorityTaskWoken ) != pdPASS
)
*     {
*         // The command to change the timers period was not executed
*         // successfully.  Take appropriate action here.
*     }
*
*     // If xHigherPriorityTaskWoken equals pdTRUE, then a context switch
*     // should be performed.  The syntax required to perform a context switch
*     // from inside an ISR varies from port to port, and from compiler to
*     // compiler.  Inspect the demos for the port you are using to find the
*     // actual syntax required.
*     if( xHigherPriorityTaskWoken != pdFALSE )
*     {
*         // Call the interrupt safe yield function here (actual function
*         // depends on the FreeRTOS port being used).
*     }
* }
*
```

Parameters:

- **xTimer** -- The handle of the timer that is having its period changed.
- **xNewPeriod** -- The new period for xTimer. Timer periods are specified in tick periods, so the constant portTICK_PERIOD_MS can be used to convert a time that has been specified in milliseconds. For example, if the timer must expire after 100 ticks, then xNewPeriod should be set to 100. Alternatively, if the timer must expire after 500ms, then xNewPeriod can be set to (500 / portTICK_PERIOD_MS) provided configTICK_RATE_HZ is less than or equal to 1000.
- **pxHigherPriorityTaskWoken** -- The timer service/daemon task spends most of its time in the Blocked state, waiting for messages to arrive on the timer command queue. Calling xTimerChangePeriodFromISR() writes a message to the timer command queue, so has the potential to transition the timer service/ daemon task out of the Blocked state. If calling

xTimerChangePeriodFromISR() causes the timer service/daemon task to leave the Blocked state, and the timer service/daemon task has a priority equal to or greater than the currently executing task (the task that was interrupted), then *pxHigherPriorityTaskWoken will get set to pdTRUE internally within the xTimerChangePeriodFromISR() function. If xTimerChangePeriodFromISR() sets this value to pdTRUE then a context switch should be performed before the interrupt exits.

Returns: pdFAIL will be returned if the command to change the timers period could not be sent to the timer command queue. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

`xTimerResetFromISR`(xTimer, pxHigherPriorityTaskWoken)

A version of xTimerReset() that can be called from an interrupt service routine.

Example usage:

```
* // This scenario assumes xBacklightTimer has already been created.  When a
* // key is pressed, an LCD back-light is switched on.  If 5 seconds pass
* // without a key being pressed, then the LCD back-light is switched off.  In
* // this case, the timer is a one-shot timer, and unlike the example given for
* // the xTimerReset() function, the key press event handler is an interrupt
* // service routine.
*
* // The callback function assigned to the one-shot timer.  In this case the
* // parameter is not used.
* void vBacklightTimerCallback( TimerHandle_t pxTimer )
* {
*     // The timer expired, therefore 5 seconds must have passed since a key
*     // was pressed.  Switch off the LCD back-light.
*     vSetBacklightState( BACKLIGHT_OFF );
* }
*
* // The key press interrupt service routine.
* void vKeyPressEventInterruptHandler( void )
* {
* BaseType_t xHigherPriorityTaskWoken = pdFALSE;
*
*     // Ensure the LCD back-light is on, then reset the timer that is
*     // responsible for turning the back-light off after 5 seconds of
*     // key inactivity.  This is an interrupt service routine so can only
*     // call FreeRTOS API functions that end in "FromISR".
*     vSetBacklightState( BACKLIGHT_ON );
*
*     // xTimerStartFromISR() or xTimerResetFromISR() could be called here
*     // as both cause the timer to re-calculate its expiry time.
*     // xHigherPriorityTaskWoken was initialised to pdFALSE when it was
*     // declared (in this function).
*     if( xTimerResetFromISR( xBacklightTimer, &xHigherPriorityTaskWoken ) !=
pdPASS )
*     {
*         // The reset command was not executed successfully.  Take appropriate
*         // action here.
*     }
*
*     // Perform the rest of the key processing here.
*
*     // If xHigherPriorityTaskWoken equals pdTRUE, then a context switch
*     // should be performed.  The syntax required to perform a context switch
*     // from inside an ISR varies from port to port, and from compiler to
*     // compiler.  Inspect the demos for the port you are using to find the
*     // actual syntax required.
*     if( xHigherPriorityTaskWoken != pdFALSE )
*     {
*         // Call the interrupt safe yield function here (actual function
*         // depends on the FreeRTOS port being used).
*     }
* }
*
```

Parameters:

- **xTimer** -- The handle of the timer that is to be started, reset, or restarted.
- **pxHigherPriorityTaskWoken** -- The timer service/daemon task spends most of its time in the Blocked state, waiting for messages to arrive on the timer command queue. Calling xTimerResetFromISR()

writes a message to the timer command queue, so has the potential to transition the timer service/daemon task out of the Blocked state. If calling xTimerResetFromISR() causes the timer service/daemon task to leave the Blocked state, and the timer service/ daemon task has a priority equal to or greater than the currently executing task (the task that was interrupted), then *pxHigherPriorityTaskWoken will get set to pdTRUE internally within the xTimerResetFromISR() function. If xTimerResetFromISR() sets this value to pdTRUE then a context switch should be performed before the interrupt exits.

Returns: pdFAIL will be returned if the reset command could not be sent to the timer command queue. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system, although the timers expiry time is relative to when xTimerResetFromISR() is actually called. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

Type Definitions

typedef struct tmrTimerControl ***TimerHandle_t**

typedef void (***TimerCallbackFunction_t**)(TimerHandle_t xTimer)

Defines the prototype to which timer callback functions must conform.

typedef void (***PendedFunction_t**)(void*, uint32_t)

Defines the prototype to which functions used with the xTimerPendFunctionCallFromISR() function must conform.

Event Group API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/event_groups.h
- This header file can be included with:

```
#include "freertos/event_groups.h"
```

Functions

EventGroupHandle_t **xEventGroupCreate**(void)

Create a new event group.

Internally, within the FreeRTOS implementation, event groups use a [small] block of memory, in which the event group's structure is stored. If an event groups is created using xEventGroupCreate() then the required memory is automatically dynamically allocated inside the xEventGroupCreate() function. (see https://www.FreeRTOS.org/a00111.html). If an event group is created using xEventGroupCreateStatic() then the application writer must instead provide the memory that will get used by the event group. xEventGroupCreateStatic() therefore allows an event group to be created without using any dynamic memory allocation.

Although event groups are not related to ticks, for internal implementation reasons the number of bits available for use in an event group is dependent on the configUSE_16_BIT_TICKS setting in FreeRTOSConfig.h. If configUSE_16_BIT_TICKS is 1 then each event group contains 8 usable bits (bit 0 to bit 7). If configUSE_16_BIT_TICKS is set to 0 then each event group has 24 usable bits (bit 0 to bit 23). The EventBits_t type is used to store event bits within an event group.

Example usage:

```
// Declare a variable to hold the created event group.
EventGroupHandle_t xCreatedEventGroup;

// Attempt to create the event group.
xCreatedEventGroup = xEventGroupCreate();

// Was the event group created successfully?
if( xCreatedEventGroup == NULL )
{
// The event group was not created because there was insufficient
// FreeRTOS heap available.
}
else
{
// The event group was created.
}
```

Returns: If the event group was created then a handle to the event group is returned. If there was insufficient FreeRTOS heap available to create the event group then NULL is returned. See https://www.FreeRTOS.org/a00111.html

EventGroupHandle_t **xEventGroupCreateStatic**(StaticEventGroup_t *pxEventGroupBuffer)

Create a new event group.

Internally, within the FreeRTOS implementation, event groups use a [small] block of memory, in which the event group's structure is stored. If an event groups is created using xEventGroupCreate() then the required memory is automatically dynamically allocated inside the xEventGroupCreate() function. (see https://www.FreeRTOS.org/a00111.html). If an event group is created using xEventGroupCreateStatic() then the application writer must instead provide the memory that will get used by the event group. xEventGroupCreateStatic() therefore allows an event group to be created without using any dynamic memory allocation.

Although event groups are not related to ticks, for internal implementation reasons the number of bits available for use in an event group is dependent on the configUSE_16_BIT_TICKS setting in FreeRTOSConfig.h. If configUSE_16_BIT_TICKS is 1 then each event group contains 8 usable bits (bit 0 to bit 7). If configUSE_16_BIT_TICKS is set to 0 then each event group has 24 usable bits (bit 0 to bit 23). The EventBits_t type is used to store event bits within an event group.

Example usage:

```
// StaticEventGroup_t is a publicly accessible structure that has the same
// size and alignment requirements as the real event group structure.  It is
// provided as a mechanism for applications to know the size of the event
// group (which is dependent on the architecture and configuration file
// settings) without breaking the strict data hiding policy by exposing the
// real event group internals.  This StaticEventGroup_t variable is passed
// into the xSemaphoreCreateEventGroupStatic() function and is used to store
// the event group's data structures
StaticEventGroup_t xEventGroupBuffer;

// Create the event group without dynamically allocating any memory.
xEventGroup = xEventGroupCreateStatic( &xEventGroupBuffer );
```

Parameters:	**pxEventGroupBuffer** -- pxEventGroupBuffer must point to a variable of type StaticEventGroup_t, which will be then be used to hold the event group's data structures, removing the need for the memory to be allocated dynamically.
Returns:	If the event group was created then a handle to the event group is returned. If pxEventGroupBuffer was NULL then NULL is returned.

EventBits_t **xEventGroupWaitBits**(EventGroupHandle_t xEventGroup, *const* EventBits_t uxBitsToWaitFor, *const* BaseType_t xClearOnExit, *const* BaseType_t xWaitForAllBits, TickType_t xTicksToWait)

[Potentially] block to wait for one or more bits to be set within a previously created event group.

This function cannot be called from an interrupt.

Example usage:

```
#define BIT_0 ( 1 << 0 )
#define BIT_4 ( 1 << 4 )

void aFunction( EventGroupHandle_t xEventGroup )
{
EventBits_t uxBits;
const TickType_t xTicksToWait = 100 / portTICK_PERIOD_MS;

// Wait a maximum of 100ms for either bit 0 or bit 4 to be set within
// the event group.  Clear the bits before exiting.
     uxBits = xEventGroupWaitBits(
                 xEventGroup,    // The event group being tested.
                 BIT_0 | BIT_4,  // The bits within the event group to wait for.
                 pdTRUE,         // BIT_0 and BIT_4 should be cleared before
returning.
                 pdFALSE,        // Don't wait for both bits, either bit will do.
                 xTicksToWait ); // Wait a maximum of 100ms for either bit to be
set.

if( ( uxBits & ( BIT_0 | BIT_4 ) ) == ( BIT_0 | BIT_4 ) )
     {
// xEventGroupWaitBits() returned because both bits were set.
     }
else if( ( uxBits & BIT_0 ) != 0 )
     {
// xEventGroupWaitBits() returned because just BIT_0 was set.
     }
else if( ( uxBits & BIT_4 ) != 0 )
     {
// xEventGroupWaitBits() returned because just BIT_4 was set.
     }
else
     {
// xEventGroupWaitBits() returned because xTicksToWait ticks passed
// without either BIT_0 or BIT_4 becoming set.
     }
}
```

Parameters:

- **xEventGroup** -- The event group in which the bits are being tested. The event group must have previously been created using a call to xEventGroupCreate().
- **uxBitsToWaitFor** -- A bitwise value that indicates the bit or bits to test inside the event group. For example, to wait for bit 0 and/or bit 2 set uxBitsToWaitFor to 0x05. To wait for bits 0 and/or bit 1 and/or bit 2 set uxBitsToWaitFor to 0x07. Etc.
- **xClearOnExit** -- If xClearOnExit is set to pdTRUE then any bits within uxBitsToWaitFor that are set within the event group will be cleared before xEventGroupWaitBits() returns if the wait condition was met (if the function returns for a reason other than a timeout). If xClearOnExit is set to pdFALSE then the bits set in the event group are not altered when the call to xEventGroupWaitBits() returns.
- **xWaitForAllBits** -- If xWaitForAllBits is set to pdTRUE then xEventGroupWaitBits() will return when either all the bits in uxBitsToWaitFor are set or the specified block time expires. If

xWaitForAllBits is set to pdFALSE then xEventGroupWaitBits() will return when any one of the bits set in uxBitsToWaitFor is set or the specified block time expires. The block time is specified by the xTicksToWait parameter.

- **xTicksToWait** -- The maximum amount of time (specified in 'ticks') to wait for one/all (depending on the xWaitForAllBits value) of the bits specified by uxBitsToWaitFor to become set. A value of portMAX_DELAY can be used to block indefinitely (provided INCLUDE_vTaskSuspend is set to 1 in FreeRTOSConfig.h).

Returns: The value of the event group at the time either the bits being waited for became set, or the block time expired. Test the return value to know which bits were set. If xEventGroupWaitBits() returned because its timeout expired then not all the bits being waited for will be set. If xEventGroupWaitBits() returned because the bits it was waiting for were set then the returned value is the event group value before any bits were automatically cleared in the case that xClearOnExit parameter was set to pdTRUE.

EventBits_t **xEventGroupClearBits**(EventGroupHandle_t xEventGroup, *const* EventBits_t uxBitsToClear)

Clear bits within an event group. This function cannot be called from an interrupt.

Example usage:

```
#define BIT_0 ( 1 << 0 )
#define BIT_4 ( 1 << 4 )

void aFunction( EventGroupHandle_t xEventGroup )
{
EventBits_t uxBits;

// Clear bit 0 and bit 4 in xEventGroup.
     uxBits = xEventGroupClearBits(
                              xEventGroup,    // The event group being updated.
                              BIT_0 | BIT_4 );// The bits being cleared.

if( ( uxBits & ( BIT_0 | BIT_4 ) ) == ( BIT_0 | BIT_4 ) )
     {
// Both bit 0 and bit 4 were set before xEventGroupClearBits() was
// called.  Both will now be clear (not set).
     }
else if( ( uxBits & BIT_0 ) != 0 )
     {
// Bit 0 was set before xEventGroupClearBits() was called.  It will
// now be clear.
     }
else if( ( uxBits & BIT_4 ) != 0 )
     {
// Bit 4 was set before xEventGroupClearBits() was called.  It will
// now be clear.
     }
else
     {
// Neither bit 0 nor bit 4 were set in the first place.
     }
}
```

Parameters:

- **xEventGroup** -- The event group in which the bits are to be cleared.
- **uxBitsToClear** -- A bitwise value that indicates the bit or bits to clear in the event group. For example, to clear bit 3 only, set uxBitsToClear to 0x08. To clear bit 3 and bit 0 set uxBitsToClear to 0x09.

Returns: The value of the event group before the specified bits were cleared.

EventBits_t **xEventGroupSetBits**(EventGroupHandle_t xEventGroup, *const* EventBits_t uxBitsToSet)

Set bits within an event group. This function cannot be called from an interrupt. xEventGroupSetBitsFromISR() is a version that can be called from an interrupt.

Setting bits in an event group will automatically unblock tasks that are blocked waiting for the bits.

Example usage:

```
#define BIT_0 ( 1 << 0 )
#define BIT_4 ( 1 << 4 )

void aFunction( EventGroupHandle_t xEventGroup )
{
EventBits_t uxBits;

// Set bit 0 and bit 4 in xEventGroup.
     uxBits = xEventGroupSetBits(
                          xEventGroup,    // The event group being updated.
                          BIT_0 | BIT_4 );// The bits being set.

if( ( uxBits & ( BIT_0 | BIT_4 ) ) == ( BIT_0 | BIT_4 ) )
     {
// Both bit 0 and bit 4 remained set when the function returned.
     }
else if( ( uxBits & BIT_0 ) != 0 )
     {
// Bit 0 remained set when the function returned, but bit 4 was
// cleared.  It might be that bit 4 was cleared automatically as a
// task that was waiting for bit 4 was removed from the Blocked
// state.
     }
else if( ( uxBits & BIT_4 ) != 0 )
     {
// Bit 4 remained set when the function returned, but bit 0 was
// cleared.  It might be that bit 0 was cleared automatically as a
// task that was waiting for bit 0 was removed from the Blocked
// state.
     }
else
     {
// Neither bit 0 nor bit 4 remained set.  It might be that a task
// was waiting for both of the bits to be set, and the bits were
// cleared as the task left the Blocked state.
     }
}
```

Parameters:

- **xEventGroup** -- The event group in which the bits are to be set.
- **uxBitsToSet** -- A bitwise value that indicates the bit or bits to set. For example, to set bit 3 only, set uxBitsToSet to 0x08. To set bit 3 and bit 0 set uxBitsToSet to 0x09.

Returns:

The value of the event group at the time the call to xEventGroupSetBits() returns. There are two reasons why the returned value might have the bits specified by the uxBitsToSet parameter cleared. First, if setting a bit results in a task that was waiting for the bit leaving the blocked state then it is possible the bit will be cleared automatically (see the xClearBitOnExit parameter of xEventGroupWaitBits()). Second, any unblocked (or otherwise Ready state) task that has a priority above that of the task that called xEventGroupSetBits() will execute and may change the event group value before the call to xEventGroupSetBits() returns.

EventBits_t **xEventGroupSync**(EventGroupHandle_t xEventGroup, *const* EventBits_t uxBitsToSet, *const* EventBits_t uxBitsToWaitFor, TickType_t xTicksToWait)

Atomically set bits within an event group, then wait for a combination of bits to be set within the same event group. This functionality is typically used to synchronise multiple tasks, where each task has to wait for the other tasks to reach a synchronisation point before proceeding.

This function cannot be used from an interrupt.

The function will return before its block time expires if the bits specified by the uxBitsToWait parameter are set, or become set within that time. In this case all the bits specified by uxBitsToWait will be automatically cleared before the function returns.

Example usage:

```
// Bits used by the three tasks.
#define TASK_0_BIT     ( 1 << 0 )
#define TASK_1_BIT     ( 1 << 1 )
#define TASK_2_BIT     ( 1 << 2 )

#define ALL_SYNC_BITS ( TASK_0_BIT | TASK_1_BIT | TASK_2_BIT )

// Use an event group to synchronise three tasks.  It is assumed this event
// group has already been created elsewhere.
EventGroupHandle_t xEventBits;

void vTask0( void *pvParameters )
{
EventBits_t uxReturn;
TickType_t xTicksToWait = 100 / portTICK_PERIOD_MS;

for( ;; )
  {
// Perform task functionality here.

// Set bit 0 in the event flag to note this task has reached the
// sync point.  The other two tasks will set the other two bits defined
// by ALL_SYNC_BITS.  All three tasks have reached the synchronisation
// point when all the ALL_SYNC_BITS are set.  Wait a maximum of 100ms
// for this to happen.
     uxReturn = xEventGroupSync( xEventBits, TASK_0_BIT, ALL_SYNC_BITS,
xTicksToWait );

if( ( uxReturn & ALL_SYNC_BITS ) == ALL_SYNC_BITS )
     {
// All three tasks reached the synchronisation point before the call
// to xEventGroupSync() timed out.
     }
 }
}

void vTask1( void *pvParameters )
{
for( ;; )
  {
// Perform task functionality here.

// Set bit 1 in the event flag to note this task has reached the
// synchronisation point.  The other two tasks will set the other two
// bits defined by ALL_SYNC_BITS.  All three tasks have reached the
// synchronisation point when all the ALL_SYNC_BITS are set.  Wait
// indefinitely for this to happen.
     xEventGroupSync( xEventBits, TASK_1_BIT, ALL_SYNC_BITS, portMAX_DELAY );

// xEventGroupSync() was called with an indefinite block time, so
// this task will only reach here if the synchronisation was made by all
// three tasks, so there is no need to test the return value.
  }
}

void vTask2( void *pvParameters )
{
for( ;; )
  {
// Perform task functionality here.

// Set bit 2 in the event flag to note this task has reached the
// synchronisation point.  The other two tasks will set the other two
```

```
// bits defined by ALL_SYNC_BITS.  All three tasks have reached the
// synchronisation point when all the ALL_SYNC_BITS are set.  Wait
// indefinitely for this to happen.
     xEventGroupSync( xEventBits, TASK_2_BIT, ALL_SYNC_BITS, portMAX_DELAY );

// xEventGroupSync() was called with an indefinite block time, so
// this task will only reach here if the synchronisation was made by all
// three tasks, so there is no need to test the return value.
 }
}
```

Parameters:

- **xEventGroup** -- The event group in which the bits are being tested. The event group must have previously been created using a call to xEventGroupCreate().
- **uxBitsToSet** -- The bits to set in the event group before determining if, and possibly waiting for, all the bits specified by the uxBitsToWait parameter are set.
- **uxBitsToWaitFor** -- A bitwise value that indicates the bit or bits to test inside the event group. For example, to wait for bit 0 and bit 2 set uxBitsToWaitFor to 0x05. To wait for bits 0 and bit 1 and bit 2 set uxBitsToWaitFor to 0x07. Etc.
- **xTicksToWait** -- The maximum amount of time (specified in 'ticks') to wait for all of the bits specified by uxBitsToWaitFor to become set.

Returns: The value of the event group at the time either the bits being waited for became set, or the block time expired. Test the return value to know which bits were set. If xEventGroupSync() returned because its timeout expired then not all the bits being waited for will be set. If xEventGroupSync() returned because all the bits it was waiting for were set then the returned value is the event group value before any bits were automatically cleared.

EventBits_t **xEventGroupGetBitsFromISR**(EventGroupHandle_t xEventGroup)

A version of xEventGroupGetBits() that can be called from an ISR.

Parameters: **xEventGroup** -- The event group being queried.

Returns: The event group bits at the time xEventGroupGetBitsFromISR() was called.

---

void **vEventGroupDelete**(EventGroupHandle_t xEventGroup)

Delete an event group that was previously created by a call to xEventGroupCreate(). Tasks that are blocked on the event group will be unblocked and obtain 0 as the event group's value.

Parameters: **xEventGroup** -- The event group being deleted.

BaseType_t **xEventGroupGetStaticBuffer**(EventGroupHandle_t xEventGroup, StaticEventGroup_t **ppxEventGroupBuffer)

Retrieve a pointer to a statically created event groups's data structure buffer. It is the same buffer that is supplied at the time of creation.

Parameters:
- **xEventGroup** -- The event group for which to retrieve the buffer.
- **ppxEventGroupBuffer** -- Used to return a pointer to the event groups's data structure buffer.

Returns: pdTRUE if the buffer was retrieved, pdFALSE otherwise.

Macros

xEventGroupClearBitsFromISR(xEventGroup, uxBitsToClear)

A version of xEventGroupClearBits() that can be called from an interrupt.

Setting bits in an event group is not a deterministic operation because there are an unknown number of tasks that may be waiting for the bit or bits being set. FreeRTOS does not allow nondeterministic operations to be performed while interrupts are disabled, so protects event groups that are accessed from tasks by suspending the scheduler rather than disabling interrupts. As a result event groups cannot be accessed directly from an interrupt service routine. Therefore xEventGroupClearBitsFromISR() sends a message to the timer task to have the clear operation performed in the context of the timer task.

Example usage:

```
#define BIT_0 ( 1 << 0 )
#define BIT_4 ( 1 << 4 )

// An event group which it is assumed has already been created by a call to
// xEventGroupCreate().
EventGroupHandle_t xEventGroup;

void anInterruptHandler( void )
{
// Clear bit 0 and bit 4 in xEventGroup.
     xResult = xEventGroupClearBitsFromISR(
                         xEventGroup,     // The event group being updated.
                         BIT_0 | BIT_4 ); // The bits being set.

if( xResult == pdPASS )
     {
// The message was posted successfully.
          portYIELD_FROM_ISR(pdTRUE);
     }
}
```

Note

If this function returns pdPASS then the timer task is ready to run and a portYIELD_FROM_ISR(pdTRUE) should be executed to perform the needed clear on the event group. This behavior is different from xEventGroupSetBitsFromISR because the parameter xHigherPriorityTaskWoken is not present.

Parameters:
- **xEventGroup** -- The event group in which the bits are to be cleared.
- **uxBitsToClear** -- A bitwise value that indicates the bit or bits to clear. For example, to clear bit 3 only, set uxBitsToClear to 0x08. To clear bit 3 and bit 0 set uxBitsToClear to 0x09.

Returns: If the request to execute the function was posted successfully then pdPASS is returned, otherwise pdFALSE is returned. pdFALSE will be returned if the timer service queue was full.

`xEventGroupSetBitsFromISR`(xEventGroup, uxBitsToSet, pxHigherPriorityTaskWoken)

A version of xEventGroupSetBits() that can be called from an interrupt.

Setting bits in an event group is not a deterministic operation because there are an unknown number of tasks that may be waiting for the bit or bits being set. FreeRTOS does not allow nondeterministic operations to be performed in interrupts or from critical sections. Therefore xEventGroupSetBitsFromISR() sends a message to the timer task to have the set operation performed in the context of the timer task - where a scheduler lock is used in place of a critical section.

Example usage:

```
#define BIT_0 ( 1 << 0 )
#define BIT_4 ( 1 << 4 )

// An event group which it is assumed has already been created by a call to
// xEventGroupCreate().
EventGroupHandle_t xEventGroup;

void anInterruptHandler( void )
{
BaseType_t xHigherPriorityTaskWoken, xResult;

// xHigherPriorityTaskWoken must be initialised to pdFALSE.
     xHigherPriorityTaskWoken = pdFALSE;

// Set bit 0 and bit 4 in xEventGroup.
     xResult = xEventGroupSetBitsFromISR(
                         xEventGroup,    // The event group being updated.
                         BIT_0 | BIT_4   // The bits being set.
                         &xHigherPriorityTaskWoken );

// Was the message posted successfully?
if( xResult == pdPASS )
     {
// If xHigherPriorityTaskWoken is now set to pdTRUE then a context
// switch should be requested.  The macro used is port specific and
// will be either portYIELD_FROM_ISR() or portEND_SWITCHING_ISR() -
// refer to the documentation page for the port being used.
          portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
     }
}
```

Parameters:

- **xEventGroup** -- The event group in which the bits are to be set.
- **uxBitsToSet** -- A bitwise value that indicates the bit or bits to set. For example, to set bit 3 only, set uxBitsToSet to 0x08. To set bit 3 and bit 0 set uxBitsToSet to 0x09.
- **pxHigherPriorityTaskWoken** -- As mentioned above, calling this function will result in a message being sent to the timer daemon task. If the priority of the timer daemon task is higher than the priority of the currently running task (the task the interrupt interrupted) then *pxHigherPriorityTaskWoken will be set to pdTRUE by xEventGroupSetBitsFromISR(), indicating that a context switch should be requested before the interrupt exits. For that reason *pxHigherPriorityTaskWoken must be initialised to pdFALSE. See the example code below.

Returns: If the request to execute the function was posted successfully then pdPASS is returned, otherwise pdFALSE is returned. pdFALSE will be returned if the timer service queue was full.

`xEventGroupGetBits`(xEventGroup)

Returns the current value of the bits in an event group. This function cannot be used from an interrupt.

Parameters:
- **xEventGroup** -- The event group being queried.

Returns: The event group bits at the time xEventGroupGetBits() was called.

Type Definitions

typedef struct EventGroupDef_t ***EventGroupHandle_t**

typedef TickType_t **EventBits_t**

Stream Buffer API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/stream_buffer.h
- This header file can be included with:

```
#include "freertos/stream_buffer.h"
```

Functions

BaseType_t **xStreamBufferGetStaticBuffers**(StreamBufferHandle_t xStreamBuffer, uint8_t **ppucStreamBufferStorageArea, StaticStreamBuffer_t **ppxStaticStreamBuffer)

Retrieve pointers to a statically created stream buffer's data structure buffer and storage area buffer. These are the same buffers that are supplied at the time of creation.

Parameters:
- **xStreamBuffer** -- The stream buffer for which to retrieve the buffers.
- **ppucStreamBufferStorageArea** -- Used to return a pointer to the stream buffer's storage area buffer.
- **ppxStaticStreamBuffer** -- Used to return a pointer to the stream buffer's data structure buffer.

Returns: pdTRUE if buffers were retrieved, pdFALSE otherwise.

size_t **xStreamBufferSend**(StreamBufferHandle_t xStreamBuffer, *const* void *pvTxData, size_t xDataLengthBytes, TickType_t xTicksToWait)

Sends bytes to a stream buffer. The bytes are copied into the stream buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to

be multiple different writers then the application writer must place each call to a writing API function (such as xStreamBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers then the application writer must place each call to a reading API function (such as xStreamBufferReceive()) inside a critical section and set the receive block time to 0.

Use xStreamBufferSend() to write to a stream buffer from a task. Use xStreamBufferSendFromISR() to write to a stream buffer from an interrupt service routine (ISR).

Example use:

```
void vAFunction( StreamBufferHandle_t xStreamBuffer )
{
size_t xBytesSent;
uint8_t ucArrayToSend[] = { 0, 1, 2, 3 };
char *pcStringToSend = "String to send";
const TickType_t x100ms = pdMS_TO_TICKS( 100 );

 // Send an array to the stream buffer, blocking for a maximum of 100ms to
 // wait for enough space to be available in the stream buffer.
 xBytesSent = xStreamBufferSend( xStreamBuffer, ( void * ) ucArrayToSend, sizeof(
ucArrayToSend ), x100ms );

 if( xBytesSent != sizeof( ucArrayToSend ) )
 {
// The call to xStreamBufferSend() times out before there was enough
// space in the buffer for the data to be written, but it did
// successfully write xBytesSent bytes.
 }

 // Send the string to the stream buffer.  Return immediately if there is not
 // enough space in the buffer.
 xBytesSent = xStreamBufferSend( xStreamBuffer, ( void * ) pcStringToSend, strlen(
pcStringToSend ), 0 );

 if( xBytesSent != strlen( pcStringToSend ) )
 {
// The entire string could not be added to the stream buffer because
// there was not enough free space in the buffer, but xBytesSent bytes
// were sent.  Could try again to send the remaining bytes.
 }
}
```

Parameters:

- **xStreamBuffer** -- The handle of the stream buffer to which a stream is being sent.
- **pvTxData** -- A pointer to the buffer that holds the bytes to be copied into the stream buffer.
- **xDataLengthBytes** -- The maximum number of bytes to copy from pvTxData into the stream buffer.
- **xTicksToWait** -- The maximum amount of time the task should remain in the Blocked state to wait for enough space to become available in the stream buffer, should the stream buffer contain too

little space to hold the another xDataLengthBytes bytes. The block time is specified in tick periods, so the absolute time it represents is dependent on the tick frequency. The macro pdMS_TO_TICKS() can be used to convert a time specified in milliseconds into a time specified in ticks. Setting xTicksToWait to portMAX_DELAY will cause the task to wait indefinitely (without timing out), provided INCLUDE_vTaskSuspend is set to 1 in FreeRTOSConfig.h. If a task times out before it can write all xDataLengthBytes into the buffer it will still write as many bytes as possible. A task does not use any CPU time when it is in the blocked state.

Returns: The number of bytes written to the stream buffer. If a task times out before it can write all xDataLengthBytes into the buffer it will still write as many bytes as possible.

---

size_t **xStreamBufferSendFromISR**(StreamBufferHandle_t xStreamBuffer, *const* void *pvTxData, size_t xDataLengthBytes, BaseType_t **const* pxHigherPriorityTaskWoken)

Interrupt safe version of the API function that sends a stream of bytes to the stream buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to be multiple different writers then the application writer must place each call to a writing API function (such as xStreamBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers then the application writer must place each call to a reading API function (such as xStreamBufferReceive()) inside a critical section and set the receive block time to 0.

Use xStreamBufferSend() to write to a stream buffer from a task. Use xStreamBufferSendFromISR() to write to a stream buffer from an interrupt service routine (ISR).

Example use:

```
// A stream buffer that has already been created.
StreamBufferHandle_t xStreamBuffer;

void vAnInterruptServiceRoutine( void )
{
size_t xBytesSent;
char *pcStringToSend = "String to send";
BaseType_t xHigherPriorityTaskWoken = pdFALSE; // Initialised to pdFALSE.

 // Attempt to send the string to the stream buffer.
 xBytesSent = xStreamBufferSendFromISR( xStreamBuffer,
                                        ( void * ) pcStringToSend,
                                        strlen( pcStringToSend ),
                                        &xHigherPriorityTaskWoken );

 if( xBytesSent != strlen( pcStringToSend ) )
 {
// There was not enough free space in the stream buffer for the entire
// string to be written, ut xBytesSent bytes were written.
 }

 // If xHigherPriorityTaskWoken was set to pdTRUE inside
 // xStreamBufferSendFromISR() then a task that has a priority above the
 // priority of the currently executing task was unblocked and a context
 // switch should be performed to ensure the ISR returns to the unblocked
 // task.  In most FreeRTOS ports this is done by simply passing
 // xHigherPriorityTaskWoken into portYIELD_FROM_ISR(), which will test the
 // variables value, and perform the context switch if necessary.  Check the
 // documentation for the port in use for port specific instructions.
 portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

Parameters:

- **xStreamBuffer** -- The handle of the stream buffer to which a stream is being sent.
- **pvTxData** -- A pointer to the data that is to be copied into the stream buffer.
- **xDataLengthBytes** -- The maximum number of bytes to copy from pvTxData into the stream buffer.
- **pxHigherPriorityTaskWoken** -- It is possible that a stream buffer will have a task blocked on it waiting for data. Calling xStreamBufferSendFromISR() can make data available, and so cause a task that was waiting for data to leave the Blocked state. If calling xStreamBufferSendFromISR() causes a task to leave the Blocked state, and the unblocked task has a priority higher than the currently executing task (the task that was interrupted), then, internally, xStreamBufferSendFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE. If xStreamBufferSendFromISR() sets this value to pdTRUE, then normally a context switch should be performed before the interrupt is exited. This will ensure that the interrupt returns directly to the highest priority Ready state task.

*pxHigherPriorityTaskWoken should be set to pdFALSE before it is passed into the function. See the example code below for an example.

Returns: The number of bytes actually written to the stream buffer, which will be less than xDataLengthBytes if the stream buffer didn't have enough free space for all the bytes to be written.

---

size_t **xStreamBufferReceive**(StreamBufferHandle_t xStreamBuffer, void *pvRxData, size_t xBufferLengthBytes, TickType_t xTicksToWait)

Receives bytes from a stream buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to be multiple different writers then the application writer must place each call to a writing API function (such as xStreamBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers then the application writer must place each call to a reading API function (such as xStreamBufferReceive()) inside a critical section and set the receive block time to 0.

Use xStreamBufferReceive() to read from a stream buffer from a task. Use xStreamBufferReceiveFromISR() to read from a stream buffer from an interrupt service routine (ISR).

Example use:

```
void vAFunction( StreamBuffer_t xStreamBuffer )
{
uint8_t ucRxData[ 20 ];
size_t xReceivedBytes;
const TickType_t xBlockTime = pdMS_TO_TICKS( 20 );

 // Receive up to another sizeof( ucRxData ) bytes from the stream buffer.
 // Wait in the Blocked state (so not using any CPU processing time) for a
 // maximum of 100ms for the full sizeof( ucRxData ) number of bytes to be
 // available.
 xReceivedBytes = xStreamBufferReceive( xStreamBuffer,
                                        ( void * ) ucRxData,
sizeof( ucRxData ),
                                        xBlockTime );

 if( xReceivedBytes > 0 )
 {
// A ucRxData contains another xReceivedBytes bytes of data, which can
// be processed here....
 }
}
```

Parameters:

- **xStreamBuffer** -- The handle of the stream buffer from which bytes are to be received.
- **pvRxData** -- A pointer to the buffer into which the received bytes will be copied.
- **xBufferLengthBytes** -- The length of the buffer pointed to by the pvRxData parameter. This sets the maximum number of bytes to receive in one call. xStreamBufferReceive will return as many bytes as possible up to a maximum set by xBufferLengthBytes.
- **xTicksToWait** -- The maximum amount of time the task should remain in the Blocked state to wait for data to become available if the stream buffer is empty. xStreamBufferReceive() will return immediately if xTicksToWait is zero. The block time is specified in tick periods, so the absolute time it represents is dependent on the tick frequency. The macro pdMS_TO_TICKS() can be used to convert a time specified in milliseconds into a time specified in ticks. Setting xTicksToWait to portMAX_DELAY will cause the task to wait indefinitely (without timing out), provided INCLUDE_vTaskSuspend is set to 1 in FreeRTOSConfig.h. A task does not use any CPU time when it is in the Blocked state.

Returns: The number of bytes actually read from the stream buffer, which will be less than xBufferLengthBytes if the call to xStreamBufferReceive() timed out before xBufferLengthBytes were available.

---

size_t **xStreamBufferReceiveFromISR**(StreamBufferHandle_t xStreamBuffer, void *pvRxData, size_t xBufferLengthBytes, BaseType_t **const** pxHigherPriorityTaskWoken)

An interrupt safe version of the API function that receives bytes from a stream buffer.

Use xStreamBufferReceive() to read bytes from a stream buffer from a task. Use xStreamBufferReceiveFromISR() to read bytes from a stream buffer from an interrupt service routine (ISR).

Example use:

```
// A stream buffer that has already been created.
StreamBuffer_t xStreamBuffer;

void vAnInterruptServiceRoutine( void )
{
uint8_t ucRxData[ 20 ];
size_t xReceivedBytes;
BaseType_t xHigherPriorityTaskWoken = pdFALSE;  // Initialised to pdFALSE.

 // Receive the next stream from the stream buffer.
 xReceivedBytes = xStreamBufferReceiveFromISR( xStreamBuffer,
                                               ( void * ) ucRxData,
sizeof( ucRxData ),
                                               &xHigherPriorityTaskWoken );

 if( xReceivedBytes > 0 )
 {
// ucRxData contains xReceivedBytes read from the stream buffer.
// Process the stream here....
 }

 // If xHigherPriorityTaskWoken was set to pdTRUE inside
 // xStreamBufferReceiveFromISR() then a task that has a priority above the
 // priority of the currently executing task was unblocked and a context
 // switch should be performed to ensure the ISR returns to the unblocked
 // task.  In most FreeRTOS ports this is done by simply passing
 // xHigherPriorityTaskWoken into portYIELD_FROM_ISR(), which will test the
 // variables value, and perform the context switch if necessary.  Check the
 // documentation for the port in use for port specific instructions.
 portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

Parameters:

- **xStreamBuffer** -- The handle of the stream buffer from which a stream is being received.
- **pvRxData** -- A pointer to the buffer into which the received bytes are copied.
- **xBufferLengthBytes** -- The length of the buffer pointed to by the pvRxData parameter. This sets the maximum number of bytes to receive in one call. xStreamBufferReceive will return as many bytes as possible up to a maximum set by xBufferLengthBytes.
- **pxHigherPriorityTaskWoken** -- It is possible that a stream buffer will have a task blocked on it waiting for space to become available. Calling xStreamBufferReceiveFromISR() can make space available, and so cause a task that is waiting for space to leave the Blocked state. If calling xStreamBufferReceiveFromISR() causes a task to leave the Blocked state, and the unblocked task has a priority higher than the currently executing task (the task that was interrupted), then, internally, xStreamBufferReceiveFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE. If xStreamBufferReceiveFromISR() sets this value to pdTRUE, then normally a context switch should be performed before the interrupt is exited. That will ensure the interrupt returns directly to the

highest priority Ready state task. *pxHigherPriorityTaskWoken should be set to pdFALSE before it is passed into the function. See the code example below for an example.

Returns: The number of bytes read from the stream buffer, if any.

---

void **vStreamBufferDelete**(StreamBufferHandle_t xStreamBuffer)

Deletes a stream buffer that was previously created using a call to xStreamBufferCreate() or xStreamBufferCreateStatic(). If the stream buffer was created using dynamic memory (that is, by xStreamBufferCreate()), then the allocated memory is freed.

A stream buffer handle must not be used after the stream buffer has been deleted.

Parameters: **xStreamBuffer** -- The handle of the stream buffer to be deleted.

BaseType_t **xStreamBufferIsFull**(StreamBufferHandle_t xStreamBuffer)

Queries a stream buffer to see if it is full. A stream buffer is full if it does not have any free space, and therefore cannot accept any more data.

Parameters: **xStreamBuffer** -- The handle of the stream buffer being queried.

Returns: If the stream buffer is full then pdTRUE is returned. Otherwise pdFALSE is returned.

BaseType_t **xStreamBufferIsEmpty**(StreamBufferHandle_t xStreamBuffer)

Queries a stream buffer to see if it is empty. A stream buffer is empty if it does not contain any data.

Parameters: **xStreamBuffer** -- The handle of the stream buffer being queried.

Returns: If the stream buffer is empty then pdTRUE is returned. Otherwise pdFALSE is returned.

BaseType_t **xStreamBufferReset**(StreamBufferHandle_t xStreamBuffer)

Resets a stream buffer to its initial, empty, state. Any data that was in the stream buffer is discarded. A stream buffer can only be reset if there are no tasks blocked waiting to either send to or receive from the stream buffer.

Parameters: **xStreamBuffer** -- The handle of the stream buffer being reset.

Returns: If the stream buffer is reset then pdPASS is returned. If there was a task blocked waiting to send to or read from the stream buffer then the stream buffer is not reset and pdFAIL is returned.

---

size_t **xStreamBufferSpacesAvailable**(StreamBufferHandle_t xStreamBuffer)

Queries a stream buffer to see how much free space it contains, which is equal to the amount of data that can be sent to the stream buffer before it is full.

Parameters: **xStreamBuffer** -- The handle of the stream buffer being queried.

Returns: The number of bytes that can be written to the stream buffer before the stream buffer would be full.

---

size_t **xStreamBufferBytesAvailable**(StreamBufferHandle_t xStreamBuffer)

Queries a stream buffer to see how much data it contains, which is equal to the number of bytes that can be read from the stream buffer before the stream buffer would be empty.

Parameters: **xStreamBuffer** -- The handle of the stream buffer being queried.

Returns: The number of bytes that can be read from the stream buffer before the stream buffer would be empty.

BaseType_t **xStreamBufferSetTriggerLevel**(StreamBufferHandle_t xStreamBuffer, size_t xTriggerLevel)

A stream buffer's trigger level is the number of bytes that must be in the stream buffer before a task that is blocked on the stream buffer to wait for data is moved out of the blocked state. For example, if a task is blocked on a read of an empty stream buffer that has a trigger level of 1 then the task will be unblocked when a single byte is written to the buffer or the task's block time expires. As another example, if a task is blocked on a read of an empty stream buffer that has a trigger level of 10 then the task will not be unblocked until the stream buffer contains at least 10 bytes or the task's block time expires. If a reading task's block time expires before the trigger level is reached then the task will still receive however many bytes are actually available. Setting a trigger level of 0 will result in a trigger level of 1 being used. It is not valid to specify a trigger level that is greater than the buffer size.

A trigger level is set when the stream buffer is created, and can be modified using xStreamBufferSetTriggerLevel().

Parameters:

- **xStreamBuffer** -- The handle of the stream buffer being updated.
- **xTriggerLevel** -- The new trigger level for the stream buffer.

Returns: If xTriggerLevel was less than or equal to the stream buffer's length then the trigger level will be updated and pdTRUE is returned. Otherwise pdFALSE is returned.

BaseType_t **xStreamBufferSendCompletedFromISR**(StreamBufferHandle_t xStreamBuffer, BaseType_t *pxHigherPriorityTaskWoken)

For advanced users only.

The sbSEND_COMPLETED() macro is called from within the FreeRTOS APIs when data is sent to a message buffer or stream buffer. If there was a task that was blocked on the message or stream buffer waiting for data to arrive then the sbSEND_COMPLETED() macro sends a notification to the task to remove it from the Blocked state.

xStreamBufferSendCompletedFromISR() does the same thing. It is provided to enable application writers to implement their own version of sbSEND_COMPLETED(), and MUST NOT BE USED AT ANY OTHER TIME.

See the example implemented in FreeRTOS/Demo/Minimal/MessageBufferAMP.c for additional information.

Parameters:
- **xStreamBuffer** -- The handle of the stream buffer to which data was written.
- **pxHigherPriorityTaskWoken** -- *pxHigherPriorityTaskWoken should be initialised to pdFALSE before it is passed into xStreamBufferSendCompletedFromISR(). If calling xStreamBufferSendCompletedFromISR() removes a task from the Blocked state, and the task has a priority above the priority of the currently running task, then *pxHigherPriorityTaskWoken will get set to pdTRUE indicating that a context switch should be performed before exiting the ISR.

Returns: If a task was removed from the Blocked state then pdTRUE is returned. Otherwise pdFALSE is returned.

BaseType_t **xStreamBufferReceiveCompletedFromISR**(StreamBufferHandle_t xStreamBuffer, BaseType_t *pxHigherPriorityTaskWoken)

For advanced users only.

The sbRECEIVE_COMPLETED() macro is called from within the FreeRTOS APIs when data is read out of a message buffer or stream buffer. If there was a task that was blocked on the message or stream buffer waiting for data to arrive then the sbRECEIVE_COMPLETED() macro sends a notification to the task to remove it from the Blocked state. xStreamBufferReceiveCompletedFromISR() does the same thing. It is provided to enable application writers to implement their own version of sbRECEIVE_COMPLETED(), and MUST NOT BE USED AT ANY OTHER TIME.

See the example implemented in FreeRTOS/Demo/Minimal/MessageBufferAMP.c for additional information.

Parameters:
- **xStreamBuffer** -- The handle of the stream buffer from which data was read.
- **pxHigherPriorityTaskWoken** -- *pxHigherPriorityTaskWoken should be initialised to pdFALSE before it is passed into xStreamBufferReceiveCompletedFromISR(). If calling xStreamBufferReceiveCompletedFromISR() removes a task from the Blocked state, and the task has a priority above the priority of the currently running task, then *pxHigherPriorityTaskWoken will get set to pdTRUE indicating that a context switch should be performed before exiting the ISR.

Returns: If a task was removed from the Blocked state then pdTRUE is returned. Otherwise pdFALSE is returned.

Macros

xStreamBufferCreateWithCallback(xBufferSizeBytes, xTriggerLevelBytes, pxSendCompletedCallback, pxReceiveCompletedCallback)

Creates a new stream buffer using dynamically allocated memory. See xStreamBufferCreateStatic() for a version that uses statically allocated memory (memory that is allocated at compile time).

configSUPPORT_DYNAMIC_ALLOCATION must be set to 1 or left undefined in FreeRTOSConfig.h for xStreamBufferCreate() to be available.

Example use:

```
void vAFunction( void )
{
StreamBufferHandle_t xStreamBuffer;
const size_t xStreamBufferSizeBytes = 100, xTriggerLevel = 10;

 // Create a stream buffer that can hold 100 bytes.  The memory used to hold
 // both the stream buffer structure and the data in the stream buffer is
 // allocated dynamically.
 xStreamBuffer = xStreamBufferCreate( xStreamBufferSizeBytes, xTriggerLevel );

 if( xStreamBuffer == NULL )
 {
// There was not enough heap memory space available to create the
// stream buffer.
 }
 else
 {
// The stream buffer was created successfully and can now be used.
 }
}
```

Parameters:

- **xBufferSizeBytes** -- The total number of bytes the stream buffer will be able to hold at any one time.
- **xTriggerLevelBytes** -- The number of bytes that must be in the stream buffer before a task that is blocked on the stream buffer to wait for data is moved out of the blocked state. For example, if a task is blocked on a read of an empty stream buffer that has a trigger level of 1 then the task will be unblocked when a single byte is written to the buffer or the task's block time expires. As another example, if a task is blocked on a read of an empty stream buffer that has a trigger level of 10 then the task will not be unblocked until the stream buffer contains at least 10 bytes or the task's block time expires. If a reading task's block time expires before the trigger level is reached then the task will still receive however many bytes are actually available. Setting a trigger level of 0 will result in a trigger level of 1 being used. It is not valid to specify a trigger level that is greater than the buffer size.

- **pxSendCompletedCallback** -- Callback invoked when number of bytes at least equal to trigger level is sent to the stream buffer. If the parameter is NULL, it will use the default implementation provided by sbSEND_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.
- **pxReceiveCompletedCallback** -- Callback invoked when more than zero bytes are read from a stream buffer. If the parameter is NULL, it will use the default implementation provided by sbRECEIVE_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.

Returns: If NULL is returned, then the stream buffer cannot be created because there is insufficient heap memory available for FreeRTOS to allocate the stream buffer data structures and storage area. A non-NULL value being returned indicates that the stream buffer has been created successfully - the returned value should be stored as the handle to the created stream buffer.

`xStreamBufferCreateStaticWithCallback`(xBufferSizeBytes, xTriggerLevelBytes, pucStreamBufferStorageArea, pxStaticStreamBuffer, pxSendCompletedCallback, pxReceiveCompletedCallback)

Creates a new stream buffer using statically allocated memory. See xStreamBufferCreate() for a version that uses dynamically allocated memory.

configSUPPORT_STATIC_ALLOCATION must be set to 1 in FreeRTOSConfig.h for xStreamBufferCreateStatic() to be available.

Example use:

```
// Used to dimension the array used to hold the streams.  The available space
// will actually be one less than this, so 999.
#define STORAGE_SIZE_BYTES 1000

// Defines the memory that will actually hold the streams within the stream
// buffer.
static uint8_t ucStorageBuffer[ STORAGE_SIZE_BYTES ];

// The variable used to hold the stream buffer structure.
  StaticStreamBuffer_t xStreamBufferStruct;

void MyFunction( void )
  {
  StreamBufferHandle_t xStreamBuffer;
const size_t xTriggerLevel = 1;

   xStreamBuffer = xStreamBufferCreateStatic( sizeof( ucStorageBuffer ),
                                              xTriggerLevel,
                                              ucStorageBuffer,
                                              &xStreamBufferStruct );

// As neither the pucStreamBufferStorageArea or pxStaticStreamBuffer
// parameters were NULL, xStreamBuffer will not be NULL, and can be used to
// reference the created stream buffer in other stream buffer API calls.

// Other code that uses the stream buffer can go here.
  }
```

Parameters:

- **xBufferSizeBytes** -- The size, in bytes, of the buffer pointed to by the pucStreamBufferStorageArea parameter.
- **xTriggerLevelBytes** -- The number of bytes that must be in the stream buffer before a task that is blocked on the stream buffer to wait for data is moved out of the blocked state. For example, if a task is blocked on a read of an empty stream buffer that has a trigger level of 1 then the task will be unblocked when a single byte is written to the buffer or the task's block time expires. As another example, if a task is blocked on a read of an empty stream buffer that has a trigger level of 10 then the task will not be unblocked until the stream buffer contains at least 10 bytes or the task's block time expires. If a reading task's block time expires before the trigger level is reached then the task will still receive however many bytes are actually available. Setting a trigger level of 0 will result in a trigger level of 1 being used. It is not valid to specify a trigger level that is greater than the buffer size.
- **pucStreamBufferStorageArea** -- Must point to a uint8_t array that is at least xBufferSizeBytes big. This is the array to which streams are copied when they are written to the stream buffer.
- **pxStaticStreamBuffer** -- Must point to a variable of type StaticStreamBuffer_t, which will be used to hold the stream buffer's data structure.

- **pxSendCompletedCallback** -- Callback invoked when number of bytes at least equal to trigger level is sent to the stream buffer. If the parameter is NULL, it will use the default implementation provided by sbSEND_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.
- **pxReceiveCompletedCallback** -- Callback invoked when more than zero bytes are read from a stream buffer. If the parameter is NULL, it will use the default implementation provided by sbRECEIVE_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.

Returns: If the stream buffer is created successfully then a handle to the created stream buffer is returned. If either pucStreamBufferStorageArea or pxStaticstreamBuffer are NULL then NULL is returned.

Type Definitions

typedef struct StreamBufferDef_t ***StreamBufferHandle_t**

typedef void (***StreamBufferCallbackFunction_t**)(StreamBufferHandle_t xStreamBuffer, BaseType_t xIsInsideISR, BaseType_t **const* pxHigherPriorityTaskWoken)

Type used as a stream buffer's optional callback.

Message Buffer API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/message_buffer.h
- This header file can be included with:

```
#include "freertos/message_buffer.h"
```

Macros

xMessageBufferCreateWithCallback(xBufferSizeBytes, pxSendCompletedCallback, pxReceiveCompletedCallback)

Creates a new message buffer using dynamically allocated memory. See xMessageBufferCreateStatic() for a version that uses statically allocated memory (memory that is allocated at compile time).

configSUPPORT_DYNAMIC_ALLOCATION must be set to 1 or left undefined in FreeRTOSConfig.h for xMessageBufferCreate() to be available.

Example use:

```
void vAFunction( void )
{
MessageBufferHandle_t xMessageBuffer;
const size_t xMessageBufferSizeBytes = 100;

 // Create a message buffer that can hold 100 bytes.  The memory used to hold
 // both the message buffer structure and the messages themselves is allocated
 // dynamically.  Each message added to the buffer consumes an additional 4
 // bytes which are used to hold the length of the message.
 xMessageBuffer = xMessageBufferCreate( xMessageBufferSizeBytes );

 if( xMessageBuffer == NULL )
 {
// There was not enough heap memory space available to create the
// message buffer.
 }
 else
 {
// The message buffer was created successfully and can now be used.
 }
```

Parameters:

- **xBufferSizeBytes** -- The total number of bytes (not messages) the message buffer will be able to hold at any one time. When a message is written to the message buffer an additional sizeof(size_t) bytes are also written to store the message's length. sizeof(size_t) is typically 4 bytes on a 32-bit architecture, so on most 32-bit architectures a 10 byte message will take up 14 bytes of message buffer space.
- **pxSendCompletedCallback** -- Callback invoked when a send operation to the message buffer is complete. If the parameter is NULL or xMessageBufferCreate() is called without the parameter, then it will use the default implementation provided by sbSEND_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.
- **pxReceiveCompletedCallback** -- Callback invoked when a receive operation from the message buffer is complete. If the parameter is NULL or xMessageBufferCreate() is called without the parameter, it will use the default implementation provided by sbRECEIVE_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.

Returns: If NULL is returned, then the message buffer cannot be created because there is insufficient heap memory available for FreeRTOS to allocate the message buffer data structures and storage area. A non-

NULL value being returned indicates that the message buffer has been created successfully - the returned value should be stored as the handle to the created message buffer.

---

xMessageBufferCreateStaticWithCallback(xBufferSizeBytes, pucMessageBufferStorageArea, pxStaticMessageBuffer, pxSendCompletedCallback, pxReceiveCompletedCallback)

Creates a new message buffer using statically allocated memory. See xMessageBufferCreate() for a version that uses dynamically allocated memory.

Example use:

```
// Used to dimension the array used to hold the messages.  The available space
// will actually be one less than this, so 999.
#define STORAGE_SIZE_BYTES 1000

// Defines the memory that will actually hold the messages within the message
// buffer.
static uint8_t ucStorageBuffer[ STORAGE_SIZE_BYTES ];

// The variable used to hold the message buffer structure.
StaticMessageBuffer_t xMessageBufferStruct;

void MyFunction( void )
{
MessageBufferHandle_t xMessageBuffer;

 xMessageBuffer = xMessageBufferCreateStatic( sizeof( ucStorageBuffer ),
                                              ucStorageBuffer,
                                              &xMessageBufferStruct );

 // As neither the pucMessageBufferStorageArea or pxStaticMessageBuffer
 // parameters were NULL, xMessageBuffer will not be NULL, and can be used to
 // reference the created message buffer in other message buffer API calls.

 // Other code that uses the message buffer can go here.
}
```

Parameters:

- **xBufferSizeBytes** -- The size, in bytes, of the buffer pointed to by the pucMessageBufferStorageArea parameter. When a message is written to the message buffer an additional sizeof(size_t) bytes are also written to store the message's length. sizeof(size_t) is typically 4 bytes on a 32-bit architecture, so on most 32-bit architecture a 10 byte message will take up 14 bytes of message buffer space. The maximum number of bytes that can be stored in the message buffer is actually (xBufferSizeBytes - 1).
- **pucMessageBufferStorageArea** -- Must point to a uint8_t array that is at least xBufferSizeBytes big. This is the array to which messages are copied when they are written to the message buffer.

- **pxStaticMessageBuffer** -- Must point to a variable of type StaticMessageBuffer_t, which will be used to hold the message buffer's data structure.
- **pxSendCompletedCallback** -- Callback invoked when a new message is sent to the message buffer. If the parameter is NULL or xMessageBufferCreate() is called without the parameter, then it will use the default implementation provided by sbSEND_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.
- **pxReceiveCompletedCallback** -- Callback invoked when a message is read from a message buffer. If the parameter is NULL or xMessageBufferCreate() is called without the parameter, it will use the default implementation provided by sbRECEIVE_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.

Returns: If the message buffer is created successfully then a handle to the created message buffer is returned. If either pucMessageBufferStorageArea or pxStaticmessageBuffer are NULL then NULL is returned.

`xMessageBufferGetStaticBuffers`(xMessageBuffer, ppucMessageBufferStorageArea, ppxStaticMessageBuffer)

Retrieve pointers to a statically created message buffer's data structure buffer and storage area buffer. These are the same buffers that are supplied at the time of creation.

Parameters:
- **xMessageBuffer** -- The message buffer for which to retrieve the buffers.
- **ppucMessageBufferStorageArea** -- Used to return a pointer to the message buffer's storage area buffer.
- **ppxStaticMessageBuffer** -- Used to return a pointer to the message buffer's data structure buffer.

Returns: pdTRUE if buffers were retrieved, pdFALSE otherwise.

`xMessageBufferSend`(xMessageBuffer, pvTxData, xDataLengthBytes, xTicksToWait)

Sends a discrete message to the message buffer. The message can be any length that fits within the buffer's free space, and is copied into the buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to

be multiple different writers then the application writer must place each call to a writing API function (such as xMessageBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers then the application writer must place each call to a reading API function (such as xMessageBufferRead()) inside a critical section and set the receive block time to 0.

Use xMessageBufferSend() to write to a message buffer from a task. Use xMessageBufferSendFromISR() to write to a message buffer from an interrupt service routine (ISR).

Example use:

```
void vAFunction( MessageBufferHandle_t xMessageBuffer )
{
size_t xBytesSent;
uint8_t ucArrayToSend[] = { 0, 1, 2, 3 };
char *pcStringToSend = "String to send";
const TickType_t x100ms = pdMS_TO_TICKS( 100 );

 // Send an array to the message buffer, blocking for a maximum of 100ms to
 // wait for enough space to be available in the message buffer.
 xBytesSent = xMessageBufferSend( xMessageBuffer, ( void * ) ucArrayToSend, sizeof(
ucArrayToSend ), x100ms );

 if( xBytesSent != sizeof( ucArrayToSend ) )
 {
// The call to xMessageBufferSend() times out before there was enough
// space in the buffer for the data to be written.
 }

 // Send the string to the message buffer.  Return immediately if there is
 // not enough space in the buffer.
 xBytesSent = xMessageBufferSend( xMessageBuffer, ( void * ) pcStringToSend,
strlen( pcStringToSend ), 0 );

 if( xBytesSent != strlen( pcStringToSend ) )
 {
// The string could not be added to the message buffer because there was
// not enough free space in the buffer.
 }
}
```

Parameters:

- **xMessageBuffer** -- The handle of the message buffer to which a message is being sent.
- **pvTxData** -- A pointer to the message that is to be copied into the message buffer.
- **xDataLengthBytes** -- The length of the message. That is, the number of bytes to copy from pvTxData into the message buffer. When a message is written to the message buffer an additional sizeof(size_t) bytes are also written to store the message's length. sizeof(size_t) is typically 4 bytes on a 32-bit architecture, so on

most 32-bit architecture setting xDataLengthBytes to 20 will reduce the free space in the message buffer by 24 bytes (20 bytes of message data and 4 bytes to hold the message length).

- **xTicksToWait** -- The maximum amount of time the calling task should remain in the Blocked state to wait for enough space to become available in the message buffer, should the message buffer have insufficient space when xMessageBufferSend() is called. The calling task will never block if xTicksToWait is zero. The block time is specified in tick periods, so the absolute time it represents is dependent on the tick frequency. The macro pdMS_TO_TICKS() can be used to convert a time specified in milliseconds into a time specified in ticks. Setting xTicksToWait to portMAX_DELAY will cause the task to wait indefinitely (without timing out), provided INCLUDE_vTaskSuspend is set to 1 in FreeRTOSConfig.h. Tasks do not use any CPU time when they are in the Blocked state.

Returns: The number of bytes written to the message buffer. If the call to xMessageBufferSend() times out before there was enough space to write the message into the message buffer then zero is returned. If the call did not time out then xDataLengthBytes is returned.

`xMessageBufferSendFromISR`(xMessageBuffer, pvTxData, xDataLengthBytes, pxHigherPriorityTaskWoken)

Interrupt safe version of the API function that sends a discrete message to the message buffer. The message can be any length that fits within the buffer's free space, and is copied into the buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to be multiple different writers then the application writer must place each call to a writing API function (such as xMessageBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers then the application writer must place each call to a reading API function (such as xMessageBufferRead()) inside a critical section and set the receive block time to 0.

Use xMessageBufferSend() to write to a message buffer from a task. Use xMessageBufferSendFromISR() to write to a message buffer from an interrupt service routine (ISR).

Example use:

```
// A message buffer that has already been created.
MessageBufferHandle_t xMessageBuffer;

void vAnInterruptServiceRoutine( void )
{
size_t xBytesSent;
char *pcStringToSend = "String to send";
BaseType_t xHigherPriorityTaskWoken = pdFALSE; // Initialised to pdFALSE.

 // Attempt to send the string to the message buffer.
 xBytesSent = xMessageBufferSendFromISR( xMessageBuffer,
                                         ( void * ) pcStringToSend,
                                         strlen( pcStringToSend ),
                                         &xHigherPriorityTaskWoken );

 if( xBytesSent != strlen( pcStringToSend ) )
 {
// The string could not be added to the message buffer because there was
// not enough free space in the buffer.
 }

 // If xHigherPriorityTaskWoken was set to pdTRUE inside
 // xMessageBufferSendFromISR() then a task that has a priority above the
 // priority of the currently executing task was unblocked and a context
 // switch should be performed to ensure the ISR returns to the unblocked
 // task.  In most FreeRTOS ports this is done by simply passing
 // xHigherPriorityTaskWoken into portYIELD_FROM_ISR(), which will test the
 // variables value, and perform the context switch if necessary.  Check the
 // documentation for the port in use for port specific instructions.
 portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

Parameters:

- **xMessageBuffer** -- The handle of the message buffer to which a message is being sent.
- **pvTxData** -- A pointer to the message that is to be copied into the message buffer.
- **xDataLengthBytes** -- The length of the message. That is, the number of bytes to copy from pvTxData into the message buffer. When a message is written to the message buffer an additional sizeof(size_t) bytes are also written to store the message's length. sizeof(size_t) is typically 4 bytes on a 32-bit architecture, so on most 32-bit architecture setting xDataLengthBytes to 20 will reduce the free space in the message buffer by 24 bytes (20 bytes of message data and 4 bytes to hold the message length).
- **pxHigherPriorityTaskWoken** -- It is possible that a message buffer will have a task blocked on it waiting for data. Calling xMessageBufferSendFromISR() can make data available, and so cause a task that was waiting for data to leave the Blocked state. If calling xMessageBufferSendFromISR() causes a task to leave the Blocked state, and the unblocked task has a priority higher than the currently executing task (the task that was interrupted), then, internally, xMessageBufferSendFromISR() will set

*pxHigherPriorityTaskWoken to pdTRUE. If xMessageBufferSendFromISR() sets this value to pdTRUE, then normally a context switch should be performed before the interrupt is exited. This will ensure that the interrupt returns directly to the highest priority Ready state task. *pxHigherPriorityTaskWoken should be set to pdFALSE before it is passed into the function. See the code example below for an example.

Returns: The number of bytes actually written to the message buffer. If the message buffer didn't have enough free space for the message to be stored then 0 is returned, otherwise xDataLengthBytes is returned.

`xMessageBufferReceive`(xMessageBuffer, pvRxData, xBufferLengthBytes, xTicksToWait)

Receives a discrete message from a message buffer. Messages can be of variable length and are copied out of the buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to be multiple different writers then the application writer must place each call to a writing API function (such as xMessageBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers then the application writer must place each call to a reading API function (such as xMessageBufferRead()) inside a critical section and set the receive block time to 0.

Use xMessageBufferReceive() to read from a message buffer from a task. Use xMessageBufferReceiveFromISR() to read from a message buffer from an interrupt service routine (ISR).

Example use:

```
void vAFunction( MessageBuffer_t xMessageBuffer )
{
uint8_t ucRxData[ 20 ];
size_t xReceivedBytes;
const TickType_t xBlockTime = pdMS_TO_TICKS( 20 );

 // Receive the next message from the message buffer.  Wait in the Blocked
 // state (so not using any CPU processing time) for a maximum of 100ms for
 // a message to become available.
 xReceivedBytes = xMessageBufferReceive( xMessageBuffer,
                                         ( void * ) ucRxData,
sizeof( ucRxData ),
                                         xBlockTime );

 if( xReceivedBytes > 0 )
 {
// A ucRxData contains a message that is xReceivedBytes long.  Process
// the message here....
 }
}
```

Parameters:

- **xMessageBuffer** -- The handle of the message buffer from which a message is being received.
- **pvRxData** -- A pointer to the buffer into which the received message is to be copied.
- **xBufferLengthBytes** -- The length of the buffer pointed to by the pvRxData parameter. This sets the maximum length of the message that can be received. If xBufferLengthBytes is too small to hold the next message then the message will be left in the message buffer and 0 will be returned.
- **xTicksToWait** -- The maximum amount of time the task should remain in the Blocked state to wait for a message, should the message buffer be empty. xMessageBufferReceive() will return immediately if xTicksToWait is zero and the message buffer is empty. The block time is specified in tick periods, so the absolute time it represents is dependent on the tick frequency. The macro pdMS_TO_TICKS() can be used to convert a time specified in milliseconds into a time specified in ticks. Setting xTicksToWait to portMAX_DELAY will cause the task to wait indefinitely (without timing out), provided INCLUDE_vTaskSuspend is set to 1 in FreeRTOSConfig.h. Tasks do not use any CPU time when they are in the Blocked state.

Returns: The length, in bytes, of the message read from the message buffer, if any. If xMessageBufferReceive() times out before a message became available then zero is returned. If the length of the message is greater than xBufferLengthBytes then the message will be left in the message buffer and zero is returned.

xMessageBufferReceiveFromISR(xMessageBuffer, pvRxData, xBufferLengthBytes, pxHigherPriorityTaskWoken)

An interrupt safe version of the API function that receives a discrete message from a message buffer. Messages can be of variable length and are copied out of the buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to be multiple different writers then the application writer must place each call to a writing API function (such as xMessageBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers then the application writer must place each call to a reading API function (such as xMessageBufferRead()) inside a critical section and set the receive block time to 0.

Use xMessageBufferReceive() to read from a message buffer from a task. Use xMessageBufferReceiveFromISR() to read from a message buffer from an interrupt service routine (ISR).

Example use:

```
// A message buffer that has already been created.
MessageBuffer_t xMessageBuffer;

void vAnInterruptServiceRoutine( void )
{
uint8_t ucRxData[ 20 ];
size_t xReceivedBytes;
BaseType_t xHigherPriorityTaskWoken = pdFALSE;  // Initialised to pdFALSE.

 // Receive the next message from the message buffer.
 xReceivedBytes = xMessageBufferReceiveFromISR( xMessageBuffer,
                                                ( void * ) ucRxData,
sizeof( ucRxData ),
                                                &xHigherPriorityTaskWoken );

 if( xReceivedBytes > 0 )
 {
// A ucRxData contains a message that is xReceivedBytes long.  Process
// the message here....
 }

 // If xHigherPriorityTaskWoken was set to pdTRUE inside
 // xMessageBufferReceiveFromISR() then a task that has a priority above the
 // priority of the currently executing task was unblocked and a context
 // switch should be performed to ensure the ISR returns to the unblocked
 // task.  In most FreeRTOS ports this is done by simply passing
 // xHigherPriorityTaskWoken into portYIELD_FROM_ISR(), which will test the
 // variables value, and perform the context switch if necessary.  Check the
 // documentation for the port in use for port specific instructions.
 portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

Parameters:

- **xMessageBuffer** -- The handle of the message buffer from which a message is being received.
- **pvRxData** -- A pointer to the buffer into which the received message is to be copied.
- **xBufferLengthBytes** -- The length of the buffer pointed to by the pvRxData parameter. This sets the maximum length of the message that can be received. If xBufferLengthBytes is too small to hold the next message then the message will be left in the message buffer and 0 will be returned.
- **pxHigherPriorityTaskWoken** -- It is possible that a message buffer will have a task blocked on it waiting for space to become available. Calling xMessageBufferReceiveFromISR() can make space available, and so cause a task that is waiting for space to leave the Blocked state. If calling xMessageBufferReceiveFromISR() causes a task to leave the Blocked state, and the unblocked task has a priority higher than the currently executing task (the task that was interrupted), then, internally, xMessageBufferReceiveFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE. If xMessageBufferReceiveFromISR() sets this value to pdTRUE, then normally a context switch should be performed before the interrupt is exited. That will ensure the interrupt returns directly to the highest priority Ready state task. *pxHigherPriorityTaskWoken should be set to pdFALSE before it is passed into the function. See the code example below for an example.

Returns: The length, in bytes, of the message read from the message buffer, if any.

`vMessageBufferDelete`(xMessageBuffer)

Deletes a message buffer that was previously created using a call to xMessageBufferCreate() or xMessageBufferCreateStatic(). If the message buffer was created using dynamic memory (that is, by xMessageBufferCreate()), then the allocated memory is freed.

A message buffer handle must not be used after the message buffer has been deleted.

Parameters:

- **xMessageBuffer** -- The handle of the message buffer to be deleted.

`xMessageBufferIsFull`(xMessageBuffer)

Tests to see if a message buffer is full. A message buffer is full if it cannot accept any more messages, of any size, until space is made available by a message being removed from the message buffer.

Parameters:

- **xMessageBuffer** -- The handle of the message buffer being queried.

Returns: If the message buffer referenced by xMessageBuffer is full then pdTRUE is returned. Otherwise pdFALSE is returned.

xMessageBufferIsEmpty(xMessageBuffer)

Tests to see if a message buffer is empty (does not contain any messages).

Parameters:
- **xMessageBuffer** -- The handle of the message buffer being queried.

Returns: If the message buffer referenced by xMessageBuffer is empty then pdTRUE is returned. Otherwise pdFALSE is returned.

xMessageBufferReset(xMessageBuffer)

Resets a message buffer to its initial empty state, discarding any message it contained.

A message buffer can only be reset if there are no tasks blocked on it.

Parameters:
- **xMessageBuffer** -- The handle of the message buffer being reset.

Returns: If the message buffer was reset then pdPASS is returned. If the message buffer could not be reset because either there was a task blocked on the message queue to wait for space to become available, or to wait for a a message to be available, then pdFAIL is returned.

xMessageBufferSpaceAvailable(xMessageBuffer)

message_buffer.h

```
size_t xMessageBufferSpaceAvailable( MessageBufferHandle_t xMessageBuffer );
```

Returns the number of bytes of free space in the message buffer.

Parameters:
- **xMessageBuffer** -- The handle of the message buffer being queried.

Returns: The number of bytes that can be written to the message buffer before the message buffer would be full. When a message is written to the message buffer an additional sizeof(size_t) bytes are also written to store the message's length. sizeof(size_t) is typically 4 bytes on a 32-bit architecture, so if xMessageBufferSpacesAvailable() returns 10, then the size of the largest message that can be written to the message buffer is 6 bytes.

xMessageBufferSpacesAvailable(xMessageBuffer)

xMessageBufferNextLengthBytes(xMessageBuffer)

Returns the length (in bytes) of the next message in a message buffer. Useful if xMessageBufferReceive() returned 0 because the size of the buffer passed into xMessageBufferReceive() was too small to hold the next message.

Parameters:
- **xMessageBuffer** -- The handle of the message buffer being queried.

Returns: The length (in bytes) of the next message in the message buffer, or 0 if the message buffer is empty.

`xMessageBufferSendCompletedFromISR`(xMessageBuffer, pxHigherPriorityTaskWoken)

For advanced users only.

The sbSEND_COMPLETED() macro is called from within the FreeRTOS APIs when data is sent to a message buffer or stream buffer. If there was a task that was blocked on the message or stream buffer waiting for data to arrive then the sbSEND_COMPLETED() macro sends a notification to the task to remove it from the Blocked state. xMessageBufferSendCompletedFromISR() does the same thing. It is provided to enable application writers to implement their own version of sbSEND_COMPLETED(), and MUST NOT BE USED AT ANY OTHER TIME.

See the example implemented in FreeRTOS/Demo/Minimal/MessageBufferAMP.c for additional information.

Parameters:

- **xMessageBuffer** -- The handle of the stream buffer to which data was written.
- **pxHigherPriorityTaskWoken** -- *pxHigherPriorityTaskWoken should be initialised to pdFALSE before it is passed into xMessageBufferSendCompletedFromISR(). If calling xMessageBufferSendCompletedFromISR() removes a task from the Blocked state, and the task has a priority above the priority of the currently running task, then *pxHigherPriorityTaskWoken will get set to pdTRUE indicating that a context switch should be performed before exiting the ISR.

Returns: If a task was removed from the Blocked state then pdTRUE is returned. Otherwise pdFALSE is returned.

`xMessageBufferReceiveCompletedFromISR`(xMessageBuffer, pxHigherPriorityTaskWoken)

For advanced users only.

The sbRECEIVE_COMPLETED() macro is called from within the FreeRTOS APIs when data is read out of a message buffer or stream buffer. If there was a task that was blocked on the message or stream buffer waiting for data to arrive then the sbRECEIVE_COMPLETED() macro sends a notification to the task to remove it from the Blocked state. xMessageBufferReceiveCompletedFromISR() does the same thing. It is provided to enable application writers to implement their own version of sbRECEIVE_COMPLETED(), and MUST NOT BE USED AT ANY OTHER TIME.

See the example implemented in FreeRTOS/Demo/Minimal/MessageBufferAMP.c for additional information.

Parameters:

- **xMessageBuffer** -- The handle of the stream buffer from which data was read.

- **pxHigherPriorityTaskWoken** -- *pxHigherPriorityTaskWoken should be initialised to pdFALSE before it is passed into xMessageBufferReceiveCompletedFromISR(). If calling xMessageBufferReceiveCompletedFromISR() removes a task from the Blocked state, and the task has a priority above the priority of the currently running task, then *pxHigherPriorityTaskWoken will get set to pdTRUE indicating that a context switch should be performed before exiting the ISR.

Returns: If a task was removed from the Blocked state then pdTRUE is returned. Otherwise pdFALSE is returned.

Type Definitions

typedef StreamBufferHandle_t **`MessageBufferHandle_t`**

Type by which message buffers are referenced. For example, a call to xMessageBufferCreate() returns an MessageBufferHandle_t variable that can then be used as a parameter to xMessageBufferSend(), xMessageBufferReceive(), etc. Message buffer is essentially built as a stream buffer hence its handle is also set to same type as a stream buffer handle.

Was this page helpful?